

牛客面经

- java 版本变更

- Java 8

- 引入 Lambda 表达式和函数式接口（Functional Interface）：允许 (参数) -> 表达式/语句块的语法形式。
 - 引入流（Stream）API + 函数式接口：简化集合操作。
 - 默认方法（default methods）和静态方法在接口中。
 - 引入 java.time（新日期时间 API）等

- Java 9 Java 16

- (Java 9) 模块系统（Project Jigsaw）——模块化的语言/平台机制。
 - (Java 10) 局部变量类型推断（var 关键字）——简化局部变量声明
 - (Java 14 16) 记录类（record）——用于定义不可变数据载体，更简洁。在 Java 16 成为正式特性。
 - (Java 16) instanceof 的模式匹配（Pattern Matching for instanceof）——简化类型检查与转换。
 - (Java 17) 密封类（sealed classes & interfaces）——控制哪些子类/实现可以扩展。

- Java 17 (LTS) 与 Java 21 (LTS) 及之后

- Java 21 中正式/准备特性的语言增强包括：
 - 模式匹配 for switch（Pattern Matching for switch）——可以在 switch 中使用类型 / 守卫（when）模式。
 - 记录模式（Record Patterns）——在模式匹配中解构 record 类实例。
 - 字符串模板（String Templates，预览特性）——增强字符串字面量形式。
 - 未命名类、实例 main 方法（Unnamed Classes and Instance Main Methods，预览）——简化“Hello World”结构。

- Java 中一个对象的生命周期

- NEW 创建对象

- 使用阶段：方法调用，参数传递
 - 不再被引用
 - 垃圾回收标记
 - 垃圾回收

- 什么是泛型，泛型擦除

- 泛型就是让类、方法、接口在编译期就能够检查类型，而不是等运行时才报错。
 - 以前用 Object，装进去是啥不知道，取出来还要强转；泛型就是“给容器贴上类型标签”，告诉编译器里面放的是什么。泛型本质是编译期的类型约束。
 - Java 的泛型是“伪泛型”。Java 使用的是 Type Erasure（类型擦除）技术。这意味着：
 - 泛型只在编译期有效
 - 运行期泛型信息被擦除（删除）
 - JVM 根本不知道什么是 List 或 List
 - 最终在运行时，所有泛型都变成了原始类型（raw type）

```
List<String> a = new ArrayList<>();  
List<Integer> b = new ArrayList<>();
```

```
System.out.println(a.getClass() == b.getClass());
```

- 输出 true 说明 JVM 看不到泛型类型。

- 为什么要泛型擦除？

- 因为 Java 是在 Java5 才加入泛型的，而 JVM 早就存在。

- 为了保持向下兼容性，不能修改 JVM 的 class 格式结构，所以：
 - 泛型只保留在编译期
 - 编译后擦除成普通类型
 - JVM 还是用原来的 class 文件格式运行
- ▶ 泛型擦除带来的问题
 - 不能使用基本类型
 - 不能获取泛型参数类型
 - 不能 new T()
 - 不能创建泛型数组
 - 方法签名冲突
- ▶ 如何绕过泛型擦除的限制？
 - 使用反射的 ParameterizedType
 - 虽然运行时实例没有泛型，但字段的泛型信息保留在 .class 文件中。
 - 使用传入的 Class 对象
- ▶ 一个面试最喜欢的例子：为什么不能重载泛型方法
 - 方法签名冲突
- ▶ 最常见的泛型通配符总结
 - ? —— 不关心类型
 - ? extends Number —— 上界通配符（读）
 - 只读不让写
 - ? super Integer —— 下界通配符（写）
 - 可写入 Integer、其子类。
- 说说反射
 - ▶ 反射就是在运行时动态地获取类的信息，并操作类的属性、方法、构造方法、注解等能力。
 - ▶ 反射能做什么？
 - 获取类的完整结构
 - 包括：
 - ▶ 类名、父类、接口
 - ▶ 所有字段（private 也能拿到）
 - ▶ 所有方法（private 也能调用）
 - ▶ 构造方法
 - ▶ 注解
 - 动态创建对象
 - 不需要 new 用户()，可以通过名字来创建：
 - 动态调用方法
 - 方法名可以写成字符串
 - 访问/修改 private 字段
 - 突破访问权限（Java 安全机制允许）
 - 配合注解实现框架功能
 - ▶ 为什么要反射？
 - 让代码在运行时才决定行为，提供高度灵活性。
 - 例如：Spring IOC：创建、注入 Bean 全靠反射，MyBatis：将 SQL 结果映射到对象
 - Java 项目流程分三步：
 - ▶ 编译（javac）
 - ▶ 包括 Spring Boot 依赖一起打成一个 fat jar。
 - ▶ 运行
 - ▶ 反射的缺点

- 性能慢（但 JVM 已做很多优化）
 - 因为反射需要：查找类信息，检查权限，方法调用需要动态分派
 - 但现代 JVM 对 `Method.invoke()` 有优化（如 `LambdaMetafactory`）。
 - 破坏封装性
 - 可以访问 `private` 字段，不符合 OOP 设计原则。
 - `LambdaMetafactory` 是什么？
 - `LambdaMetafactory` 是 JVM 提供的“动态生成函数对象”的工厂，让 Java 的 lambda 表达式在运行时被优化成接近普通方法调用的速度。
 - lambda 看似像语法糖，但底层不是匿名类，而是 JVM 动态生成一个 `invokedynamic` 调用点，再通过 `LambdaMetafactory` 给你生成一个高效的函数对象。替换掉传统反射的 `Method.invoke()`。
 - 为什么要 `LambdaMetafactory`？
 - Java 8 前，如果你想：用反射调用方法或把方法当函数传递基本都靠 `Method.invoke()`
 - 但 `Method.invoke()` 速度慢，因为
 - 要做权限检查
 - 参数要封装到 `Object[]`
 - 结果要拆箱/装箱
 - 调用无法内联
 - 于是 Java 8 引入了更底层的优化机制：`MethodHandle` + `LambdaMetafactory` + `invokedynamic`
 - `LambdaMetafactory` 的核心作用
 - 为 lambda 生成“隐藏类”实现，隐藏类（Hidden Class）不会出现在 `classpath` / `ClassLoader` 中，专门为动态代码生成。
 - 例如：`Runnable r = () -> System.out.println("Hi");`
 - 这个类的 `run()` 方法会直接内联调用 `System.out.println`
 - 用 `MethodHandle` 替代反射 `Method.invoke`
 - 例如你想动态调用：`public void foo(String name)`
 - 旧反射做法：`method.invoke(obj, "Morty");`
 - 新优化做法：


```
MethodHandle mh = lookup.findVirtual(User.class, "foo",
MethodType.methodType(void.class, String.class));
mh.invokeExact(obj, "Morty");
```
 - 如果你用 `LambdaMetafactory` 再包一层，能直接生成：


```
interface FooInvoker {
void call(User u, String arg);
}
```

 - 执行速度接近普通函数调用。
 - 让 JVM 能“内联”动态调用
- JVM 是如何优化反射的？
 - 用 `MethodHandle` 代替 `Method.invoke()`
 - 可直接调用 `native-level` 指令
 - 基于签名（`MethodType`）进行静态类型检查
 - 性能接近普通方法调用
 - 不需要 `Object[]` 封装参数
 - 不需要太多权限检查
 - `invokedynamic` 指令
 - `invokedynamic` 是 JVM 支持的“动态调用点”，优势

- 第一次调用建立 bootstrap call site
 - 后续调用绑定到已确定的 MethodHandle
 - 可内联优化
 - 调用成本很低
- LambdaMetafactory 动态生成调用类
 - JVM 会动态生成一个“隐藏类”，内部是直接方法调用或者直接绑定 MethodHandle：
 - 无反射开销
 - 无参数封装
 - 无安全检查
 - JVM 能 inline
 - 还能逃逸分析
- Hidden Class 进一步提高效率
 - 专用于运行时生成类
 - 不污染 ClassLoader
 - 可以被 GC 回收
 - 还能绑定 MethodHandle 实现极快调用
- Reflection 使用缓存和替代路径
 - 第一次反射调用慢，之后会使用快速路径。
- 逃逸分析是什么
 - JVM 在 JIT 编译时分析一个对象是否会“逃出”当前作用域，根据结果决定是否要把这个对象放在堆上。
 - 也就是说，JVM 会判断：
 - 这个对象只在方法内部使用？
 - 会不会被返回、赋值给全局变量、多线程访问？
 - 是否安全地优化？
 - 如果一个对象不会“逃逸”，那 JVM 就可以做大量优化。
 - 逃逸分析能优化什么？
 - 栈上分配 (Stack Allocation)
 - 对象本来要 new 在堆上，但如果没有逃逸，JVM 可以直接把它放在栈上。
 - 不用走 GC
 - 分配/回收都更快
 - 标量替换 (Scalar Replacement)
 - 如果一个对象不会逃逸，JVM 可以把对象拆开为多个局部变量，甚至不创建对象。
 - 例如：

```
class Point {
    int x, y;
}

void foo() {
    Point p = new Point();
    p.x = 1;
    p.y = 2;
    int r = p.x + p.y;
}
```

- JIT 会优化成

```
int x = 1;
int y = 2;
int r = x + y;
```

- Point 对象会被完全消除，根本不会创建。
- 消除同步
 - 如果一个对象不会被多个线程访问，那么同步是无意义的。

```
public void useStringBuffer() {
    StringBuffer sb = new StringBuffer(); // 线程安全类，但未逃逸
    sb.append("A"); // 原本需要锁
    sb.append("B");
}
```

- 如果 sb 明确不会逃逸，多线程访问不可能发生，JVM 直接移除锁
- 逃逸分析的判断逻辑
 - JVM 会检查
 - 是否把对象存到全局变量？
 - 是否把 this 传出去？
 - 是否返回对象？
 - 是否赋值给外部引用？
 - 如何在 JVM 中启用逃逸分析？
 - 默认是开启的，但你可以手动查看优化
- JIT 编译是什么？
 - JIT 是 Java 虚拟机在运行时，把热点代码编译成机器码，直接让 CPU 执行的机制。
 - Java 本来是解释执行（Interpreter）——一行一行解释字节码 → 性能慢。当 JVM 发现某段代码“经常执行”时，它会：
 - 动态编译该段字节码为本地机器码（x86 指令等）
 - 直接让 CPU 执行，无需解释，性能超快
 - 为什么 Java 需要 JIT？
 - 因为 Java 原本是跨平台 + 字节码 + 解释执行。
 - 如果一直解释运行：
 - 每次执行都要翻译字节码 → 慢
 - HotSpot JVM 的 JIT 有两个编译器
 - Client 编译器
 - 快速编译、优化少，适合冷代码
 - Server 编译器
 - 深度优化，但编译慢，适合长期热点
 - JIT 是如何判断“热点代码”的？
 - JVM 会对每个方法和循环进行计数
 - 方法调用次数（hot method）
 - 循环次数（hot loop）
 - 当执行次数超过阈值后：
 - JVM 把该段代码判为热点
 - 触发 JIT 编译
 - JIT 能做哪些优化？
 - 方法内联


```
int a = user.getAge();
```

 - JVM 会把 getAge() 的字节码直接复制到调用处 → 消除调用成本。
 - 逃逸分析（放在 JIT 内做）
 - JIT 会判断对象是否逃逸，再进行：
 - 栈上分配

- 标量替换
 - 锁消除
- 这能让很多 new 对象“消失”。
- 循环优化
 - 循环展开
 - 循环无关代码外提
 - 循环常量折叠
- 消除无效代码
- 分支预测优化
- 寄存器分配优化
 - 把常用变量放在 CPU 寄存器中，而不是栈。
- 锁优化
- JIT 编译器什么时候触发？
 - 具体阈值由 JVM 参数控制。
 - 循环是 1 万次循环回边 → 触发 JIT
 - 方法是：2 千次调用 → 触发 C1，1 万次调用 → 触发 C2
- JIT 编译出来的机器码在哪里？
 - Code Cache（代码缓存）
 - 可以查看-XX:+PrintCompilation -XX:+PrintAssembly
- Stream 和传统的 for 循环有什么区别吗？
 - for 循环通常略快
 - 传统 for 循环是命令式、无额外抽象成本，编译器优化充分，因此在大部分情况下性能稍好。
 - Stream 是函数式编程抽象，中间操作是链式调用，会产生很多函数对象、lambda、迭代器层级，因此性能通常 ≈ for 循环略慢一些。
 - Stream 更简洁、更表达式化，可读性高
 - Parallel Stream 可以轻松利用多核，内部使用 ForkJoinPool 自动并行计算。
 - for 循环要并行，你得手写线程池/分片逻辑，成本很高
- 静态字段、静态方法、静态代码块、成员属性、成员方法、构造函数执行顺序
 - 类加载阶段
 - 加载静态字段
 - 执行静态代码块
 - 创建对象阶段（每次 new 时都会执行）
 - 分配对象内存，设置成员变量默认值
 - 初始化成员属性
 - 执行代码块
 - 执行构造函数
- 常见设计模式
 - 策略模式
 - 策略模式（Strategy Pattern）是软件设计中一种常用的行为型设计模式，主要用于在运行时动态选择算法或行为，而不是在编译时就确定。
 - 策略模式包含三个角色：
 - Strategy（策略接口）
 - ConcreteStrategy（具体策略类）
 - Context（上下文类）
 - 持有一个策略对象的引用，负责与客户端交互，屏蔽策略的具体实现

```

// 策略接口
public interface PayStrategy {
    void pay(double amount);
}

// 具体策略类
public class WeChatPay implements PayStrategy {
    @Override
    public void pay(double amount) {
        System.out.println("使用微信支付: " + amount + " 元");
    }
}

public class AliPay implements PayStrategy {
    @Override
    public void pay(double amount) {
        System.out.println("使用支付宝支付: " + amount + " 元");
    }
}

public class BankCardPay implements PayStrategy {
    @Override
    public void pay(double amount) {
        System.out.println("使用银行卡支付: " + amount + " 元");
    }
}

// 上下文类
public class PayContext {
    private PayStrategy strategy;

    // 可以在构造时注入, 也可以运行时切换
    public PayContext(PayStrategy strategy) {
        this.strategy = strategy;
    }

    public void setStrategy(PayStrategy strategy) {
        this.strategy = strategy;
    }

    public void executePay(double amount) {
        strategy.pay(amount);
    }
}

// 使用
public class Main {
    public static void main(String[] args) {
        PayContext context = new PayContext(new WeChatPay());
        context.executePay(100); // 使用微信支付

        context.setStrategy(new AliPay());
        context.executePay(200); // 改用支付宝支付
    }
}

```

- io 流涉及到哪些设计模式
 - 装饰器模式 (Decorator Pattern)
 - 动态地给对象增加功能, 而不改变其本身。

- `BufferedInputStream` 和 `DataInputStream` 都是对原始 `FileInputStream` 的装饰，增加了缓冲和数据读取功能。
- 适配器模式 (Adapter Pattern)
 - 目的：将一个接口转换成客户端期望的另一种接口。
 - `InputStreamReader` 和 `OutputStreamWriter`，将字节流 (`InputStream` / `OutputStream`) 适配为字符流 (`Reader` / `Writer`)。
- 策略模式 (Strategy Pattern)
 - 定义一系列算法，把它们封装起来，使它们可以互换。在 `BufferedReader` 中，通过传入不同的 `Reader`，可以灵活使用不同的数据源
- 模板方法模式 (Template Method Pattern)
 - 在父类中定义算法骨架，子类实现具体步骤。
- `String` 为什么不可变
 - 安全性需求是设计根源
 - Java 诞生之初，就希望它是安全的语言。`String` 经常被用于：类加载器 (`ClassLoader`) 中类名，网络连接 URL，数据库连接凭证
 - 如果 `String` 可变，一个恶意对象可以修改原本的字符串内容，导致安全漏洞。
 - 常量池优化是设计驱动
 - Java 引入了字符串常量池 (`String Pool`)，目的在于节省内存
 - 如果 `String` 可变：修改 `s1` 内容会破坏 `s2` 的值，常量池机制无法安全共享
 - `hashCode` 缓存与高效查找
 - `String` 常用作 `HashMap` 键，如果可变，`map` 找不到原来的键
 - 不可变保证了 `hashCode()` 一次计算就永远有效。
 - 线程安全是设计自然结果
 - 多线程共享同一个 `String` 不会出现竞态
- JVM 的字符串常量池说一说，运行时常量池说一说
 - 字符串常量池
 - 堆 (Heap) 中独立的 `StringTable` 结构
 - KEY 是字符串的内容 (即字符序列本身)，通过内容计算哈希值 (`hash = value[].hashCode()`)
 - VALUE 是堆中实际的 `java.lang.String` 对象引用 (oop 指针)
 - 专门缓存字符串对象的引用，实现字符串复用
 - 为什么 `StringTable` 不树化？
 - `StringTable` 的桶数量巨大，冲突率低。
 - 常量池字符串数量一般不大
 - 内部结构是 HotSpot 自己实现的，没必要复杂化
 - `StringTable` 冲突严重会怎样？
 - JVM 允许调整桶数量
 - 运行时常量池 (Runtime Constant Pool)
 - JVM 方法区 (元空间) 中
 - Class 文件常量池的运行时常量池表现形式 (加载后存放在方法区)
 - 字面量
 - 如整数常量 100、字符串字面量 "abc"。
 - 字符串字面量是编译期常量，所以放在运行时常量池用于类加载复用；而字符串哈希 code 存放的是 new 的 string 在堆
 - 哈希缓存，`String` 与对象实例的字符数组 `value` 强绑定，只能存在堆对象内部。
 - 符号引用
 - 类和接口的全限定名；

- 字段的名称和描述符;
- 方法的名称和描述符。
- 动态常量

- FINAL 作用

- final 修饰变量
 - 表示变量的值不能被修改
- 修饰引用类型
 - final 限定的是“引用”不能变，但对象内容仍可变。
- 修饰成员变量
 - 必须在：定义时赋值，或构造方法中赋值，否则会编译错误。
- final 修饰方法
 - 表示该方法不能被子类重写 (override)。
- final 修饰类
 - 表示该类不能被继承。

- 编译时多态和运行时多态

- 编译时多态 (静态多态)

```
class Printer {
    void print(int a) {
        System.out.println("打印整数: " + a);
    }

    void print(String s) {
        System.out.println("打印字符串: " + s);
    }
}

public class Demo {
    public static void main(String[] args) {
        Printer p = new Printer();
        p.print(10);        // 调用 print(int)
        p.print("Hello");   // 调用 print(String)
    }
}
```

- 同一个类中存在多个同名方法，但参数类型或数量不同，编译器会根据调用时的参数列表决定调用哪个方法。
- 调用哪个 print() 方法在编译阶段就已确定。

- 运行时多态 (动态多态)

```
class Animal {
    void speak() {
        System.out.println("动物叫");
    }
}

class Dog extends Animal {
    @Override
    void speak() {
        System.out.println("狗叫");
    }
}
```

```

public class Demo {
    public static void main(String[] args) {
        Animal a = new Dog(); // 父类引用指向子类对象
        a.speak();             // 调用 Dog 的 speak() (运行时确定)
    }
}

```

- 子类重写父类方法，通过父类引用指向子类对象时，调用的是子类重写后的方法。
- a.speak() 在编译期只知道有这个方法，但具体调用哪个版本在运行时才决定（通过虚方法表动态绑定到 Dog 的实现）。

- Java 里面有哪些基本数据类型，分别占多少个字节？

数据类型	英文关键字	占用字节数	占用位数	取值范围	
整型	byte	1 字节	8 位	-128 ~ 127	
整型	short	2 字节	16 位	-32768 ~ 32767	
整型	int	4 字节	32 位	-2 ³¹ ~ 2 ³¹ -1	
整型	long	8 字节	64 位	-2 ⁶³ ~ 2 ⁶³ -1	
浮点型	float	4 字节	32 位	单精度 (约 7 位小数)	
浮点型	double	8 字节	64 位	双精度 (约 15~16 位小数)	
字符型	char	2 字节	16 位	Unicode 字符 (0 ~ 65535)	
布尔型	boolean	理论上 1 bit, 实际实现依赖 JVM	通常按 1 字节处理	true / false	

- Java 语言规范没有明确规定 boolean 大小
- 在数组中：boolean 会被优化成 1 个字节，在对象中：可能被 JVM 填充对齐，占用 1 字节或更多

- Java 数据结构

- Collection

- List
 - ArrayList、LinkedList、Vector
- set
 - HashSet、LinkedHashSet、TreeSet
- Queue
 - LinkedList、PriorityQueue
- Deque
 - ArrayDeque、LinkedList

- Map

- HashMap、LinkedHashMap、TreeMap、Hashtable、ConcurrentHashMap

- Java 里面 LinkedList 和 ArrayList 的优势分别是什么，各自适用场景有哪些？

- ArrayList 的优势

- 随机访问快
- ArrayList 使用连续内存（底层是数组），空间利用率高，连续内存，数据紧凑，连续内存的元素在物理内存也通常是紧挨着的，CPU 一次从内存读取会顺便把旁边几个元素也

读入缓存行 (Cache Line)。而 LinkedList: 每个节点分散在内存不同地方, 每次访问 next 节点都可能要去内存找 → 慢得多

- 遍历快, 连续存储+顺序访问, 可预读优化。
- 读多写少的业务数据

▶ LinkedList 的优势

- 头部/中间插入删除快 — $O(1)$
- 不像 ArrayList 满了需要 grow() 扩容复制。
- 可实现队列、双端队列
- 大批元素频繁插入删除但不常查找, 任务调度器

• ArrayList 的扩容机制

▶ 什么时候扩容?

- 如果当前内部数组 elementData 已经满了 ($size == elementData.length$), 就会触发扩容

▶ 扩容后的新容量是多少

- 扩容为原来的 1.5 倍
- 1.5 得到一个平衡: 减少扩容次数, 同时避免浪费空间太多

▶ 扩容过程做了什么

- 分配一个新的更大的数组
- 把旧数组的数据拷贝过去
 - 使用的是 Arrays.copyOf() (底层是 System.arraycopy(), 属于本地方法, 速度极快)

▶ 第一次添加的扩容特殊情况

- 刚创建 ArrayList 时内部数组容量不是 10, 而是 空数组
- 第一次添加元素会扩容到 默认容量 10:

• ArrayList 的线程安全性如何, 若要在多线程场景下使用列表, 有哪些解决方案?

▶ ArrayList 不是线程安全的。

▶ 多线程下

- 使用 Collections.synchronizedList
 - 读写都加锁
- 使用 CopyOnWriteArrayList
 - 写时复制 (add/remove 时复制新数组) 写操作成本高 (复制数组 $O(n)$) 占内存
 - 写操作加锁 (ReentrantLock), 保证写操作互斥
 - 读操作无锁, 性能高
- 手动加锁

• 介绍一下堆的实现方式, 插入删除时间复杂度

▶ 堆是一种完全二叉树 (complete binary tree)

▶ 大根堆 (最大堆): 任意节点的值 $\geq$ 其子节点的值。

▶ 小根堆 (最小堆): 任意节点的值 $\leq$ 其子节点的值。

▶ 堆通常使用 数组 (array) 实现, 而不是链表或树节点结构。

▶ 假设数组下标从 1 开始

- 父节点下标 $i / 2$
- 左子节点下标 $2 * i$
- 右子节点下标 $2 * i + 1$

▶ 堆的主要操作

- 插入 (Insert)
 - 将新元素放到数组末尾;
 - 向上调整 (上浮 / sift-up) 直到满足堆性质。
 - 时间复杂度: $O(\log n)$

- 删除 (Delete) 通常删除的是堆顶元素 (最大/最小)。
 - 将堆顶与最后一个元素交换;
 - 删除最后一个元素;
 - 向下调整 (下沉 / sift-down) 以恢复堆性质。
 - $O(\log n)$
 - 建堆 (Heapify)
 - 逐个插入: $O(n \log n)$
 - 从最后一个非叶子节点开始下沉: $O(n)$
- On 建堆

四、示例 (建大根堆)

数组:

csharp

复制代码

[4, 6, 8, 5, 9]

目标: 建成 大根堆

完全二叉树:

markdown

复制代码

```
      4(1)
     /  \
    6(2) 8(3)
   /  \
  5(4) 9(5)
```

从最后一个非叶子节点 $i = n/2 = 2$ 开始:

从最后一个非叶子节点 $i = n/2 = 2$ 开始:

1 $i = 2$

- 当前节点 = 6
- 左子节点 = 5, 右子节点 = 9
- 最大子节点是 9 → 交换

csharp

复制代码

[4, 9, 8, 5, 6]

2 $i = 1$

- 当前节点 = 4
- 左子节点 = 9, 右子节点 = 8
- 最大子节点是 9 → 交换

csharp

复制代码

[9, 4, 8, 5, 6]



然后继续下沉：

- 新位置是 $i = 2$ ，子节点 5、6
- 最大子节点 6 → 交换

```
csharp
```

 复制代码

```
[9, 6, 8, 5, 4]
```

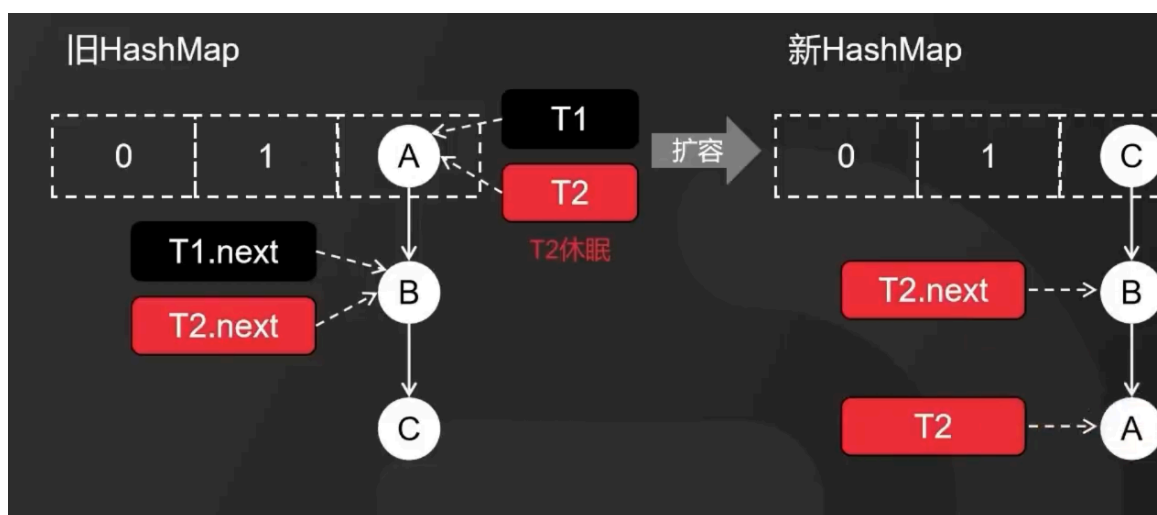
✅ 结果：[9, 6, 8, 5, 4] → 大根堆建成。

- Caffeine、Guava 等本地缓存与 Java 普通哈希方法的区别是什么？
 - HashMap / ConcurrentHashMap
 - 本质是一个纯粹的键值存储容器。
 - 不提供过期、淘汰、统计等缓存功能。
 - 内存占用完全由 JVM GC 管理。
 - hashmap 不是线程安全的，concurrenthashmap 线程安全
 - Caffeine / Guava Cache
 - 支持自动过期、大小限制、LRU/LFU 淘汰策略、异步刷新等。
 - 可以统计命中率、加载未命中数据（通过 CacheLoader）。
 - 支持 容量限制，超过限制时自动根据策略淘汰：
 - LRU (最近最少使用)
 - LFU (最不经常使用，Guava 支持)
 - 基于权重 (Caffeine)
 - 支持 时间过期：
 - 写入后过期 (expireAfterWrite)
 - 访问后过期 (expireAfterAccess)
 - 内部通常支持高并发访问 (Caffeine 基于 CAS + Segment + 写延迟队列)。
- 用线程池或多线程扫描缓存淘汰数据，会为系统带来多少额外开销？有其他更好的方法吗？
 - 额外开销主要有以下几个方面：
 - CPU 开销
 - 每次扫描需要访问缓存的内部数据结构（哈希表/链表/跳表等）。如果缓存很大，比如百万级或亿级条目，遍历整个缓存会占用大量 CPU 时间。
 - 内存开销
 - 多线程扫描时可能需要额外的临时数据结构（比如线程本地队列或标记数组），尤其在分段锁/分片缓存中，每个线程可能持有额外引用。
 - 锁/同步开销
 - 如果缓存内部数据结构需要加锁 (Segment 或 synchronized)，并发扫描会导致锁竞争。
 - 高并发情况下，可能会影响正常读写操作的吞吐量。
 - 上下文切换
 - 大量线程会带来上下文切换成本，尤其是线程数 > CPU 核数时。
 - 现代高性能缓存（如 Caffeine）都尽量避免全量扫描，使用“懒惰淘汰 + 局部维护”的策略：
 - 惰性/访问时淘汰 (Lazy Eviction)：只有在访问缓存时，才检查该条目是否过期或需要淘汰。
 - 优点：无需定期扫描，CPU 开销几乎为零
 - 缺点：不访问的缓存可能长时间占用空间，但一般配合容量限制可控。
 - 分段/分片淘汰 (Segmented Eviction)
 - 将缓存分成 N 段，每段独立管理 LRU/过期策略。

- 只扫描活跃段或随机抽样部分条目，不需要全量扫描。
- 写入触发淘汰（Write-Driven Eviction）
 - 当缓存容量接近阈值时，在写操作时顺便淘汰最老或最不常访问的条目。
- 近似 LRU / CLOCK 算法
 - 可以用环形数组或队列，每次淘汰时随机检查部分条目。
 - 不严格维护全局访问顺序，只维护近似顺序。
- 异步批量淘汰队列
 - 只有写入时将条目放入延迟队列，由后台线程异步清理，但不扫描整个缓存。
- java 中 hashMap 是干什么的，主要用途？插入，删除时间复杂度？
 - HashMap 是 Java 中最常用的 键值对（key-value）映射容器，属于 java.util 包。
 - 它的核心功能是：
 - 通过 key 快速查找、插入、删除 value
 - 不保证顺序（与插入顺序无关）
 - 允许 null 键和 null 值（但 Hashtable 不允许）
 - 主要用途
 - 快速查找
 - 比如缓存、索引、计数器（词频统计、IP 访问次数等）
 - map.get(key) 能在平均 $O(1)$ 时间内获取结果
 - 存储键值对映射关系
 - 如 `userId` $\rightarrow$ `UserInfo`
 - `username` $\rightarrow$ `password`
 - 实现集合类
 - HashSet 实际上就是基于 HashMap 实现的（key 存值，value 为固定常量）
 - 时间复杂度分析
 - 插入查找删除 平均 $O(1)$ 最差 $O(n)$
 - 当链表变成红黑树后，最坏情况会优化为： $O(\log n)$
- treeMap 和 hashMap 的区别和底层实现
 - 是否顺序存储：
 - treeMap 按 key 顺序存储
 - hashMap 不按顺序存储
 - key 可以为 null 吗？value 呢？
 - treeMap 的 key 不能为 null，因为需要排序
 - hashMap 可以为 null，但是只有一个，因为有覆盖问题
 - value 都可以为 null，且多个
 - 线程安全
 - 都不是线程安全的
 - 插入，删除时间复杂度
 - treeMap 是 $\log n$ ，因为基于红黑树
 - hashMap 平均 $O(1)$ ，最坏为 $O(n)$ ，碰撞严重退化成链表，但是链表长度大于 8 会变成红黑树（jdk1.8 以后）
 - 底层实现
 - treeMap 的底层实现是红黑树
 - hashMap 的底层实现是数组+链表或红黑树
- 为什么要红黑树不要 AVL 树
 - AVL 树严格平衡二叉树，维护代价高
 - 红黑树平衡策略相对宽松

- 那为什么还要链表
 - 红黑树占用更多内存，红黑树节点大小是普通节点大小的两倍
 - 红黑树节点记录 key, value, color, parent, left, right
- 如果链表转为红黑树，还会转回来吗
 - 树节点小于等于 6
 - 为什么是 6 而不是 8
 - 避免单节点反复添加删除导致的树化与去树化频繁
- hashmap 的底层实现
 - 由数组+链表或红黑树构成
 - put 时，先计算 key 的 hash 值
 - 然后 $\text{hash} \& (n-1)$ 计算索引
 - 如果索引所在位置没有元素则直接放在数组中，如果有
 - 如果 key 不同，则产生哈希冲突，如果链表的元素小于 8 或者数组长度小于 64，在链表后加入元素
 - 如果 key 相同，直接替换
 - 如果链表的元素大于 8 且数组长度 ≥ 64 ，则把链表转换成红黑树
 - 为什么要数组长度大于等于 64
 - 避免频繁树化，减小内存占用
 - 数组容量小的时候，扩容自然会减少哈希冲突
 - 容量小更容易扩容，如果不限制数组长度，可能树化完又扩容了
 - 为什么是 8
 - 时间效率和空间占用的平衡
 - 因为红黑树占用内存大，能不用就不用
 - 链表冲突到 8 的几率很小
 - 扩容机制：
 - 如果数组小于 64，链表长度超过 8，优先扩容
 - 如果当前 hashmap 的容量 $>$ 容量 * 负载因子，则触发扩容 默认容量 16, 负载因子 0.75
 - 为什么是 0.75
 - 时间和空间效率的平衡
 - 0.5 扩容频繁，查得快
 - 1 的话查询效率慢，省空间
 - 扩容容量 $\times 2$ ，创建新数组，重新计算每个元素的新位置。
 - rehash 过程
 - 遍历旧表，对每个桶进行重新分配
 - 如果桶中只有一个节点：直接根据新 hash 重新放入新表。
 - 如果是链表结构：
 - 低位组 $((e.\text{hash} \& \text{oldCap}) == 0)$ 留在原索引；
 - 高位组 $((e.\text{hash} \& \text{oldCap}) != 0)$ 移动到 $\text{index} + \text{oldCap}$ 。
 - 如果是树结构：
 - 拆分为两棵树或链表，逻辑同上。
 - 为什么容量总是 2 的幂
 - 方便用 $(n-1) \& \text{hash}$ 快速计算索引，比直接取余运算快很多
 - 为什么 hash 高低位混合

- 避免哈希分布偏移，提高均匀性
- 为什么链表树化阈值 8
 - 实际测试中链表长度超过 8 概率极低，树化提升极少但增加复杂度
- 为什么扩容只移位不重算 hash
 - 节省 CPU 时间，利用位运算特性
- 为什么负载因子 0.75
 - 平衡时间与空间效率
- JDK 8 前后的区别
 - JDK 7，对于桶的哈希冲突问题，采用头插法解决，链表也是纯链表，没有红黑树的扩展，扩容全部重 hash
 - JDK8，采用尾插法避免多线程扩容时死循环，采用链表+红黑树的结果，扩容利用 (hash & oldCap) 判断位置
 - JDK8 使 key 的哈希值的高 16 位和低 16 位异或，使得哈希值同时拥有高位和低位的特性，哈希分布更均匀
 - JDK8 对元素迁移机制优化
- hashmap 头插法多线程下的死循环问题



- 其实就是头插法下，链表扩容时桶搬迁后是逆序的
- 但是多线程环境下，A,B 线程在搬迁前指向同一个头节点，若 A 节点时间片用完在休眠，B 节点开始搬迁，搬迁后，因为逆序的原因，等到 A 节点执行时间片的时候，就和新 hashmap 的存储顺序相反了，结果就会产生死循环
- 头插法 put 流程
 - 新元素计算到 key 所在 index 有多个元素时
 - 直接将新元素的 next 指向数组 index 指向的元素
 - 再让数组 index 指向新元素就可以了
 - 也就是说，数组[index]存的是链表头节点的索引，每次 put 新元素时将新元素 next 指向数组 index 指向的头节点，改变链表的头节点为新元素，再让数组 index 指向该元素即可
- 尾插法 put 流程
 - 计算到 index 后先遍历 index 所在位置的链表直到 tail
 - tail.next=新元素

- 新元素.next=null;
- 修改新元素为 tail
- hashmap 扩容时，顺着数组 index 所指向的元素一个个迁移，也就是从链表头开始迁移
- 也就是说扩容的时候先比较桶各个元素 hash & oldCap 的值，将为 0 的放在一个链表，不为 0 的放在另一个链表，这个过程用的尾插法，当遍历完一个桶所有元素后，将原桶 index 所在位置指向为 0 的链表的头节点索引，新桶指向不为 0 的
- hashset 和 hashmap 的区别
 - hashset 是基于 hashmap 实现的，不允许重复元素，无序
 - 主要是基于 hashmap 的 key 唯一实现的，value 为一个常量对象
- hashmap 保证顺序吗
 - 不保证，如果要保证可以用 linkedhashmap
 - 排序的话用 treemap
- linkedhashmap 使用过吗
 - LRU 缓存的实现
- hashmap 为什么线程不安全
 - HashMap 的设计是为了单线程环境的高效访问，因此
 - 没有加锁
 - 没有使用 volatile 等同步机制
 - 没有 CAS 等原子操作
 - 多线程 put 导致数据覆盖
 - 扩容 (resize) 时的线程安全问题
 - JDK 1.7: 扩容时链表可能反转成环导致死循环;
 - 头插法导致多线程扩容时死循环问题
 - 两个线程指向同一个头节点，A 线程休眠，B 线程扩容后链表逆序，A 线程继续执行时链表顺序和新 hashmap 不一致，导致死循环
 - JDK 1.8: 结构改为数组 + 链表/红黑树，虽然死循环问题修复了，但并发写入仍然会导致数据丢失或覆盖。
 - 尾插法避免死循环，但并发写入仍会覆盖数据
- hashmap 线程不安全，可以如何优化
 - hashtable，锁全表，效率慢
 - concurrenthashmap，效率高
 - synchronized，效率慢
 - Java 的锁有哪些
 - 内置锁 (Synchronized)
 - 显式锁 (ReentrantLock)
 - 读写锁 (ReadWriteLock)
 - 偏向锁 / 轻量级锁 / 重量级锁 / 自旋锁 (JVM 层优化)
 - 乐观锁/CAS
 - 其他锁机制
 - StampedLock
 - Semaphore:
 - CountDownLatch / CyclicBarrier / Phaser:
- synchronized 原理

- ▶ `synchronized` 是基于原子性的内部锁机制，是可重入的，因此在一个线程调用 `synchronized` 方法的同时在其方法体内部调用该对象另一个 `synchronized` 方法，也就是说一个线程得到一个对象锁后再次请求该对象锁，是允许的，这就是 `synchronized` 的可重入性。
 - `synchronized` 底层是利用计算机系统 `mutex Lock` 实现的。每一个可重入锁都会关联一个线程 ID 和一个锁状态 `status`。
 - 当一个线程请求方法时，会去检查锁状态。
 - 如果锁状态是 0，代表该锁没有被占用，使用 CAS 操作获取锁，将线程 ID 替换成自己的线程 ID。
 - 如果锁状态不是 0，代表有线程在访问该方法。此时，如果线程 ID 是自己的线程 ID，如果是可重入锁，会将 `status` 自增 1，然后获取到该锁，进而执行相应的方法；如果是非重入锁，就会进入阻塞队列等待。
 - 在释放锁时，
 - 如果是可重入锁的，每一次退出方法，就会将 `status` 减 1，直至 `status` 的值为 0，最后释放该锁。
 - 如果非可重入锁的，线程退出方法，直接就会释放该锁。
- 如何在多线程环境下使用 `HashMap`，除了加同步锁方案和 `ConcurrentHashMap` 以外还能想到什么
 - ▶ 读写锁 (`ReentrantReadWriteLock`)
 - 多读可以并发执行；
 - 写独占
 - 适合读多写少
 - 锁粒度仍然在“整个 Map”层面，性能不如分段锁。
 - ▶ 使用 Copy-On-Write 机制
 - 每次写入时复制一份新的 Map，读操作无锁
 - 写时要加 `synchronized` 锁
 - 写操作代价大（复制整个 map）；
 - ▶ 使用分段锁（分区锁）思想
 - 并发度更高，多个分区可同时写；
 - 原理接近 `ConcurrentHashMap 1.7` 的“Segment”分段锁。
 - ▶ 消息队列写
 - ▶ 使用 `ThreadLocal` + 合并机制。让每个线程维护自己的本地副本，最后再合并。
- `ConcurrentHashMap` 底层
 - ▶ 在 JDK 1.7 中它使用的是数组加链表的形式实现的，而数组又分为：大数组 `Segment` 和小数组 `HashEntry`。`Segment` 是一种可重入锁 (`ReentrantLock`)，在 `ConcurrentHashMap` 里扮演锁的角色；`HashEntry` 则用于存储键值对数据。一个 `ConcurrentHashMap` 里包含一个 `Segment` 数组，一个 `Segment` 里包含一个 `HashEntry` 数组，每个 `HashEntry` 是一个链表结构的元素。
 - ▶ 在 JDK 1.7 中，`ConcurrentHashMap` 虽然是线程安全的，但因为它的底层实现是数组 + 链表的形式，所以在数据比较多的情况下访问是很慢的，因为要遍历整个链表，而 JDK 1.8 则使用了数组 + 链表/红黑树的方式优化了 `ConcurrentHashMap` 的实现
 - ▶ JDK 1.8 `ConcurrentHashMap`
 - JDK 1.8 `ConcurrentHashMap` 主要通过 `volatile` + CAS 或者 `synchronized` 来实现的线程安全的。添加元素时首先会判断容器是否为空：
 - 如果为空则使用 `volatile` 加 CAS 来初始化
 - 如果容器不为空，则根据存储的元素计算该位置是否为空。
 - 如果根据存储的元素计算结果为空，则利用 CAS 设置该节点；

- 如果根据存储的元素计算结果不为空，则使用 `synchronized`，然后，遍历桶中的数据，并替换或新增节点到桶中，最后再判断是否需要转为红黑树，这样就能保证并发访问时的线程安全了。
- 如果把上面的执行用一句话归纳的话，就相当于是 `ConcurrentHashMap` 通过对头结点加锁来保证线程安全的，锁的粒度相比 `Segment` 来说更小了，发生冲突和加锁的频率降低了，并发操作的性能就提高了。
- 而且 JDK 1.8 使用的是红黑树优化了之前的固定链表，那么当数据量比较大的时候，查询性能也得到了很大的提升，从之前的 $O(n)$ 优化到了 $O(\log n)$ 的时间复杂度。
- 分段锁怎么加锁的？
 - 在 `ConcurrentHashMap` 中，对于插入、更新、删除等操作，需要先定位到具体的 `Segment`，然后再在该 `Segment` 上加锁，而不是像传统的 `HashMap` 一样对整个数据结构加锁。这样可以使得不同 `Segment` 之间的操作并行进行，提高了并发性能。
 - `get` 过程中使用了大量的 `volatile` 关键字，并没有加锁，保证了可见性
- 分段锁是可重入的吗？
 - JDK 1.7 `ConcurrentHashMap` 中的分段锁是用了 `ReentrantLock`，是一个可重入的锁。
- 已经用了 `synchronized`，为什么还要用 CAS 呢
 - `ConcurrentHashMap` 使用这两种手段来保证线程安全主要是一种权衡的考虑，在某些操作中使用 `synchronized`，还是使用 CAS，主要是根据锁竞争程度来判断的。
 - 比如：在 `putVal` 中，如果计算出来的 hash 槽没有存放元素，那么就可以直接使用 CAS 来进行设置值，这是因为在设置元素的时候，因为 hash 值经过了各种扰动后，造成 hash 碰撞的几率较低，那么我们可以预测使用较少的自旋来完成具体的 hash 落槽操作。
 - 当发生了 hash 碰撞的时候说明容量不够用了或者已经有大量线程访问了，因此这时候使用 `synchronized` 来处理 hash 碰撞比 CAS 效率要高，因为发生了 hash 碰撞大概率来说是线程竞争比较强烈。
- JDK8 `ConcurrentHashMap` 扩容机制——transfer + 多线程协助扩容（迁移机制）。
 - `HashMap/ConcurrentHashMap` 扩容的核心步骤：
 - 新建一个更大容量的数组（2 倍）
 - 把旧数组的每个桶（链表/红黑树）迁移到新数组
 - 迁移完成后替换引用
 - `HashMap` 扩容是单线程迁移，慢 + 会阻塞。`ConcurrentHashMap` 采用多线程协助迁移，因此扩容不会拖垮整个应用。
 - `ConcurrentHashMap` 扩容触发条件
 - `size >= threshold (capacity * loadFactor)`
 - 并且另一个线程还没有在扩容中时，当前线程会设置 `sizeCtl = -(1 + 当前参与扩容的线程数)`
 - transfer 迁移的核心思想
 - 扩容本质是把旧表（table）的某一段（range）迁移到新表（nextTable）。
 - 关键点：多个线程可以同时执行 transfer，但它们处理的是不同 segment 的桶范围。
 - 分段迁移：每个线程抢任务（work-stealing 思路）
 - JDK8 中使用 `transferIndex` 来控制迁移进度。
 - 它初始值指向旧 table 的最后一个 bucket 索引
 - 每个线程通过 CAS 竞争一段范围，比如一次迁移 16 个桶-
 - 成功后，只负责这个范围的桶迁移，迁移完以后再抢下一个范围
 - 每个桶如何迁移？
 - 加锁桶头节点，`synchronized`

- 分成两类链表：低位链表 和 高位链表
 - 哈希再扩容一倍时，节点会落到原索引：i,或者新索引：i + oldCapacity
 - 这里用位运算判断
 - (h & oldCap) == 0 → low list (留在原位)
 - (h & oldCap) != 0 → high list (搬到高位)
- ForwardingNode：避免重复迁移
 - 某个桶迁移完成后，会放置一个 ForwardingNode：表示这个桶已经迁移过了,后续线程遇到它时不再迁移
- JDK8 ConcurrentHashMap 扩容的整体模型
 - transferIndex 保证线程不会重复迁移同一个桶范围
 - 扩容时会创建一个新表
 - nextTable = new Node[oldCapacity * 2]
 - 然后由多个线程共同把 oldTable 的数据迁移到 nextTable。
 - 单个桶是怎么迁移的？
 - 遇到 ForwardingNode：说明已迁移，跳过
 - 迁移前对桶头节点加 synchronized 锁
 - 桶拆分为 low / high 两条链表
 - (h & oldCap) == 0 → low list
 - (h & oldCap) != 0 → high list
 - 迁移完成后放置 ForwardingNode
- 写操作 (put) 遇到扩容时怎么处理？
 - 首先通过 ForwardingNode 跳转到 nextTable
 - 反向触发写线程 也加入扩容工作。
 - 也就是说，扩容是“边写边迁移”的流式迁移，这样不会因为扩容阻塞业务。
- ConcurrentHashMap 实现缓存的时间复杂度是多少？如何做到的？
 - 当桶内为链表时，GET,PUT,DELETE，CONTAINSKEY 的复杂度为 O(1)
 - 当为红黑树时，为 O(logn)
- 为什么 hashmap 中允许 key 或者 value 为 null？而 concurrentHashMap 不允许 key 或者 value 为 null
 - 在 map 中，调用 map.get(key)方法得到的值是 null，那你无法判断这个 key 是在 map 里面没有映射过，还是这个 key 本身设置就 null。这种情况下，在非并发安全的 map 中，可以通过 map.contains(key)的方法来判断。但是在考虑并发安全的 map 中，两次调用的过程中，这个值是很有可能被改变的。
- hashmap, concurrenthashmap 有什么区别
 - 线程安全性
 - 多线程下 hasmap 可能出现链表丢失、死循环 (JDK7)、数据覆盖
 - 并发策略
 - HashMap：没有任何同步机制
 - ConcurrentHashMap：使用分段锁 / CAS + synchronized
 - JDK7：Segment 分段锁
 - 整个 Map 拆成多个 Segment（默认 16）每个 Segment 有自己的锁
 - JDK8：彻底重写，不再使用 Segment
 - CAS（无锁操作）+ 自旋
 - synchronized 锁节点链表或红黑树
 - 红黑树结构提升冲突桶性能

- put 时使用 CAS 尝试占位（无锁很快）
 - 失败再 锁链表头节点
 - 扩容时用 transfer + 多线程协助扩容
- null 不能放入 ConcurrentHashMap
- 性能 HashMap 最快
- 判断多线程下 synchronized 加在不同锁对象的效果
 - 如果多个线程进入的代码块用的是不同的锁对象
 - 完全不存在互斥
 - 不会阻塞彼此
 - 就像没有加锁一样并发执行
 - 即使锁的“内容一样”，但对象不是同一个，也不会同步
 - 同一个对象 → 才会互斥
 - 三种常见锁对象及区别
 - 锁 this（实例锁）
 - 不同实例 new 出来 → 不同步
 - 多个对象实例 → 各自的锁不同
 - 锁类对象（类锁）
 - 全局唯一 → 所有线程都同步。
 - 锁自定义对象
- 说一说 synchronized 的实现原理
 - synchronized 底层是 monitor，代码块 synchronized(obj)：synchronized 关键字经过编译之后，会在同步代码块前后分别形成 monitorenter 和 monitorexit 字节码指令，在执行 monitorenter 指令的时候，首先尝试获取对象的锁，如果这个锁没有被锁定或者当前线程已经拥有了那个对象的锁，锁的计数器就加 1，在执行 monitorexit 指令时会将锁的计数器减 1，当减为 0 的时候就释放锁。如果获取对象锁一直失败，那当前线程就要阻塞等待，直到对象锁被另一个线程释放为止。
 - 修饰方法 synchronized method，JVM 会标记 ACC_SYNCHRONIZED 标志，进入方法时自动加锁，退出时释放锁。Monitor 本质上是 JVM 对象（HotSpot 用 ObjectMonitor），内部有：
 - owner：持有锁的线程
 - EntryList：等待锁的线程队列
 - WaitSet：调用 wait() 的线程队列
 - 锁存在于对象头（Mark Word）中
 - synchronized 的锁升级
 - synchronized 是可重入锁，monitor 维护一个计数器
 - synchronized 是可见性 + 有序性的保证
- synchronized 和 reentrantlock

对比项	synchronized	ReentrantLock
所属	JVM 关键字	JUC (<code>java.util.concurrent.locks</code>) 类
锁类型	隐式锁 (自动加解锁)	显式锁 (手动加解锁)
可重入性	✅ 可重入	✅ 可重入
公平性	❌ 不支持公平锁	✅ 可选公平/非公平
可中断	❌ 不能中断等待锁的线程	✅ 可中断 (<code>lockInterruptibly()</code>)
尝试加锁	❌ 不支持	✅ 支持 <code>tryLock()</code>
超时等待	❌ 不支持	✅ 支持 <code>tryLock(timeout)</code>
条件变量	❌ 只能用 <code>wait/notify</code>	✅ 多个 <code>Condition</code> 对象
性能优化	JDK1.6+ 优化很好 (偏向锁、轻量锁、自旋)	性能稳定、适合高并发
解锁时机	自动 (代码块结束自动释放)	手动 (必须 <code>finally</code> 中 <code>unlock</code>)

▸ `synchronized` 是 Java 关键字，由 JVM 原生支持；底层是利用计算机系统 `mutex`，依赖对象头中的 `monitor`

一、对象头结构概览

在 HotSpot 虚拟机中，一个普通 Java 对象的内存布局分为三部分：

区域	作用
Mark Word (标记字段)	存储锁状态、哈希码、GC 分代年龄等
Class Pointer (类型指针)	指向对象的类元数据 (即类型信息)
Instance Data (实例数据)	对象真正的字段内容

五、总结：对象头和 Monitor 存放内容概览

位置	内容	说明
对象头 (Mark Word)	锁标志位、线程ID、GC年龄、偏向标志、指针等	每个对象都有
Monitor (ObjectMonitor)	当前持锁线程、等待队列、重入计数、关联对象	仅在重量级锁时存在

▸ `reentrantlock` 是普通类，基于 `AbstractQueuedSynchronizer` (AQS) 实现；锁的获取和释放是通过 `CAS + FIFO` 等待队列 (CLH 队列)；

• 介绍一下 AQS

- AQS 全称为 AbstractQueuedSynchronizer，是 Java 中的一个抽象类。AQS 是一个用于构建锁、同步器、协作工具类的工具类（框架）。
- AQS 核心思想是，如果被请求的共享资源空闲，那么就将当前请求资源的线程设置为有效的工作线程，将共享资源设置为锁定状态；如果共享资源被占用，就需要一定的阻塞等待唤醒机制来保证锁分配。这个机制主要用的是 CLH 队列的变体实现的，将暂时获取不到锁的线程加入到队列中。
- CLH: Craig、Landin and Hagersten 队列，是单向链表，AQS 中的队列是 CLH 变体的虚拟双向队列（FIFO），AQS 是通过将每条请求共享资源的线程封装成一个节点来实现锁的分配。
- AQS 使用一个 Volatile 的 int 类型的成员变量来表示同步状态，通过内置的 FIFO 队列来完成资源获取的排队工作，通过 CAS 完成对 State 值的修改。
- Sync 是 AQS 的实现。AQS 主要完成的任务：
 - 同步状态（比如说计数器）的原子性管理；
 - 线程的阻塞和解除阻塞；
 - 队列的管理。
- 在公平锁下：进入阻塞队列的线程，只有队首那个会在 state=0 时尝试获取资源。
- 在非公平锁下：队首线程会被唤醒尝试，但新来的线程也可能插队抢到锁。
- AQS 原理
 - AQS 最核心的就是三大部分：
 - 状态：state；
 - 控制线程抢锁和配合的 FIFO 队列（双向链表）；
 - 期望协作工具类去实现的获取/释放等重要方法（重写）。
 - 状态 state
 - 这里 state 的具体含义，会根据具体实现类的不同而不同：比如在 Semaphore 里，他表示剩余许可证的数量；在 CountdownLatch 里，它表示还需要倒数的数量；在 ReentrantLock 中，state 用来表示“锁”的占有情况，包括可重入计数，当 state 的值为 0 的时候，标识该 Lock 不被任何线程所占有。
 - state 是 volatile 修饰的，并被并发修改，所以修改 state 的方法都需要保证线程安全，比如 getState、setState 以及 compareAndSetState 操作来读取和更新这个状态。这些方法都依赖于 unsafe 类。
 - FIFO 队列
 - 这个队列用来存放“等待的线程”，AQS 就是“排队管理器”，当多个线程争用同一把锁时，必须有排队机制将那些没能拿到锁的线程串在一起。当锁释放时，锁管理器就会挑选一个合适的线程来占有这个刚刚释放的锁。
 - AQS 会维护一个等待的线程队列，把线程都放到这个队列里，这个队列是双向链表形式。
 - 实现获取/释放等方法
 - 这里的获取和释放方法，是利用 AQS 的协作工具类里最重要的方法，是由协作类自己去实现的，并且含义各不相同；
 - 获取方法：获取操作会以来 state 变量，经常会阻塞（比如获取不到锁的时候）。在 Semaphore 中，获取就是 acquire 方法，作用是获取一个许可证；而在 CountdownLatch 里面，获取就是 await 方法，作用是等待，直到倒数结束；
 - 释放方法：在 Semaphore 中，释放就是 release 方法，作用是释放一个许可证；在 CountdownLatch 里面，获取就是 countDown 方法，作用是将倒数的数减一；
 - 需要每个实现类重写 tryAcquire 和 tryRelease 等方法。
 - 当线程尝试加锁或释放锁时，AQS 会通过判断 exclusiveOwnerThread 来判断是否“属于自己”：

- CAS 和 AQS 有什么关系？
 - CAS 是一种乐观锁机制，它包含三个操作数：内存位置 (V)、预期值 (A) 和新值 (B)。CAS 操作的逻辑是，如果内存位置 V 的值等于预期值 A，则将其更新为新值 B，否则不做任何操作。整个过程是原子性的，通常由硬件指令支持，如在现代处理器上，`cmpxchg` 指令可以实现 CAS 操作。
 - AQS 是一个用于构建锁和同步器的框架，许多同步器如 `ReentrantLock`、`Semaphore`、`CountDownLatch` 等都是基于 AQS 构建的。AQS 使用一个 `volatile` 的整数变量 `state` 来表示同步状态，通过内置的 FIFO 队列来管理等待线程。它提供了一些基本的操作，如 `acquire` (获取资源) 和 `release` (释放资源)，这些操作会修改 `state` 的值，并根据 `state` 的值来判断线程是否可以获取或释放资源。AQS 的 `acquire` 操作通常会先尝试获取资源，如果失败，线程将被添加到等待队列中，并阻塞等待。`release` 操作会释放资源，并唤醒等待队列中的线程。
 - CAS 为 AQS 提供原子操作支持：AQS 内部使用 CAS 操作来更新 `state` 变量，以实现线程安全的状态修改。在 `acquire` 操作中，当线程尝试获取资源时，会使用 CAS 操作尝试将 `state` 从一个值更新为另一个值，如果更新失败，说明资源已被占用，线程会进入等待队列。在 `release` 操作中，当线程释放资源时，也会使用 CAS 操作将 `state` 恢复到相应的值，以保证状态更新的原子性。
- 如何用 AQS 实现一个可重入的公平锁？
 - 继承 `AbstractQueuedSynchronizer`：创建一个内部类继承自 `AbstractQueuedSynchronizer`，重写 `tryAcquire`、`tryRelease`、`isHeldExclusively` 等方法，这些方法将用于实现锁的获取、释放和判断锁是否被当前线程持有。
 - 实现可重入逻辑：在 `tryAcquire` 方法中，检查当前线程是否已经持有锁，如果是，则增加锁的持有次数（通过 `state` 变量）；如果不是，尝试使用 CAS 操作来获取锁。
 - 在 `tryAcquire` 方法中，按照队列顺序来获取锁，即先检查等待队列中是否有线程在等待，如果有，当前线程必须进入队列等待，而不是直接竞争锁。
 - 创建锁的外部类：创建一个外部类，内部持有 `AbstractQueuedSynchronizer` 的子类对象，并提供 `lock` 和 `unlock` 方法，这些方法将调用 `AbstractQueuedSynchronizer` 子类中的方法。
- CAS 存在什么问题，应用场景有哪些
 - ABA 问题
 - 值从 $A \rightarrow B \rightarrow A$ ，CAS 检查时看到值没变，误以为没有被修改过。
 - 解决方案：使用版本号机制，即带版本戳的 CAS。
 - 自旋 (Spin) 开销大
 - CAS 在失败时会不断重试 (自旋)，高并发下可能会占用大量 CPU。
 - 解决方案：限制自旋次数
 - 只能保证一个变量的原子性
 - 解决方案：使用 `AtomicReference` 封装多个变量；
 - 应用场景：
 - 原子类 (`AtomicInteger`、`AtomicBoolean` 等)
 - `ConcurrentHashMap`
 - AQS (`AbstractQueuedSynchronizer`)
 -
- JAVA 中的异常捕获
 - Java 的异常体系是基于类继承层次设计的，根类是 `java.lang.Throwable`：
 - `try-catch`
 - 多重 `catch`
 - `try-catch-finally`

- try-with-resources
 - 只要资源类实现了 `AutoCloseable` 接口（如 `FileInputStream`、`BufferedReader`、`Connection` 等），系统会自动在 try 结束后调用 `close()` 方法。
 - throws 声明
 - throw 关键字
 - 自定义异常
 - 继承 `Exception` 或 `RuntimeException`，添加构造方法和自定义字段。
 - try-catch：方法内部自己处理异常
 - throws：把异常抛给调用者处理，如果调用者也不处理，会一直向上传递给 JVM，最终导致程序终止。
- java 的异常类型
 - Error（错误类）
 - Error 表示 JVM 层面的严重错误，通常由系统引起，应用程序无法恢复或不应捕获。

常见的 Error：

异常类	说明
<code>OutOfMemoryError</code>	内存不足（例如创建过多对象）
<code>StackOverflowError</code>	栈溢出（例如递归过深）
<code>VirtualMachineError</code>	JVM 自身错误
<code>NoClassDefFoundError</code>	类加载错误
<code>AssertionError</code>	断言失败

👉 一般不使用 `try-catch` 捕获这些错误，应该排查系统或代码逻辑问题。



- Exception（异常类）
 - Exception 是可以被程序捕获并恢复的异常。
 - 检查型异常（Checked Exception）
 - 编译器会强制要求捕获或声明 throws。

异常类	说明
<code>IOException</code>	IO 操作失败（读写文件、网络传输等）
<code>FileNotFoundException</code>	文件不存在
<code>SQLException</code>	数据库访问异常
<code>ClassNotFoundException</code>	类未找到（反射加载失败）
<code>InterruptedException</code>	线程被中断
<code>ParseException</code>	字符串解析错误（日期格式等）
<code>MalformedURLException</code>	URL 格式错误
<code>CloneNotSupportedException</code>	对不支持克隆的对象调用 <code>clone()</code>

- 运行时异常（Runtime Exception）
 - 运行时自动检测到的逻辑错误，编译器不强制处理。

异常类	说明
<code>NullPointerException</code>	空指针引用
<code>ArrayIndexOutOfBoundsException</code>	数组下标越界
<code>StringIndexOutOfBoundsException</code>	字符串下标越界
<code>ArithmeticException</code>	算术错误（如除以零）
<code>ClassCastException</code>	类型强制转换错误
<code>IllegalArgumentException</code>	传入非法参数
<code>NumberFormatException</code>	数字格式转换错误（字符串转数字失败）
<code>UnsupportedOperationException</code>	操作不被支持
<code>ConcurrentModificationException</code>	并发修改集合错误
<code>IllegalStateException</code>	状态不合法（如线程已启动再 <code>start()</code> ）

- java 的集合有哪些
 - collection 接口体系
 - List —— 有序、可重复
 - `arraylist`
 - `linkedlist`
 - `vector` 线程安全
 - `stack` 继承自 `Vector`，后进先出（LIFO）
 - Set —— 无序、不重复
 - `hashset`
 - `treemap`

- linkedhashset
- Queue / Deque —— 队列 / 双端队列
 - PriorityQueue
 - ArrayDeque
 - LinkedList
 - Queue 是只能一端进一端出，Deque 是双端队列，两端都可进可出，Deque 也常用作栈
- Map 接口体系（键值对集合）
 - 特点：存储键值对（key-value），key 不可重复，value 可重复。
 - hashmap
 - treemap
 - linkedhashmap
 - Hashtable
 - concurrenthashmap
- ArrayList 和 LinkedList 有什么区别，添加数据和删除数据有什么区别，底层实现
 - 底层实现方面：arraylist 是动态数组 (Object[])，linkedlist 是双向链表 (Node{prev, next, item})
 - 内存分布：arraylist 是连续内存，linkedlist 是不连续内存
 - 随机访问：arraylist 是 O(1)，linkedlist 是 O(n)
 - 插入/删除：arraylist 中间操作会移动大量元素，linkedlist 中间操作只改节点指针，不需要移动其他元素
 - 扩容机制：arraylist 数组满就扩容 1.5 倍并拷贝，linkedlist 不需要扩容
 - 额外内存开销：arraylist 只存放对象引用，linkedlist 每个节点有两个额外指针（prev / next）
 - 选择 ArrayList 的场景
 - 读 > 写（读多写少）
 - 大量随机访问
 - 索引访问频繁的情况
 - 内存紧张（LinkedList 内存开销大）
 - 选择 LinkedList 的场景
 - 大量中间插入/删除
 - 需要头尾插入删除非常频繁（Deque 功能）
 - 实现队列或双端队列
- == 和 equals 的区别
 - 基本类型不能使用 equals
 - == 基本类型比，引用类型比地址
 - equals 默认比地址，重写后不一定
 - String、Integer、Date、集合类中的元素，都重写了 equals()，用来比较内容。所以 == 和 equals 可能不一样
- java 中 BIO, NIO, AIO 区别，如何实现，是否有过实际的使用
 - BIO (blocking IO)：就是传统的 java.io 包，它是基于流模型实现的，交互的方式是同步、阻塞方式，也就是说在读入输入流或者输出流时，在读写动作完成之前，线程会一直阻塞在那里，它们之间的调用是可靠的线性顺序。优点是代码比较简单、直观；缺点是 IO 的效率和扩展性很低，容易成为应用性能瓶颈。
 - NIO (non-blocking IO)：Java 1.4 引入的 java.nio 包，提供了 Channel、Selector、Buffer 等新的抽象，单线程可管理多个连接，可以构建多路复用的、同步非阻塞 IO 程序，同时提供了更接近操作系统底层高性能的数据操作方式。

- AIO (Asynchronous IO) : 是 Java 1.7 之后引入的包, 是 NIO 的升级版, 提供了异步非阻塞的 IO 操作方式, 所以人们叫它 AIO (Asynchronous IO), 异步 IO 是基于事件和回调机制实现的, 也就是应用操作之后会直接返回, 不会堵塞在那里, 当后台处理完成, 操作系统会通知相应的线程进行后续的操作。
- 底层实现区别
 - BIO: 一个请求一个线程, 线程调用 `socket.read()` 时挂起, 直到有数据。
 - NIO: 基于 I/O 多路复用 (`select/poll/epoll`), 一个线程能同时处理多个连接。
 - AIO: 基于操作系统异步 I/O 能力 (如 Linux 的 `io_uring`、Windows 的 IOCP)。

对比表

模型	线程模型	I/O 调用	事件获取方式	适用场景
BIO	1线程/1连接	阻塞	阻塞等待	小并发、简单服务
NIO	1线程/多连接	非阻塞	Selector 轮询	中高并发、Netty
AIO	1线程/多连接	非阻塞	系统回调通知	高并发、IOCP (Windows)

2. 线程在 AIO 里发起的请求类型

主要是三类 (跟 NIO Channel 的操作类似, 但变成异步) :

1. accept: 接收新连接

java

 复制代码

```
server.accept(attachment, handler);
```

- 请求操作系统: 当有新连接到来时, 回调通知我。

2. read: 读取数据

java

 复制代码

```
client.read(buffer, attachment, handler);
```

- 请求操作系统: 当数据读完后, 回调通知我 (`handler.completed`)。

3. write: 写入数据

java

 复制代码

```
client.write(buffer, attachment, handler);
```

- 请求操作系统: 等数据写入完成, 再通知我。

• NIO 底层原理

- Channel (通道)
 - 类似于流 (Stream), 但支持双向读写。
 - 可注册到 Selector 上, 进行事件监听。
- Buffer (缓冲区)
 - 所有读写都先经过 Buffer。
 - 它是内存中的一块区域, 有位置 `position`、界限 `limit`、容量 `capacity` 三个概念。

- 读写模式通过 flip()、clear() 切换。
- Selector（选择器）
 - 是多路复用器（Multiplexer），允许一个线程同时监视多个 Channel。
 - 底层依赖操作系统的多路复用机制
- 为什么 NIO 这么快？
 - BIO 模型中，一个连接一个线程，当上万连接时会造成：大量线程上下文切换；线程调度开销；内存消耗巨大。
 - NIO 使用 单线程 + Selector 同时管理多个连接，大大提升并发能力。
 - 使用操作系统底层的 I/O 多路复用
 - 使用直接内存（DirectBuffer）减少数据拷贝
- NIO 的高性能是以“流式字节”形式读取的，不会像 HTTP 那样天然分包，所以很容易出现粘包/拆包问题。
- 粘包拆包的问题
 - 消息边界混乱，无法正确解析数据。无法判断一条消息从哪里开始、到哪里结束；解析 JSON 失败；
 - 应用层逻辑复杂，增加开发维护成本。为了解决粘包/拆包问题，应用层必须：自行维护一个缓存区；对数据进行分片、拼接；自行实现协议，如：固定长度、分隔符、长度前缀等。
 - 性能下降，尤其在高并发场景，需要在应用层拼装完整消息；消息边界解析、缓存管理会带来额外 CPU 与内存开销；
 - 消息丢失或错位风险增加
 - 对粘包处理不当
- 为什么很多大厂只用 post 发请求
 - URL 长度限制
 - GET 请求参数写在 URL 上，不同浏览器、代理、服务器对 URL 长度有限制（常见 2KB 8KB）。
 - 在大厂业务场景中，请求参数可能很复杂（JSON、大量条件、批量 ID、加密串），GET 根本放不下。
 - POST 请求参数放在请求体中，长度没有限制。
 - 安全性
 - GET 请求参数暴露在 URL 上，容易出现在
 - 浏览器地址栏
 - 代理/网关日志
 - 浏览器历史记录
 - 敏感数据（如 token、手机号、订单号）用 POST 放在 body 里更安全。
 - 缓存与幂等性问题
 - HTTP 语义里：
 - GET 应该是安全、幂等的 → 浏览器、中间代理可能会缓存
 - POST 不幂等，默认不缓存
 - 在复杂的分布式系统里，大厂不想依赖中间层“自动缓存”的行为，所以干脆都用 POST，自己做缓存控制。
 - 统一网关与监控
 - 大厂通常有 API 网关、统一鉴权、监控系统。
 - 如果方法不统一（有的用 GET，有的用 DELETE），会增加接入和维护的复杂度。
 - 全部用 POST，统一在 header/body 做参数校验、签名验证、加解密，更加简单。
 - 跨域与复杂请求
 - 浏览器里，GET、POST 的简单请求比较好处理。

- 但如果需要传自定义 header（如 token、签名），很多情况都会变成 复杂请求 (CORS Preflight)。
- 大厂干脆约定一律 POST，减少前后端沟通成本。
- 是一种务实选择，而不是规范选择。
- 长连接和短连接，长连接断开后会发生什么？
 - HTTP 是以 TCP 协议传输的，也就是请求发送前要建立 TCP 连接
 - 如果每次发送都要建立 TCP 连接就是短连接，如果建立一次 TCP 连接可以进行多次请求就是长连接
 - 长连接中断会发生什么？
 - 长连接中断有几种情况
 - TCP 连接被动关闭（服务器或客户端主动关闭）
 - 服务器会发送 FIN 包，TCP 进入 FIN_WAIT 状态
 - 客户端接收到 FIN 包会进入 CLOSE_WAIT 状态
 - 之后完成四次挥手
 - 下一次请求时，客户端发现连接已关闭，就会自动重新建立 TCP 连接并重发请求。
 - TCP 连接异常中断（如网络波动、服务器崩溃）
 - 客户端会在写请求时得到 Broken pipe 或 Connection reset 错误；
 - 在在读响应时得到 Connection reset by peer；
 - TCP 状态会变成 CLOSE_WAIT 或 RESET。
 - 当前请求失败；连接池中的该连接会被移除；客户端会尝试重新建立新连接发送请求
 - 请求发送到一半时中断
 - 如果客户端发送请求数据过程中连接断了 请求直接失败；
 - 如果服务器已经收到了部分数据但没收完：服务器可能丢弃请求或返回错误（如 400、408）。
 - 客户端一般不会自动重试非幂等请求（如 POST），以免造成重复提交。
- MTU
 - MTU (Maximum Transmission Unit, 最大传输单元) 指的是一次可以在网络层传输的最大数据包大小（不含链路层头部）。
 - 常见情况下 MTU = 1500 字节（特别是在以太网环境）。应用层实际能传输的 TCP 数据 (MSS) 约为 1460 字节。
 - 如果数据包超过 MTU，会被 分片 (Fragmentation)；
 - 分片会导致性能下降；
 - 在 VPN、隧道、MPLS 环境中，还可能引发“路径 MTU 问题” (PMTU 黑洞)。
- PMTU
 - PMTU (Path MTU) = 从源到目的地路径上，所有链路中最小的 MTU 值。
 - PMTU 发现机制 (Path MTU Discovery)
 - 为避免分片，IP 协议设计了 PMTU Discovery (PMTUD) 机制：
 - 发送方发包时，设置：IP 头中的 DF(Don't Fragment) = 1，表示“禁止中途分片”。
 - 如果中途某个路由器发现包太大、自己 MTU 不够
 - 丢弃该包
 - 并返回一个 ICMP “Fragmentation Needed” (Type 3, Code 4) 消息；
 - 告诉发送方：“我这里 MTU = XXX”。
 - 发送方据此降低包大小（更新路径 MTU），直到能成功传输。
 - PMTU 黑洞是什么？
 - 有些防火墙或路由器 拦截或丢弃了 ICMP 消息。

- 路由器丢包了；但发送方收不到 ICMP 通知；因为 DF=1，又不能分片；所以发送方永远不知道该减小包大小。
- 解决方案：
 - 允许 ICMP “Fragmentation Needed” 通过
 - 禁用 DF 位（允许分片）
 - 手动调整 MTU
- 常见的请求头内容 content-type 有什么
 - 常见的请求头内容
 - 通用头（General）：和请求整体有关
 - Host：指定目标主机和端口；
 - Connection：是否保持连接（如 keep-alive）。
 - 实体头（Entity）：描述请求体内容
 - Content-Type：说明请求体类型（如 application/json、multipart/form-data）；
 - Content-Length：请求体长度。
 - 客户端信息头（Client Info）：描述客户端信息，比如
 - User-Agent：浏览器或客户端信息；
 - Accept、Accept-Language、Accept-Encoding：声明可接受的响应格式、语言、压缩方式。
 - 认证与安全相关头（Auth & Security）：
 - Authorization：携带令牌（JWT / Basic Auth）；
 - Cookie：传递登录状态。
 - 跨域与自定义头（CORS / Custom）：
 - Origin：跨域请求源；
- 认证头只能是 authorization 吗
 - 认证头不只能是 Authorization，但标准规定它是唯一正式的认证字段。在 HTTP 标准（RFC 7235 / RFC 9110）中，Authorization 是唯一被标准定义的用于客户端向服务器传递认证信息的请求头。
 - 实践中：可以起别名，但属于“自定义请求头”
 - 自定义头不是“标准认证头”，它不会被某些中间件（如 Nginx、Spring Security）自动识别处理；你需要自己解析。
 - 跨域（CORS）时必须显式允许，浏览器会先发一个 OPTIONS 预检请求。后端必须允许这个头：
 - HTTP 请求头里的 Content-Type 是非常重要的一个字段，用来声明请求或响应的实体内容（body）类型，告诉服务器（或客户端）数据是以什么格式传输的。
 - 常见的 Content-Type 类型分类
 - 文本类型
 - text/plain 纯文本 普通字符串请求、日志上传
 - text/html HTML 文本 网页内容（HTML 页面）
 - text/css 前端样式文件
 - text/javascript 前端脚本
 - text/xml XML 数据传输
 - 表单类型（Form）
 - application/x-www-form-urlencoded 默认表单格式，键值对以 key1=value1&key2=value2 方式编码 普通 HTML 表单提交

- multipart/form-data 用于上传文件，数据被分块传输
- text/plain 简单表单文本上传
- JSON / XML / 结构化数据
- 文件与二进制数据
 - 二进制流
 - PDF
 - ZIP
 - 图片文件
 - 音视频文件
- 常见的返回状态码
 - 1xx 表示请求已被接收，正在继续处理。
 - 2xx 表示请求已成功被接收、理解并接受
 - 3xx 重定向告诉客户端去别的地方获取资源。
 - 4xx 客户端错误表示请求有问题，客户端需要修改请求。如 400 参数错误、401 未授权、403 禁止、404 未找到；
 - 5xx 服务器错误，500 内部错误、502 网关错误、503 服务不可用。
- 永久重定向和临时重定向区别
 - 永久重定向浏览器会缓存重定向结果，下次访问不再请求原 URL，临时重定向不缓存，下次仍访问原 URL
 - 永久重定向搜索引擎会将旧地址的权重转移给新地址
 - 永久重定向一次后浏览器直接跳过短链接服务器，无法统计访问量
 - 永久重定向不能跳转目标修改
- 输入一个表单用的是什么格式的
 - 普通表单（默认）application/x-www-form-urlencoded 登录、注册、搜索
 - 文件上传：multipart/form-data 上传图片，文件
 - JSON 提交（JS 手动发）application/json
- OSI 七层和 TCP 四层网络模型 每层干什么，有什么协议

一、OSI 七层模型（理论模型）

层次	名称	主要功能	常见协议/设备
7	应用层 (Application)	为用户提供网络服务接口，如文件传输、电子邮件、网页浏览等	HTTP, HTTPS, FTP, SMTP, POP3, DNS, SSH, Telnet
6	表示层 (Presentation)	数据的格式化、加密、压缩，使不同系统可互相理解	SSL/TLS, JPEG, MPEG, ASCII, GIF
5	会话层 (Session)	建立、管理和终止会话（控制会话的开始、保持、同步、恢复）	RPC, NetBIOS, PPTP, SOCKS
4	传输层 (Transport)	端到端的通信、可靠传输、流量控制、差错检测	TCP, UDP
3	网络层 (Network)	路由选择、逻辑寻址（IP 地址），决定数据如何走向目标	IP, ICMP, IGMP, ARP, RIP, OSPF, BGP
2	数据链路层 (Data Link)	物理寻址（MAC），成帧、差错检测、流量控制	Ethernet, PPP, HDLC, VLAN, STP
1	物理层 (Physical)	比特流传输，定义电气、机械规范（电缆、电压、速率）	RJ45, 光纤, 同轴电缆, RS-232

二、TCP/IP 四层模型（实际使用模型）

TCP/IP 层	对应 OSI 层	功能	常见协议
应用层	应用层 + 表示层 + 会话层	提供应用服务，如网页、邮件、DNS 等	HTTP, HTTPS, FTP, SMTP, DNS, SSH, Telnet, SNMP
传输层	传输层	提供端到端的可靠或不可靠传输	TCP, UDP
网络层 (网际层)	网络层	负责寻址与路由选择	IP, ICMP, IGMP, ARP, RARP
网络接口层 (链路层)	数据链路层 + 物理层	实现物理传输、帧封装、MAC 寻址	Ethernet, Wi-Fi (802.11), PPP, SLIP

- 输入一段 URL，整个过程发生了啥
 - URL 解析，分析 URL 所需要使用的传输协议和请求的资源路径。
 - 如果输入的 URL 中的协议或者主机名不合法，将会把地址栏中输入的内容传递给搜索引擎。
 - 如果没有问题，浏览器会检查 URL 中是否出现了非法字符，则对非法字符进行转义后在进行下一过程。
 - 缓存查询
 - 缓存判断：浏览器缓存 → 系统缓存（hosts 文件）→ 路由器缓存 → ISP 的 DNS 缓存，如果其中某个缓存存在，直接返回服务器的 IP 地址。
 - DNS 解析

- 如果缓存未命中，浏览器向本地 DNS 服务器发起请求，最终可能通过根域名服务器、顶级域名服务器 (.com)、权威域名服务器逐级查询，直到获取目标域名的 IP 地址。
- MAC 地址查找
 - 当浏览器得到 IP 地址后，数据传输还需要知道目的主机 MAC 地址，因为应用层下发数据给传输层，TCP 协议会指定源端口号和目的端口号，然后下发给网络层。网络层会将本机地址作为源地址，获取的 IP 地址作为目的地址。然后将下发给数据链路层，数据链路层的发送需要加入通信双方的 MAC 地址，本机的 MAC 地址作为源 MAC 地址，目的 MAC 地址需要分情况处理。通过将 IP 地址与本机的子网掩码相结合，可以判断是否与请求主机在同一个子网里，如果在同一个子网里，可以使用 ARP 协议获取到目的主机的 MAC 地址，如果不在一个子网里，那么请求应该转发给网关，由它代为转发，此时同样可以通过 ARP 协议来获取网关的 MAC 地址，此时目的主机的 MAC 地址应该为网关的地址。
- 建立 TCP 连接
 - 主机将使用目标 IP 地址和目标 MAC 地址发送一个 TCP SYN 包，请求建立一个 TCP 连接，然后交给路由器转发，等路由器转到目标服务器后，服务器回复一个 SYN-ACK 包，确认连接请求。然后，主机发送一个 ACK 包，确认已收到服务器的确认，然后 TCP 连接建立完成。
- SSL/TLS 四次握手
 - 如果使用的是 HTTPS 协议，在通信前还存在 TLS 的四次握手。
- 发送请求
 - 连接建立后，浏览器会向服务器发送 HTTP 请求。请求中包含了用户需要获取的资源的信息，例如网页的 URL、请求方法（GET、POST 等）等。
- 接受请求渲染页面
- 简单来说，当用户输入 url url 中携带了接收方的 ip 地址，本地就处理 ip 地址直到 mac，然后判断是否在同一子网下，若在则直接通过 ARP 找到目标主机 MAC 若不在则使用 ARP 获取 网关 MAC，发送到网关 网关负责查路由表 → 决定下一跳 → 最终把数据送到目标主机，发送的过程要建立 tcp 连接，如果是 https 还要建立 tls 连接，建立完连接后就能发送请求了
- ARP 协议介绍
 - ARP (Address Resolution Protocol, 地址解析协议) 是一个工作在网络层和数据链路层之间的协议
 - 将 IP 地址解析成对应的 MAC 地址 (物理地址)。
 - 计算机在局域网中通信时，实际上是通过 MAC 地址 (硬件地址) 来识别目标设备的。但是上层协议 (例如 IP 层) 使用的是 IP 地址。ARP 充当了 IP 地址 和 MAC 地址 之间的“翻译官”。
 - ARP 工作原理
 - 查询阶段 (ARP 请求)
 - 当主机 A 想知道某个 IP 地址 (比如 192.168.1.10) 对应的 MAC 地址时
 - A 会在局域网内广播一个 ARP 请求包
 - 内容大致是: “谁是 192.168.1.10? 请把你的 MAC 地址告诉我
 - ARP 请求是广播包 (目的 MAC 地址是 FF:FF:FF:FF:FF:FF)。
 - 应答阶段 (ARP 响应)
 - 局域网中所有主机都会收到请求;
 - 只有 IP 地址匹配的主机会回应;
 - 该主机单播回复: “192.168.1.10 的 MAC 地址是 xx:xx:xx:xx:xx:xx”。
 - ARP 响应是单播包 (只发给请求者)。
 - 缓存阶段 (ARP 缓存)

- A 收到应答后，会把“IP 与 MAC 的对应关系”存入本地的 ARP 缓存表
- 下次再通信时可直接使用，不必再广播请求
- 介绍一下 HTTP 协议是个什么东西
 - 它是 Web 浏览器和服务端之间通信的规则，也就是浏览器访问网站时用的“语言”。
 - HTTP 的作用主要用于客户端（浏览器、APP）向服务器发送请求，获取资源（网页、图片、JSON 数据等）。
 - 服务器根据请求返回相应内容
 - HTTP 的特点：
 - 无状态：每次请求都是独立的，服务器不记得上一次的请求。需要用户登录状态时通常用 Cookie/Token 维持。
 - 基于文本：请求和响应都是文本形式（比如请求行、请求头、响应头），便于调试。
 - 基于 TCP：HTTP 本身不直接保证可靠传输，它依赖 TCP 来保证数据完整、顺序正确。
 - 面向请求-响应：客户端发请求，服务器返回响应，典型的“问答式”通信。
- http1.1 和 2.0 区别
 - HTTP/1.1 每个请求/响应都需要单独的 TCP 连接（虽然支持 keep-alive 复用连接）。请求顺序执行（串行），容易造成队头阻塞（Head-of-Line Blocking）。文本格式（Request/Response 都是可读文本）。每个请求都会重复发送完整的 headers，冗余多。
 - HTTP/2.0 单个 TCP 连接上多路复用（multiplexing），在一个 TCP 连接里可以同时发送多个请求和响应，多个请求/响应可以同时交错发送，不必排队。解决了 HTTP/1.1 的队头阻塞问题。二进制帧（frame）格式，数据被分解为帧进行传输，更高效，也便于解析。支持请求头压缩（HPACK），减少冗余信息。支持服务端推送（Server Push），服务端可以主动发送客户端可能需要的资源，减少等待。头部压缩 + 多路复用，极大减少重复传输的数据量。
- http 在 tcp 的基础上扩展了什么功能
 - TCP 负责：
 - 可靠传输
 - 顺序传输
 - 流量控制
 - 拥塞控制
 - 面向连接
 - HTTP 在 TCP 上实现的功能
 - 请求-响应模型
 - 定义了客户端（浏览器）如何发送请求（Request）；
 - 定义了服务器如何返回响应（Response）；
 - 请求和响应都是纯文本格式，易于理解和调试。
 - 资源定位与操作语义
 - URL（统一资源定位符）语义，告诉客户端要访问什么资源；
 - 方法（Method）语义：get post 等
 - 报文结构与内容描述
 - 首部（Header）：描述数据的元信息（如类型、长度、缓存策略等）；
 - 主体（Body）：真正的传输内容；
 - 状态码（Status Code）：用来表达结果（如 200、404、500）。
 - 无状态通信机制
 - HTTP 是无状态协议，每次请求都是独立的。服务器不会自动记住客户端是谁；
 - 为什么要设计成无状态
 - 简化服务器设计——“无记忆，就不复杂”

- 简化协议，提升通用性和可移植性
- 更容易水平扩展（Scalability）
- 可靠性更高，出错恢复更简单
 - 某个请求失败，也不影响之前或之后的请求
 - 网络断开后重试很容易（直接重发请求即可）。
- 这在 TCP 层是没有的（TCP 是有状态连接，因为 TCP 在通信的整个生命周期中，双方都要维护一系列连接状态），但 HTTP 把“会话状态”交给上层
 - 使用 Cookie、Session、Token 实现用户状态；
- 缓存、代理与重定向机制
- 内容协商与压缩
- 安全与扩展（HTTPS）
- 你提到了 TCP 协议，这个 TCP 协议是什么？它有什么特点？
 - TCP 是一种面向连接的、可靠的传输协议，主要作用是在网络中两个主机之间可靠地传送数据。
 - 面向连接：通信前，需要先建立连接（三次握手）。
 - 可靠传输：保证数据完整、按顺序到达。
 - TCP 工作在 OSI 模型的传输层（TCP/IP 模型中也是传输层），为上层应用（如 HTTP、FTP、SMTP）提供可靠的数据传输服务。
 - TCP 的主要特点
 - 面向连接（Connection-oriented）
 - 数据传输前，通信双方必须先建立连接（TCP 三次握手）。
 - 传输完成后，要断开连接（四次挥手）。
 - 可靠性传输
 - 数据按序到达：TCP 会对数据分段，并加上序号，保证接收端按顺序重组。
 - 错误检测：每个段都有校验和，确保数据完整性。
 - 重传机制：丢失的数据会被重新发送（超时重传）。
 - 流量控制（Flow Control）
 - 避免发送端过快发送，导致接收端来不及处理。
 - TCP 通过滑动窗口机制动态调整发送速度。
 - 拥塞控制（Congestion Control）
 - 避免网络拥堵导致数据丢失。
 - TCP 会根据网络状况调整发送速率（慢启动、拥塞避免等算法）。
 - 面向字节流（Byte Stream）
 - TCP 把数据看成连续的字节流，而不是独立的报文。
 - 应用程序发送的数据会被分段，但接收端可以按顺序组装。
 - 全双工通信（Full Duplex）
 - 双方可以同时发送和接收数据。
- UDP 和 TCP 有什么区别
 - UDP 是无连接的，不可靠的（不保证数据到达，也不保证顺序）
 - 面向报文
 - 没有流量控制和拥塞控制
 - 头部开销小
- udp 如何实现可靠传输
 - 确认与重传机制
 - 序号 + 排序（Ordering）
 - UDP 数据包可能乱序到达，为了保证顺序，给每个包一个序号。

- 接收方根据序号排序，缺失的包可以触发重传请求
- 校验和与完整性检查
 - CRC 校验、哈希等，确保数据未被篡改或损坏。
- 滑动窗口 / 流量控制
 - 发送方维护一个窗口，发送多包而不必等待每个包的 ACK。
 - 接收方确认接收到的包，并通知窗口滑动。
- 冗余与纠错码 (FEC, Forward Error Correction)
 - 接收方即使丢失部分包，也能通过冗余信息恢复数据。
 - XOR 异或校验
 - 假设你要发送 3 个数据包 D1,D2,D3
 - 计算冗余包 $P = D1 \oplus D2 \oplus D3$ 的异或
 - 如果丢失了其中一个包，就可以用收到的三个包的异或恢复另外一个
- tcp 协议是如何保证数据的可靠传输的
 - 序列号，接收方按序号重新组装数据，用来检测丢包、乱序、重复数据
 - 确认应答机制 (ACK)
 - 超时重传
 - 快速重传：不等超时，快速判断丢包，当收到 3 个重复 ACK 时，立即重发丢失的数据包
 - 滑动窗口控制能同时发送多少数据，丢包时只重传未确认部分（不是全部）
 - 流量控制 (Flow Control) 用 接收窗口 (rwnd) 避免把数据塞爆对方
 - 拥塞控制
 - 数据校验
 - 乱序重排
- tcp 包的序号是如何定的，比如我有一个很大的包，分成了 50 份，这 50 个数据包是如何编号的
 - TCP 会给整个数据流的每个字节编号
 - 每个 TCP 包的 seq 是本包第一个字节的序号
 - 每个 TCP 包的 ack 是期望收到的下一个字节序号
 - 举例来说，包 1 的 SEQ 值为 10000,数据长度为 1000,覆盖字节范围就是 10000-10999，接收方收到包 1 (1000 字节) 回复 ACK=11000
- TCP 三次握手过程说一下？
 - 一开始，客户端和服务端都处于 CLOSE 状态。先是服务端主动监听某个端口，处于 LISTEN 状态
 - 客户端会随机初始化序号 (client_isn)，将此序号置于 TCP 首部的「序号」字段中，同时把 SYN 标志位置为 1，表示 SYN 报文。接着把第一个 SYN 报文发送给服务端，表示向服务端发起连接，该报文不包含应用层数据，之后客户端处于 SYN-SENT 状态。
 - 服务端收到客户端的 SYN 报文后，首先服务端也随机初始化自己的序号 (server_isn)，将此序号填入 TCP 首部的「序号」字段中，其次把 TCP 首部的「确认应答号」字段填入 client_isn + 1, 接着把 SYN 和 ACK 标志位置为 1。最后把该报文发给客户端，该报文也不包含应用层数据，之后服务端处于 SYN-RCVD 状态。
 - 客户端收到服务端报文后，还要向服务端回应最后一个应答报文，首先该应答报文 TCP 首部 ACK 标志位置为 1，其次「确认应答号」字段填入 server_isn + 1，最后把报文发送给服务端，这次报文可以携带客户到服务端的数据，之后客户端处于 ESTABLISHED 状态。
 - 服务端收到客户端的应答报文后，也进入 ESTABLISHED 状态。
 - 从上面的过程可以发现第三次握手是可以携带数据的，前两次握手是不可以携带数据的，这也是面试常问的题。

- ▶ 一旦完成三次握手，双方都处于 ESTABLISHED 状态，此时连接就已建立完成，客户端和服务端就可以相互发送数据了。
- tcp 为什么需要三次握手建立连接？
 - ▶ 三次握手的原因：
 - 三次握手才可以阻止重复历史连接的初始化（主要原因）
 - 三次握手才可以同步双方的初始序列号
 - 三次握手才可以避免资源浪费
 - ▶ 原因一：避免历史连接
 - 我们考虑一个场景，客户端先发送了 SYN (seq = 90) 报文，然后客户端宕机了，而且这个 SYN 报文还被网络阻塞了，服务并没有收到，接着客户端重启后，又重新向服务端建立连接，发送了 SYN (seq = 100) 报文（注意！不是重传 SYN，重传的 SYN 的序列号是一样的）。
 - 客户端连续发送多次 SYN（都是同一个四元组）建立连接的报文，在网络拥堵情况下：
 - 一个「旧 SYN 报文」比「最新的 SYN」报文早到达了服务端，那么此时服务端就会回一个 SYN + ACK 报文给客户端，此报文中的确认号是 91 (90+1)。
 - 客户端收到后，发现自己期望收到的确认号应该是 100 + 1，而不是 90 + 1，于是就会回 RST 报文。
 - 服务端收到 RST 报文后，就会释放连接。
 - 后续最新的 SYN 抵达了服务端后，客户端与服务端就可以正常的完成三次握手了。
 - 上述中的「旧 SYN 报文」称为历史连接，TCP 使用三次握手建立连接的最主要原因就是防止「历史连接」初始化了连接。
 - 如果是两次握手连接，就无法阻止历史连接，那为什么 TCP 两次握手为什么无法阻止历史连接呢？
 - 主要是因为两次握手的情况下，服务端没有中间状态给客户端来阻止历史连接，导致服务端可能建立一个历史连接，造成资源浪费。
 - 你想想，在两次握手的情况下，服务端在收到 SYN 报文后，就进入 ESTABLISHED 状态，意味着这时可以给对方发送数据，但是客户端此时还没有进入 ESTABLISHED 状态，假设这次是历史连接，客户端判断到此连接为历史连接，那么就会回 RST 报文来断开连接，而服务端在第一次握手的时候就进入 ESTABLISHED 状态，所以它可以发送数据的，但是它并不知道这个是历史连接，它只有在收到 RST 报文后，才会断开连接。
 - 可以看到，如果采用两次握手建立 TCP 连接的场景下，服务端在向客户端发送数据前，并没有阻止掉历史连接，导致服务端建立了一个历史连接，白白发送了数据，妥妥地浪费了服务端的资源。
 - 因此，要解决这种现象，最好就是在服务端发送数据前，也就是建立连接之前，要阻止掉历史连接，这样就不会造成资源浪费，而要实现这个功能，就需要三次握手。
 - ▶ 原因二：同步双方初始序列号
 - TCP 协议的通信双方，都必须维护一个「序列号」，序列号是可靠传输的一个关键因素，它的作用：
 - 接收方可以去除重复的数据；
 - 接收方可以根据数据包的序列号按序接收；
 - 可以标识发送出去的数据包中，哪些是已经被对方收到的（通过 ACK 报文中的序列号知道）；
 - 可见，序列号在 TCP 连接中占据着非常重要的作用，所以当客户端发送携带「初始序列号」的 SYN 报文的时候，需要服务端回一个 ACK 应答报文，表示客户端的 SYN 报文已被服务端成功接收，那当服务端发送「初始序列号」给客户端的时候，依然也要得到客户端的应答回应，这样一来一回，才能确保双方的初始序列号能被可靠的同步。
 - 四次握手与三次握手

- 四次握手其实也能够可靠的同步双方的初始化序号，但由于第二步和第三步可以优化成一步，所以就成了「三次握手」。
- 而两次握手只保证了一方的初始序列号能被对方成功接收，没办法保证双方的初始序列号都能被确认接收。
- ▶ 原因三：避免资源浪费
 - 如果只有「两次握手」，当客户端发生的 SYN 报文在网络中阻塞，客户端没有接收到 ACK 报文，就会重新发送 SYN，由于没有第三次握手，服务端不清楚客户端是否收到了自己回复的 ACK 报文，所以服务端每收到一个 SYN 就只能先主动建立一个连接，这会造成什么情况呢？
 - 如果客户端发送的 SYN 报文在网络中阻塞了，重复发送多次 SYN 报文，那么服务端在收到请求后就会建立多个冗余的无效链接，造成不必要的资源浪费。
 - 即两次握手会造成消息滞留情况下，服务端重复接受无用的连接请求 SYN 报文，而造成重复分配资源。
- https 是非对称加密还是对称加密，在首次握手和正式传输中
 - ▶ 首次握手（TLS 握手阶段）主要使用 非对称加密（RSA / ECC）
 - 验证服务器身份（证书、公钥）
 - 交换对称密钥（安全地协商出 session key）
 - ▶ 正式数据传输阶段使用 对称加密（AES、ChaCha20 等）
 - 对称加密 运算快，适合大量数据交换
 - 已经有安全的会话密钥（session key）
- 描述一下打开百度首页后发生的网络过程
 - ▶ 解析 URL：分析 URL 所需要使用的传输协议和请求的资源路径。如果输入的 URL 中的协议或者主机名不合法，将会把地址栏中输入的内容传递给搜索引擎。如果没有问题，浏览器会检查 URL 中是否出现了非法字符，则对非法字符进行转义后在进行下一过程。
 - ▶ 缓存判断：浏览器缓存 → 系统缓存（hosts 文件） → 路由器缓存 → ISP 的 DNS 缓存，如果其中某个缓存存在，直接返回服务器的 IP 地址。
 - ▶ DNS 解析（把域名解析为 IP 地址）：如果缓存未命中，浏览器向本地 DNS 服务器发起请求，最终可能通过根域名服务器、顶级域名服务器（.com）、权威域名服务器逐级查询，直到获取目标域名的 IP 地址。
 - ▶ 获取 MAC 地址（唯一标识网卡，只在局域网内有效，不会跨路由传播）：当浏览器得到 IP 地址后，数据传输还需要知道目的主机 MAC 地址，因为应用层下发数据给传输层，TCP 协议会指定源端口号和目的端口号，然后下发给网络层。网络层会将本机地址作为源地址，获取的 IP 地址作为目的地址。然后将下发给数据链路层，数据链路层的发送需要加入通信双方的 MAC 地址，本机的 MAC 地址作为源 MAC 地址，目的 MAC 地址需要分情况处理。通过将 IP 地址与本机的子网掩码相结合，可以判断是否与请求主机在同一个子网里，如果在同一个子网里，可以使用 ARP 协议获取到目的主机的 MAC 地址，如果不在一个子网里，那么请求应该转发给网关，由它代为转发，此时同样可以通过 ARP 协议来获取网关的 MAC 地址，此时目的主机的 MAC 地址应该为网关的地址。
 - ▶ 建立 TCP 连接：主机将使用目标 IP 地址和目标 MAC 地址发送一个 TCP SYN 包，请求建立一个 TCP 连接，然后交给路由器转发，等路由器转到目标服务器后，服务器回复一个 SYN-ACK 包，确认连接请求。然后，主机发送一个 ACK 包，确认已收到服务器的确认，然后 TCP 连接建立完成。
 - ▶ HTTPS 的 TLS 四次握手：如果使用的是 HTTPS 协议，在通信前还存在 TLS 的四次握手。
 - ▶ 发送 HTTP 请求：连接建立后，浏览器会向服务器发送 HTTP 请求。请求中包含了用户需要获取的资源的信息，例如网页的 URL、请求方法（GET、POST 等）等。
 - ▶ 服务器处理请求并返回响应：服务器收到请求后，会根据请求的内容进行相应的处理。例如，如果是请求网页，服务器会读取相应的网页文件，并生成 HTTP 响应。

- 建立 HTTP 连接后，请求是如何传递到后端程序的？
 - 客户端构造 HTTP 请求：浏览器通过 TCP 将这个请求字节流发送给服务器的端口。
 - 服务器监听某个端口（如 80 或 443）：
 - Web 服务器软件（如 Nginx、Apache、Tomcat、Node.js 的 HTTP 模块）接收到 TCP 数据流。
 - Web 容器（Web Container）是一个运行 Web 应用的服务器环境，它：
 - 负责 监听端口（如 8080）；
 - 负责 接收 HTTP 请求；
 - 负责 创建/管理 Servlet 对象；
 - 负责 调度生命周期方法
 - 解析 TCP 数据流成 HTTP 请求报文：
 - 请求行（方法、路径、HTTP 版本）
 - 请求头（Host、Cookie、User-Agent...）
 - 请求体（POST/PUT 的 JSON、表单、文件等）
 - 封装为 `HttpServletRequest` 和 `HttpServletResponse` 对象
 - 找到要处理的 Servlet（如 `DispatcherServlet`）
 - 调用 Servlet 的 `service()` 方法
 - 拦截器会在进入你的业务逻辑（Controller）之前，拿到请求头（如 Token、Cookie、Session 等）进行权限校验。
 - `DispatcherServlet` 的任务是接管所有进入应用的 HTTP 请求，然后根据配置分发给正确的 Controller。服务器根据 URL 路径 + HTTP 方法决定由哪个后端程序处理：
 - 在 Java 中：Servlet 容器（Tomcat、Jetty）根据 `web.xml` 或注解 `@RequestMapping` 匹配到具体 Controller。
- SYN 泛洪攻击
 - SYN 攻击，又称 SYN Flood 攻击，是一种常见的 DoS（Denial of Service，拒绝服务）攻击，主要针对 TCP 协议的三次握手过程。
 - SYN 攻击就是利用这个三次握手的过程
 - 攻击者发送大量 伪造源 IP 的 SYN 报文给服务器。
 - 服务器收到后，为每个 SYN 报文分配资源（TCP 半连接队列）并发送 SYN-ACK。
 - 因为源 IP 是伪造的，服务器永远收不到最后的 ACK。
 - 半连接队列被占满后，正常用户就无法建立连接了。
 - 防护方法
 - SYN Cookies
 - 不立即分配资源，而是通过 加密算法生成一个 cookie 给客户端，等客户端返回 ACK 时再分配资源。
 - 增加半连接队列大小
 - 降低 SYN 超时时间
 - 减少服务器为未完成握手保留资源的时间。
 - 防火墙和入侵防护系统
 - 流量清洗/限速
 - 半连接队列
 - 在 TCP 三次握手中，服务器接收到客户端的 SYN 后，会做两件事：
 - 生成一个 TCB（Transmission Control Block，传输控制块），用于保存这个连接的状态信息

- 客户端 IP 和端口
 - 服务端分配的初始序列号
 - 当前握手状态 (SYN_RECEIVED)
- 把这个 TCB 放入半连接队列 (SYN queue / backlog queue)
 - 这时候连接还没有完全建立，服务器在等待客户端发送最后的 ACK 来完成握手。
- 半连接队列里存的不是报文本身，而是连接状态信息 (TCB)。
- 当客户端返回 ACK 时，服务器就把这个 TCB 移入全连接队列，表示连接正式建立。
- DNS 劫持和 DNS 污染
 - DNS 劫持是攻击者或中间人通过篡改用户 DNS 解析请求，强行将域名解析到非真实 IP 地址的行为。
 - 影响范围局部 (单个用户/网络)
 - 攻击位置在用户端、路由器、ISP
 - 目的是为了钓鱼、广告、信息窃取
 - 表现：被引导到恶意网站
 - DNS 污染是通过向 DNS 解析过程中注入错误信息，让域名解析结果“污染”，使用户获取错误 IP。
 - 影响范围大规模 (ISP 或全网)
 - 攻击位置在 DNS 服务器或网络中间
 - 目的是封锁访问、干扰通信
 - 表现：访问失败或错误 IP
- 锁升级
 - 一开始某个临界区是无锁状态，如果有线程进入拿锁，就会先判断是否是偏向锁状态，如果不是就加当前线程的偏向锁，也就是把 markword 中的 id 设置为当前线程 id。如果是偏向锁状态，就会判断 markword 的 id 和当前线程是否相同
 - 相同，进入临界区
 - 不相同，产生竞争
 - 若旧线程不再运行，将偏向锁转移
 - 如果此时旧线程还在运行，撤离偏向锁，升级为轻量级锁，轻量级锁采用 CAS 乐观锁，线程不断 CAS 自旋，直到拿到锁。如果 CAS 失败，说明竞争激烈，需要升级为重量级锁
 - 重量级锁未竞争到的锁不再自旋，而是直接挂起，减少 cpu 消耗
- JUC 包含哪些核心包？
 - 并发工具类 (Thread Utilities / Synchronizers)
 - CountdownLatch 允许一个或多个线程等待其他线程完成操作。
 - CyclicBarrier 让一组线程互相等待，直到到达屏障点。
 - Semaphore 控制同时访问资源的线程数量。
 - Exchanger 两个线程之间交换数据。
 - Phaser 灵活的分阶段线程同步器，比 CyclicBarrier 更强大。
 - 锁与同步 (Locks)
 - ReentrantLock：可重入互斥锁。
 - ReentrantReadWriteLock：读写锁，允许多个线程读或单个线程写。
 - StampedLock：支持乐观读锁、写锁等。
 - LockSupport：线程阻塞和唤醒工具。
 - Condition：配合 Lock 使用的等待/通知机制。
 - 并发集合 (Concurrent Collections)
 - ConcurrentHashMap：线程安全的哈希表。

- ConcurrentSkipListMap / ConcurrentSkipListSet: 线程安全的跳表实现。
- CopyOnWriteArrayList / CopyOnWriteArraySet: 写时复制的集合, 适合读多写少场景。
- BlockingQueue 接口及实现类: ArrayBlockingQueue、LinkedBlockingQueue、PriorityBlockingQueue、DelayQueue、SynchronousQueue 等。
- ▶ 原子操作 (Atomic Variables)
 - AtomicInteger / AtomicLong / AtomicBoolean
 - AtomicReference / AtomicReferenceArray / AtomicMarkableReference / AtomicStampedReference
- ▶ 线程池与任务执行 (Executors)
 - Executor / ExecutorService / ScheduledExecutorService: 线程池和任务提交接口。
 - ThreadPoolExecutor / ScheduledThreadPoolExecutor: 线程池具体实现。
 - Executors 工厂类: 提供常用线程池创建方法。
 - Future / FutureTask / Callable / Runnable: 任务提交与返回结果。
- ▶ 并发工具方法
 - ForkJoinPool: 分治任务的线程池实现。
 - RecursiveTask / RecursiveAction: Fork/Join 框架的任务。
 - TimeUnit: 时间单位枚举, 方便线程睡眠和超时控制。
- 线程的创建方式有哪些
 - ▶ 继承 Thread 类
 - 直接继承 Thread 并重写 run() 方法。
 - 调用 start() 方法会自动启动新线程并执行 run()。
 - 缺点: Java 不支持多继承, 继承 Thread 后不能再继承其他类。
 - ▶ 实现 runnable 接口
 - 把任务逻辑和线程对象分离, 更符合面向对象思想。
 - 可以共享同一个 Runnable 对象, 实现多线程共享资源。
 - ▶ 使用 匿名内部类 或 Lambda 表达式
 - 实际底层还是实现了 Runnable。
 - ▶ 实现 Callable 接口 + FutureTask
 - 如果你需要 线程有返回值 或 抛出异常, 必须使用 Callable。
- 怎么保证 a, b 两个线程顺序执行
 - ▶ 使用 Thread.join()
 - join() 会让当前线程等待目标线程执行完成。

```

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Thread a = new Thread(() -> {
            System.out.println("A 开始执行");
            try { Thread.sleep(1000); } catch (InterruptedException ignored) {}
            System.out.println("A 执行结束");
        });

        Thread b = new Thread(() -> {
            System.out.println("B 开始执行");
            System.out.println("B 执行结束");
        });

        a.start();    // 启动A
        a.join();     // 等待A执行完毕
        b.start();    // 再启动B
    }
}

```

```
    }  
}
```

- ▶ 使用 CountdownLatch
 - CountdownLatch 内部有一个计数器。
 - b 调用 await() 时阻塞。
 - 当 a 调用 countDown() 后计数器为 0, b 解除阻塞执行。
- ▶ 使用 Future / ExecutorService
 - Future.get() 会阻塞直到任务完成, 因此可以确保 A 完成后再执行 B。

```
import java.util.concurrent.*;  
  
public class Main {  
    public static void main(String[] args) throws Exception {  
        ExecutorService executor = Executors.newFixedThreadPool(2);  
  
        Future<?> futureA = executor.submit(() -> {  
            System.out.println("A 执行中...");  
            try { Thread.sleep(1000); } catch (InterruptedException ignored) {}  
            System.out.println("A 执行完毕");  
        });  
  
        // 等待A执行完再执行B  
        futureA.get();  
        executor.submit(() -> System.out.println("B 执行中...")).get();  
  
        executor.shutdown();  
    }  
}
```

- ▶ 使用 ReentrantLock + Condition
 - B 调用 lock.Lock()后执行 condition.await()。
 - A 执行完后调用 condition.signal()唤醒 B。
- ▶ 只有 lock 不行

```
public class test3 {  
  
    static Lock lock = new ReentrantLock();  
  
    public static void main(String[] args) {  
        Thread a = new Thread(() -> {  
            lock.lock();  
            try {  
                System.out.println("A 执行中...");  
                Thread.sleep(1000);  
  
                } catch (InterruptedException e) {  
                    throw new RuntimeException(e);  
                } finally {  
                    lock.unlock();  
                }  
        });  
  
        Thread b = new Thread(() -> {
```

```

        lock.lock();
        try {

            System.out.println("B 执行中...");
        }
        finally {
            lock.unlock();
        }
    });

    a.start();
    b.start();

}

}

```

- 如这段代码，a 和 b 线程虽然启动顺序是 a 先 b 后，但是 lock 锁并不能保证 a 先执行完再执行 b，start() 只是通知 JVM 调度器去启动线程；实际上 A、B 何时真正运行是由操作系统线程调度决定的；
- 线程池原理及拒绝策略
 - 介绍一下线程池的原理
 - 线程池设定了核心线程数，和最大线程数，如果任务进来
 - 核心线程没有满，就新建核心线程执行任务
 - 如果核心线程满了，则放入阻塞队列，阻塞队列中的任务等待核心线程空闲后执行
 - 如果阻塞队列满了，如果没有达到最大线程数，就要新建线程执行新提交的任务（注意不是队列里的）
 - 如果达到了最大线程数，就要执行拒绝策略
 - 线程池的参数
 - corePoolSize：线程池核心线程数量。默认情况下，线程池中线程的数量如果 $\leq$ corePoolSize，那么即使这些线程处于空闲状态，那也不会被销毁。
 - maximumPoolSize：限制了线程池能创建的最大线程总数（包括核心线程和非核心线程），当 corePoolSize 已满并且尝试将新任务加入阻塞队列失败（即队列已满）并且当前线程数 $<$ maximumPoolSize，就会创建新线程执行此任务，但是当 corePoolSize 满并且队列满并且线程数已达 maximumPoolSize 并且又有新任务提交时，就会触发拒绝策略。
 - keepAliveTime：当线程池中线程的数量大于 corePoolSize，并且某个线程的空闲时间超过了 keepAliveTime，那么这个线程就会被销毁。
 - unit：就是 keepAliveTime 时间的单位。
 - workQueue：工作队列。当没有空闲的线程执行新任务时，该任务就会被放入工作队列中，等待执行。
 - threadFactory：线程工厂。可以用来给线程取名字等等
 - handler：拒绝策略。当一个新任务交给线程池，如果此时线程池中有空闲的线程，就会直接执行，如果没有空闲的线程，就会将该任务加入到阻塞队列中，如果阻塞队列满了，就会创建一个新线程，从阻塞队列头部取出一个任务来执行，并将新任务加入到阻塞队列末尾。如果当前线程池中线程的数量等于 maximumPoolSize，就不会创建新线程，就会去执行拒绝策略
 - 线程池工作队列满了有哪些拒绝策略？

- CallerRunsPolicy, 使用线程池的调用者所在的线程去执行被拒绝的任务, 除非线程池被停止或者线程池的任务队列已有空缺。
- AbortPolicy, 直接抛出一个任务被线程池拒绝的异常。
- DiscardPolicy, 不做任何处理, 静默拒绝提交的任务
- DiscardOldestPolicy, 抛弃最老的任务, 然后执行该任务。
- 自定义拒绝策略
- 阻塞队列有哪些
 - 有界队列
 - 基于数组实现
 - 有固定大小
 - 可选 公平/非公平 锁;
 - 常用于需要限制队列大小的场景, 防止内存溢出。
 - LinkedListQueue
 - 基于 链表 实现;
 - 默认容量为 Integer.MAX_VALUE (几乎无界)
 - 可通过构造函数指定容量。
 - 因为几乎无界, 用于高并发场景, 吞吐量高。
 - PriorityBlockingQueue
 - 无界优先级队列
 - 元素按 自然顺序 或 Comparator 排序;
 - 不保证 FIFO 顺序。
 - 内部使用 堆结构 (PriorityQueue)。
 - 适合需要优先级处理的任务场景。
 - DelayQueue
 - 基于 PriorityQueue 实现;
 - 队列中的元素必须实现 Delayed 接口;
 - 只有到期 (延时结束) 的元素才能被取出。
 - 适合定时任务、缓存过期等场景。
 - SynchronousQueue
 - 容量为 0 的队列;
 - 每次 put() 必须等待一个 take();
 - 不能缓存元素。
 - 适合直接交付任务的场景, 如线程池中的直连交付。
 - LinkedListDeque
 - 双向链表实现的 双端阻塞队列;
 - 支持在队头、队尾插入/移除;
 - 适合需要双端操作的场景, 如工作窃取算法。
 - LinkedTransferQueue
 - 无界的高性能队列;
 - 支持 直接传递 (transfer) 给消费者;
 - 性能优于 LinkedListQueue;
 - 内部基于 CAS + 链表;
 - 常用于高并发场景。
- 如何创建线程池
 - 使用 Executors 工具类


```
ExecutorService pool = Executors.newFixedThreadPool(10);
```
 - Executors 默认使用的队列往往是无界队列容易导致溢出

‣ 手动创建 ThreadPoolExecutor

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(  
    corePoolSize,        // 核心线程数  
    maximumPoolSize,     // 最大线程数  
    keepAliveTime,       // 非核心线程空闲存活时间  
    TimeUnit.SECONDS,     // 时间单位  
    new ArrayBlockingQueue<>(100), // 阻塞队列  
    Executors.defaultThreadFactory(), // 线程工厂  
    new ThreadPoolExecutor.AbortPolicy() // 拒绝策略  
);
```

‣ CPU 密集型任务 核心线程数 = CPU 核数 + 1

‣ IO 密集型任务 核心线程数 = 2 × CPU 核数

• Java 常用线程池哪些？

‣ 线程池的核心类：ThreadPoolExecutor,所有线程池的本质都是这个类：

‣ 常用线程池类型（Executors 工厂方法）

- 固定线程数线程池，固定 n 个核心线程，任务多时排队等待
- 单线程线程池，只有一个线程，顺序执行任务
- 缓存线程池，线程数不固定，可自动回收空闲线程
- 定时任务线程池，定时或周期性任务执行
- 单线程定时任务池，单线程版本的定时线程池

‣ 自定义线程池（推荐方式）阿里巴巴《Java 开发手册》不建议直接使用 Executors 创建线程池，因为容易出现资源耗尽（OOM）风险

• 线程池和直接 NEW 线程的区别是什么

‣ 直接 new Thread() = 每次都新建线程，用完就扔。

‣ 线程池 = 创建一组可复用线程，任务结束后线程不会销毁，而是留着继续执行其他任务。

‣ 性能差异

- 直接 new 线程：频繁创建和销毁线程，开销大，性能低。
- 线程池：线程复用，减少创建销毁开销，性能更好。

‣ 资源管理

- 直接 new 线程：无法控制线程数量，可能导致资源耗尽。
- 线程池：可以设置最大线程数，合理利用系统资源。

‣ 任务调度

- 直接 new 线程：任务提交后立即执行，无法排队。
- 线程池：可以使用阻塞队列排队任务，按顺序执行。

‣ 系统稳定性

- 直接 new 线程：大量线程可能导致系统不稳定。
- 线程池：通过限制线程数，提高系统稳定性。

‣ 代码维护

- 直接 new 线程：代码分散，难以维护。
- 线程池：集中管理线程，代码更清晰易维护。

• JVM 内存模型

‣ JVM 的内存结构主要分为以下几个部分：

- 程序计数器：可以看作是当前线程所执行的字节码的行号指示器，用于存储当前线程正在执行的 Java 方法的 JVM 指令地址。如果线程执行的是 Native 方法，计数器值为 null。是唯一一个在 Java 虚拟机规范中没有规定任何 OutOfMemoryError 情况的区域，生命周期与线程相同。

- Java 虚拟机栈：每个线程都有自己独立的 Java 虚拟机栈，生命周期与线程相同。每个方法在执行时都会创建一个栈帧，用于存储局部变量表、操作数栈、动态链接、方法出口等信息。可能会抛出 `StackOverflowError` 和 `OutOfMemoryError` 异常。
 - 本地方法栈：与 Java 虚拟机栈类似，主要为虚拟机使用到的 Native 方法服务，在 HotSpot 虚拟机中和 Java 虚拟机栈合二为一。本地方法执行时也会创建栈帧，同样可能出现 `StackOverflowError` 和 `OutOfMemoryError` 两种错误。
 - Java 堆：是 JVM 中最大的一块内存区域，被所有线程共享，在虚拟机启动时创建，用于存放对象实例。从内存回收角度，堆被划分为新生代和老年代，新生代又分为 Eden 区和两个 Survivor 区（From Survivor 和 To Survivor）。如果在堆中没有内存完成实例分配，并且堆也无法扩展时会抛出 `OutOfMemoryError` 异常。
 - 方法区（元空间）：在 JDK 1.8 及以后的版本中，方法区被元空间取代，使用本地内存。用于存储已被虚拟机加载的类信息、常量、静态变量等数据。虽然方法区被描述为堆的逻辑部分，但有“非堆”的别名。方法区可以选择不实现垃圾收集，内存不足时会抛出 `OutOfMemoryError` 异常。
 - 运行时常量池：是方法区的一部分，用于存放编译期生成的各种字面量和符号引用，具有动态性，运行时也可将新的常量放入池中。当无法申请到足够内存时，会抛出 `OutOfMemoryError` 异常。
 - 直接内存：不属于 JVM 运行时数据区的一部分，通过 NIO 类引入，是一种堆外内存，可以显著提高 I/O 性能。直接内存的使用受到本机总内存的限制，若分配不当，可能导致 `OutOfMemoryError` 异常。
- 字符串常量池在哪
 - JDK 6 及以前在永久代
 - JDK 7 及以后在堆，jdk8 以后永久代删除
 - 为什么从永久代移到堆
 - 在 JDK 6 时，字符串常量池放在永久代，容易导致：
 - 类加载多、字符串常量多时，永久代溢出（OOM: PermGen space）
 - 回收效率差，因为永久代由 Full GC 才能清理
 - JDK 7 把它挪到堆中，带来好处：
 - 堆空间更大、可调节；
 - GC 更灵活（年轻代 GC、CMS、G1 都能处理字符串对象）；
 - 避免因字符串过多导致永久代溢出。
 - 堆的结构：新生代（Young Gen）+ 老年代（Old Gen）
 - 新生代是大多数新对象被创建的地方。它又分为三块：
 - Eden 区：新对象首先分配在这里。当 Eden 区满时，会触发 Minor GC，将存活的对象移动到 Survivor 区。
 - Survivor 区：分为 From Survivor（S0）和 To Survivor（S1）两个区域。
 - S0 存放上次 GC 后仍然存活的对象
 - S1 轮流作为目标 Survivor 区，存放从 Eden/S0 复制过来的对象
 - 比例默认约为 8:1:1（可通过 `-XX:SurvivorRatio=8` 调整）。
 - Minor GC 流程
 - 对象创建在 Eden；
 - 当 Eden 区满时，会触发一次 Minor GC（新生代垃圾回收）。Stop-The-World
 - GC 会扫描
 - 存活下来的对象会被移动到其中一个 Survivor 空间，存活次数（Age）+1
 - 这两个区域轮流充当对象的中转站，帮助区分短暂存活的对象和长期存活的对象。
 - 若对象在多次 Minor GC 后仍然存活（超过 `MaxTenuringThreshold` 次，默认 15），则晋升到老年代；

- 老年代中的对象生命周期较长，因此 Major GC（也称为 Full GC，涉及老年代的垃圾回收）发生的频率相对较低，但其执行时间通常比 Minor GC 长。Stop-The-World
- 对象晋升机制（Tenuring）
 - 每个对象在 Survivor 区都会有一个“年龄（Age）”计数器：
 - 每经历一次 Minor GC 并存活下来，Age +1；
 - 当 Age ≥ MaxTenuringThreshold 时，晋升到老年代；
 - 也可能因为 Survivor 区空间不足而提前晋升（称为“空间担保”）。
 - 大对象区（Large Object Space / Humongous Objects）：在某些 JVM 实现中（如 G1 垃圾收集器），为大对象分配了专门的区域，称为大对象区或 Humongous Objects 区域。大对象是指需要大量连续内存空间的对象，如大数组。这类对象直接分配在老年代，以避免因频繁的老年代晋升而导致的内存碎片化问题。
- Survival 区可以怎么优化
 - Survivor 区（又叫 Survival 区，S0/S1）是新生代（Young Generation）的一部分，用来在 Minor GC 时保存从 Eden 区“幸存”的对象。
 - 优化 Survivor 区的意义在于：减少对象过早晋升到老年代（Old Gen），从而减轻 Full GC 压力。
 - 优化方向一：调整 Survivor 区大小比例
 - -XX:SurvivorRatio=8
 - 表示 Eden : Survivor = 8 : 1 : 1
 - 即新生代共 10 份，Eden 占 8，两个 Survivor 各占 1。
 - 默认是 8，可以适当调大或调小：
 - Survivor 太小会导致对象直接晋升老年代
 - Survivor 太大造成 Eden 区浪费、GC 频繁
 - 优化方向二：控制对象晋升阈值（Tenuring）
 - -XX:MaxTenuringThreshold=15
 - 对象在 Survivor 区经历几次 GC 后会晋升老年代。
 - 默认 15（最大值），有些 GC（如 G1）会自动调整。
 - 短命对象多（大量瞬时对象）设小一点，比如 3 5 快速回收这些对象
 - 中等生命周期对象多设大一点，比如 10 15 延迟晋升，减少老年代压力
 - 优化方向三：启用动态年龄判定机制
 - JVM 可能根据 Survivor 的使用情况自动调整对象晋升年龄：
 - -XX:+UseAdaptiveSizePolicy
 - 为什么短命对象多要设置“小阈值”
 - 短命对象虽然很快会死，但：
 - 它们第一次 GC 幸存下来（复制进 Survivor）
 - 第二次 GC 又被扫描、复制
 - 反复经历多次 Survivor → Survivor 的拷贝
 - 每次都要复制存活对象、扫描引用，浪费 CPU。
 - 当阈值小（比如 3）
 - 对象最多经历 3 次 Minor GC；
 - 很多中期对象在第 3 次就晋升老年代；
 - 短命对象在 1 2 次 GC 内就会被清理，不会反复复制。降低 Minor GC 停顿时间，提升吞吐量
- 元空间参与垃圾回收吗，回收什么
 - 参与，但方式不同于 Java 堆。
 - 元空间中的对象（例如类元数据）不是 GC 的主要目标，因为它们只有在类被卸载时才可能被清理。

- 当类被卸载（class unloading）时，其对应的元数据才会被 GC 释放。
- 也就是说：元空间的回收实质上是类卸载的过程。
- 什么情况下类会被卸载？
 - 该类的所有实例都被回收；
 - 加载该类的 ClassLoader 被回收；
 - 没有任何地方再引用该 Class 对象。
- 元空间回收的具体内容
 - 类的元数据（metadata）
 - 方法信息（method info）
 - 字段信息（field info）
 - 常量池
 - 方法表、接口表
 - 运行时注解信息
 - ClassLoader 相关的元数据
- MAJORGC 和 FULLGC 不一样
 - MAJORGC 只清理老年代
 - FULLGC 清理整个堆，包括新生代和老年代，有时还包括方法区

⚙️ 2. Major GC

- 触发条件：老年代空间不足；
- 清理目标：老年代；
- 使用算法：标记-清除 / 标记-整理；
- 特点：耗时比 Minor GC 长；
- 是否同时清理新生代？
 - 视收集器而定（有的会顺便触发 Minor GC，有的不会）。

👉 举个例子：

- CMS 收集器中有明确的“Major GC”阶段（主要清理老年代）；
- 而 G1、Parallel GC 中几乎没有单独的“Major GC”，它们直接触发“Full GC”。

🔴 3. Full GC

- 触发条件：
 - 老年代空间不足；
 - Metaspace（方法区）空间不足；
 - 显式调用 `System.gc()`；
 - 进行 GC 时发现引用处理、空间担保失败；
- 清理目标：
 - 新生代 + 老年代 + 方法区；
- 特点：
 - Stop-The-World（暂停所有线程）；



- 垃圾回收算法
 - 标记清除
 - old
 - 简单
 - 碎片多
 - 从 GCroot 开始，标记所有可达的对象，清除所有没有被标记的对象
 - 标记整理
 - old
 - 移动成本高
 - 避免碎片
 - 在标记清除上改进，标记存活对象，将存活对象往一端移动
 - 复制算法
 - young
 - 无碎片，效率高
 - 空间浪费一半
 - 将内存分为两块（From 和 To）；每次只使用其中一块；回收时，把存活对象复制到另一块，然后清空旧区。
 - 分代收集
 - 高性能
 - 实现复杂
 - 划分为新生代，老年代，元空间
 - 新生代 minorGC 回收 Eden + Survivor，老年代 fullgc 或 majorgc 全堆标记整理（非常慢），Mixed GC（G1）“混合回收”一部分老年代（高垃圾比例的 Region）+ 新生代的所有 Region。
- 判断垃圾算法
 - 引用计数算法
 - 每个对象都有一个引用计数器；
 - 每当有一个地方引用它，计数 +1；
 - 引用失效时，计数 -1；
 - 当计数变为 0 时，对象被认为是垃圾。
 - 缺点：
 - 无法处理循环引用问题：A -> B, B -> A 相互引用，引用计数都不为 0，但都不可达
 - 多线程下计数操作成本高。
 - 维护计数器开销大；
 - 可达性分析算法
 - 从一组称为 GC Roots（根对象）的起始点出发；
 - 通过引用关系向下搜索；
 - 能被 GC Roots 直接或间接引用的对象为“存活对象”；
 - 没被连接到任何 GC Roots 的对象被视为“垃圾”。
 - GC Roots 通常包括：
 - 虚拟机栈中引用的对象（局部变量等）；
 - 方法区中静态变量引用的对象；
 - JNI 引用的对象（native 方法）；
 - 活动线程对象本身。
 - 被标记为“不可达”并不一定马上回收，GC 会再做两次判断：
 - 是否覆写了 finalize() 方法；
 - 如果有，则会给它一次“自救”的机会（即在 finalize() 中重新建立引用）；

- 如果仍然不可达，则被认定为真正的垃圾。
- 即使对象“不可达”，也可能暂时被保留，例如：
 - 被 SoftReference / WeakReference 引用；
 - 软引用（SoftReference）
 - 内存不足时回收
 - 弱引用（WeakReference）
 - 下一次 GC 就回收
 - 虚引用（PhantomReference）
 - 随时可回收，不可直接访问对象
 - 处于老年代，GC 未触发；
 - 仍在 finalize() 执行期间。
- 垃圾收集器
 - Serial 收集器（复制算法）：新生代单线程收集器，标记和清理都是单线程，优点是简单高效；
 - ParNew 收集器（复制算法）：新生代多线程收集器，实际上是 Serial 收集器的多线程版本，在多核 CPU 环境下有着比 Serial 更好的表现；
 - 注重响应速度，降低停顿时间，适合交互式应用
 - Parallel Scavenge 收集器（复制算法）：新生代并行收集器，追求高吞吐量，高效利用 CPU。吞吐量 = 用户线程时间 / (用户线程时间 + GC 线程时间)，高吞吐量可以高效率的利用 CPU 时间，尽快完成程序的运算任务，适合后台应用等对交互相应要求不高的场景；
 - 注重高吞吐量，但是停顿长，适合计算密集型任务
 - Serial Old 收集器（标记-整理算法）：老年代单线程收集器，Serial 收集器的老年代版本；
 - Parallel Old 收集器（标记-整理算法）：老年代并行收集器，吞吐量优先，Parallel Scavenge 收集器的老年代版本；
 - CMS(Concurrent Mark Sweep)收集器（标记-清除算法）：老年代并行收集器，以获取最短回收停顿时间为目标的收集器，具有高并发、低停顿的特点，追求最短 GC 回收停顿时间。
 - G1(Garbage First)收集器（标记-整理算法）：Java 堆并行收集器，G1 收集器是 JDK1.7 提供的一个新收集器，G1 收集器基于“标记-整理”算法实现，也就是说不会产生内存碎片。此外，G1 收集器不同于之前的收集器的一个重要特点是：G1 回收的范围是整个 Java 堆(包括新生代，老年代)
 - 以上垃圾收集器的标记都采用位图
 - ZGC
 - 传统 GC 问题
 - 传统 GC（如 G1）在标记对象时，需要额外的“标记位图”；
 - 复制对象后，必须“Stop The World”修正所有引用；
 - 堆越大，停顿越久
 - ZGC 的做法
 - ZGC 把“标记信息”直接放在 对象指针的高位 上（称为 染色指针）：
 - bit 0 42 实际内存地址
 - bit 43 45 GC 标记状态（颜色）
 - bit 46 63 元数据（代信息、压缩状态等）
 - 这样，JVM 不需要停止程序去修正指针，而是通过读取染色位就知道对象状态。

染色指针结构（简化）

位区间	含义
bit 0–42	实际对象内存地址（可寻址 4TB）
bit 43–45	GC 标记颜色（白/灰/黑）
bit 46–47	压缩状态标记
bit 48–63	其他元信息（代信息、重定位标记等）

通过不同颜色，ZGC 能知道对象在 GC 流程中的状态：

颜色	含义
白色	未访问，对象可能要回收
灰色	已访问但字段未完全扫描
黑色	已访问且字段全部扫描完毕
红色（或特殊状态）	 正在移动（Relocating）

在 ZGC 的 **并发标记阶段 (Concurrent Marking)** 中，应用线程与 GC 线程是同时运行的。

这会导致一个典型的并发问题：
当 GC 线程扫描一个对象时，应用线程可能还在修改它的字段引用。

例如 📌

cpp

 复制代码

```
Thread A (应用线程): 改变 obj.field = newObj  
Thread B (GC 线程): 正在扫描 obj 的字段...
```

这意味着：

- GC 线程可能“看不到”最新的字段；
- 或“标记过了”一个对象，但其实它的某个字段还没来得及被扫描。

这就出现了你提到的现象：

“对象已被访问（marked as live），但字段未完全扫描。”

 1.

- 优点：
 - 所有对象引用都能被快速判断；
 - 对象移动、标记、重定位都可 并发进行；
 - 极大降低 STW 停顿时间。
- 工作流程

- 并发标记（Concurrent Mark）
 - 扫描对象图，标记存活对象；
 - 与应用线程并发运行；
 - 利用染色指针来标识对象状态。
- 并发重分配准备
 - 计算哪些区域可以压缩；
 - 构建移动计划
 - 不阻塞用户线程。
- 并发重分配（Relocate）
 - 将活跃对象移动到新的内存区域；
 - 更新染色指针；
 - 所有对象引用更新也可并发执行。
- 短暂的停顿阶段
 - 在阶段切换时会有极短暂停；用于同步根对象、更新线程栈等。

🍀 七、ZGC 与其他 GC 对比

特性	Serial / Parallel	CMS	G1	ZGC	Shenandoah 
是否并发	否	部分	部分	✅ 全并发	✅ 全并发
停顿时间	长	中	中	<1ms	低
吞吐率	高	中	高	高	中
支持大堆	一般	一般	32GB 左右	16TB	TB 级
碎片整理	否	否（可选）	✅	✅ 并发压缩	✅ 并发压缩
分代设计	是	是	是	JDK21起支持	是

- Shenandoah
 - 并发标记整理
 - 在移动后会在原地留下一个“前向指针”（Forwarding Pointer），指向新地址；
 - 应用线程访问对象时，如果发现前向指针，就会跳转到新地址；
 - 这样，应用线程可以并发访问对象，无需停顿等待
- CMS 的垃圾回收流程
 - CMS 的 GC 流程分为四个阶段，其中 并发阶段 是 CMS 的核心。
 - 阶段 1：初始标记（Initial Mark）
 - 类型：STW（Stop-The-World）停顿阶段
 - 作用：标记 GC Roots 直接可达的对象。
 - 特点：只标记根节点直接引用的对象，耗时很短。
 - 停顿原因：必须暂停用户线程来保证标记准确。
 - 阶段 2：并发标记（Concurrent Mark）
 - 类型：并发阶段，用户线程继续运行
 - 作用：从初始标记阶段标记的对象出发，递归标记整个可达对象图。扫描对象引用，标记所有可达对象。
 - 特点：与应用线程并发执行，时间相对较长，可能产生 浮动对象（应用线程创建的新对象未被标记）
 - 阶段 3：重新标记（Remark）

- 类型：STW 停顿阶段
- 作用：处理 并发标记阶段产生的浮动对象（并发阶段用户线程可能修改了对象引用），最终修正标记结果，保证标记准确。
- 特点：停顿时间比初始标记稍长，但仍比 Full GC 短。只处理少量的对象，因此停顿较短。
- ▶ 阶段 4：并发清理（Concurrent Sweep）
 - 类型：并发阶段
 - 作用：扫描整个老年代，将 未被标记的对象 回收。回收空间给新的对象使用。
 - 特点：与应用线程并发执行，回收完成后，堆中可能产生 内存碎片，CMS 默认不压缩。
- ▶ CMS 的注意点
 - 浮动垃圾：
 - 并发标记阶段用户线程可能创建对象或修改引用，会产生 少量浮动垃圾。
 - Remark 阶段处理大部分浮动垃圾，但可能仍有微量垃圾未及时回收。
 - 内存碎片：
 - CMS 默认 不压缩老年代，可能因为碎片导致无法分配大对象。
 - 当碎片严重时，可能触发 Full GC（Stop-The-World）进行压缩。
 - CPU 使用率：
 - 并发阶段占用 CPU，与应用线程竞争，可能影响应用性能。
 - 为什么新创建对象可能无法标记

2 为什么新创建对象可能无法标记

假设在并发标记阶段：

text

复制代码

Root → A → B

- CMS 从 Root 开始标记 A、B。
- 应用线程创建了一个新对象 C，并让 A 引用 C：

text

复制代码

Root → A → B

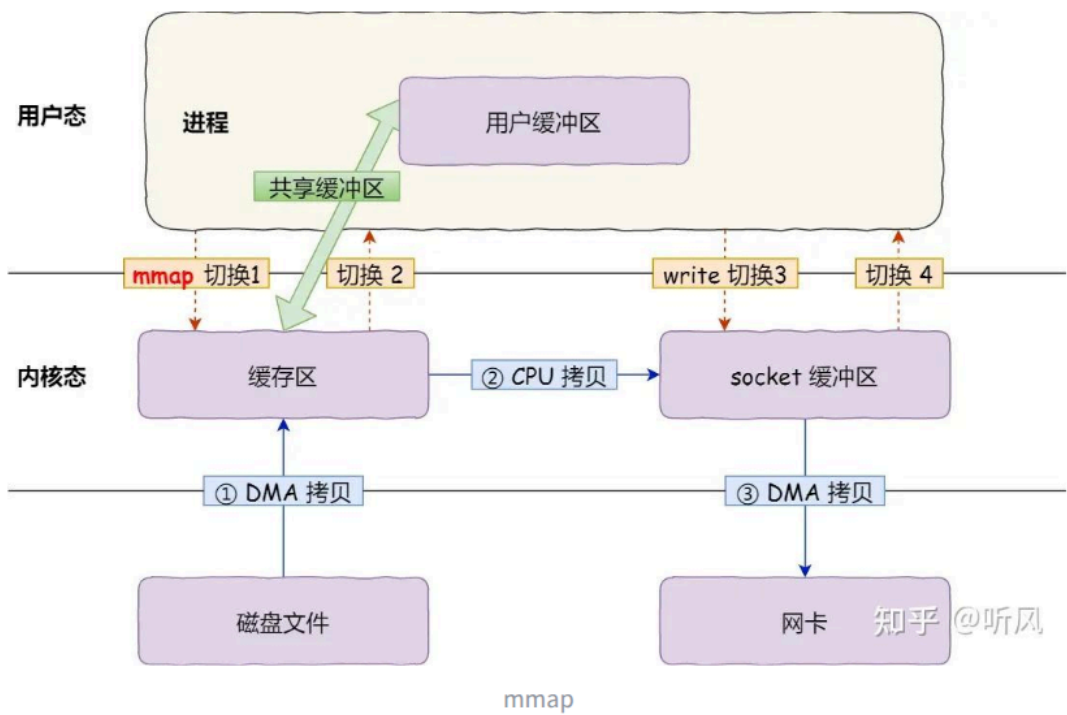
A → C （新对象）

- 由于并发标记线程已经扫描过 A，它 **不会再次扫描 A**，也就无法发现 C。
- 这个新对象 C 就会暂时 **未被标记**，成为所谓的 **浮动对象（floating garbage）**。



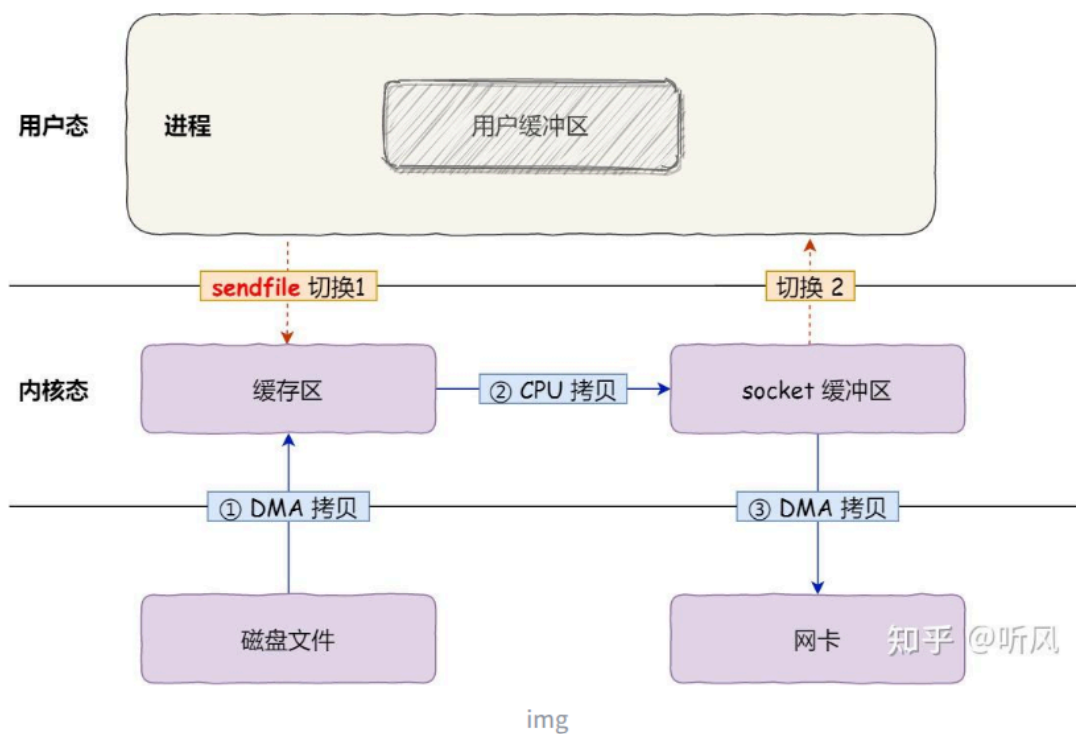
- 堆外内存
 - ▶ 堆外内存的特点
 - 不受 GC 管理，不参与垃圾回收，避免 GC 停顿对大对象的影响
 - 手动释放，使用 Cleaner 或 free 方法手动释放，否则可能内存泄漏
 - 访问速度快，避免堆内对象的复制，直接操作系统内存，尤其适合 I/O 或网络传输
 - 适用场景，大缓存（如 Redis、Netty ByteBuf）、内存映射文件、零拷贝 I/O、高性能系统
 - ▶ 堆外内存的用途
 - 减少 GC 压力
 - 大对象或大量临时对象如果放在堆上，会频繁触发 GC。
 - 将它们放在堆外，GC 不会扫描这些对象，减少 Full GC 停顿。
 - 零拷贝 I/O

- 网络通信或文件读写时，堆外内存可以直接映射系统内存，减少数据复制，提高性能。
- 传统 I/O 为什么“慢”
 - 假设我们要把一个文件发送到网络（比如发送给浏览器）
 - 有四次“拷贝”
 - 文件 → 内核缓冲区（DMA 读）
 - 内核缓冲区 → JVM 堆内存（read() 调用）
 - JVM 堆内存 → 内核 Socket 缓冲区（write() 调用）
 - Socket 缓冲区 → 网卡（DMA 发送）
 - 每次调用 read/write 都会在内核空间和用户空间（JVM）之间复制数据；
 - 数据量大时，CPU 拷贝负担很重；
 - 零拷贝（Zero-Copy）怎么优化这个过程？
 - 让操作系统自己在内核空间直接传递数据，不经过 JVM 堆。
 - 堆外内存映射的是操作系统的物理内存；
 - I/O 读写可以直接在内核态和这块堆外内存之间传输，不再复制到 JVM 堆。
- 缓存
 - 高性能缓存（如 Netty、DirectByteBuffer、Off-Heap Map）可以使用堆外内存，避免堆内大对象造成 GC 压力。
 - 堆外缓存（Off-Heap Cache）为什么更高效？
 - 如果你用 `HashMap<String, byte[]>` 存放缓存对象，那这些对象（key、value、`byte[]`）全在堆内，对象多、数据大，GC 会非常频繁（特别是 Full GC）。
 - 把缓存放到堆外 JVM 不再扫描这些数据；
- 零拷贝是什么
 - 零拷贝指在数据从一个位置传输到另一个位置的过程中，尽量避免在用户空间（User Space）和内核空间（Kernel Space）之间重复拷贝数据。
 - 零拷贝实现方式
 - mmap 映射文件
 - 使用 mmap 将文件映射到进程地址空间。
 - 文件在内核缓冲区中时，用户程序可以直接通过内存访问文件内容，而无需 read() 拷贝

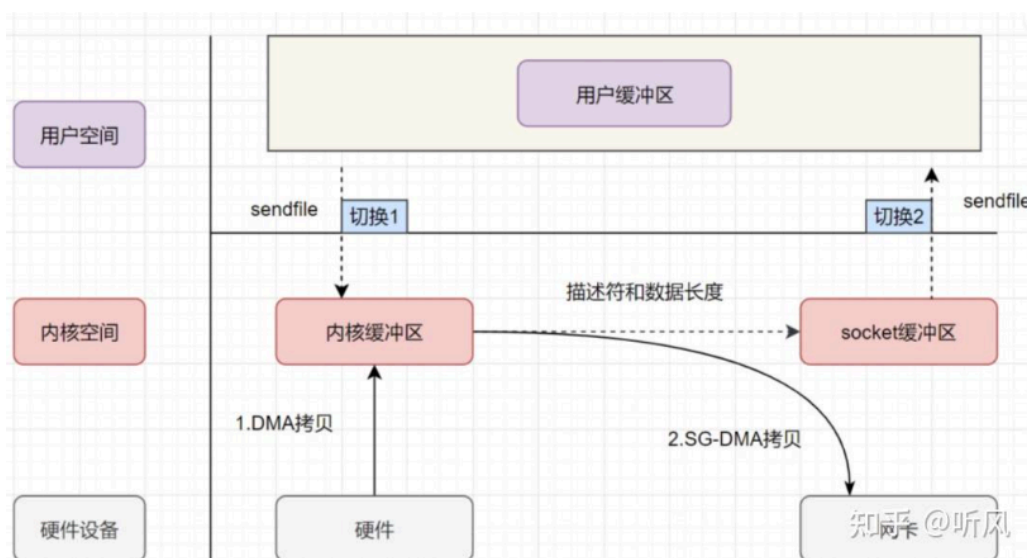


- `sendfile` 系统调用 (Linux)

- 常用于文件直接发送到网络套接字。
- 内核直接从文件缓冲区写入到网卡缓冲区，不需要拷贝到用户空间。
- `sendfile` 分两种
 - 普通 DMA



- 系统调用：用户进程通过 `sendfile` 系统调用发起文件传输请求。这一步需要从用户态切换到内核态。
 - DMA 传输数据到内核缓冲区：DMA 将磁盘上的数据拷贝到内核缓冲区。
 - 内核拷贝数据到网络缓冲区：内核将内核缓冲区的数据拷贝到网络协议栈的缓冲区。
 - DMA 传输数据到网卡：DMA 将网络协议栈的缓冲区数据拷贝到网卡。
 - 系统调用返回：系统调用返回，从内核态切换到用户态。
- `sendfile` + DMA Scatter/Gather



img

- 系统调用：用户进程通过 `sendfile` 系统调用发起文件传输请求。这一步需要从用户态切换到内核态。
 - DMA 传输数据到内核缓冲区：DMA 将磁盘上的数据拷贝到内核缓冲区。
 - DMA 根据描述信息传输数据到网卡：DMA 根据网络缓冲区中的数据描述信息，直接从内核缓冲区传输数据到网卡。
 - 系统调用返回：系统调用返回，从内核态切换到用户态。
- 网络协议栈优化
- 零拷贝也可以用于网络接收数据，例如通过 DMA（直接内存访问）将网卡数据直接写入内核缓冲区，然后直接映射给用户程序。

技术	数据拷贝次数	上下文切换次数	优点	缺点
传统I/O	4次	4次	简单易用	性能较差，频繁上下文切换和数据拷贝
mmap + write	3次	4次	减少了一次CPU拷贝	仍需上下文切换，适合大文件传输
sendfile	2次	2次	减少了上下文切换和CPU拷贝	需要硬件支持，适用场景有限
sendfile + DMA	2次	2次	完全避免CPU拷贝，性能最佳	依赖硬件支持，适用场景有限
splice	2次	2次	数据传输完全在内核空间完成，灵活性高	实现复杂，适用场景较广但效率稍逊
tee	2次	2次	数据可以同时发送到多个目标，灵活性高	实现复杂，适用场景较广但效率稍逊

- 什么是 DMA
 - DMA (Direct Memory Access, 直接内存访问) 是一种让外设 (如硬盘、网卡、声卡) 在不经过 CPU 干预的情况下直接读写内存的技术。它是实现零拷贝和高性能数据传输的重要手段之一。
- 什么是网卡缓冲区
 - 网卡是计算机的网络接口硬件，也就是外设。
 - 网卡内部有自己的缓冲区 (Buffer)，用于临时存储发送或接收的数据包。
 - 这个缓冲区位于网卡硬件里，不在主机内存中，因此严格意义上是“外设内存”。
- 什么是文件缓冲区
 - 文件缓冲区 (File Buffer) 是操作系统内核用来临时存放文件数据的一块内存区域，通常在内核空间中。
 - 当应用程序读取或写入文件时，数据不会直接从磁盘到用户程序，而是先经过这个缓冲区。
 - 作用类似“中转站”，减少磁盘 I/O 操作，提高性能。
 - 文件缓冲区的工作流程
 - 文件读取 (read)
 - 用户进程调用 read() 请求文件内容。
 - 操作系统检查文件缓冲区 (内核页缓存)
 - 如果缓冲区有数据 (命中缓存)，直接拷贝到用户空间。
 - 如果没有数据 (缓存未命中)，从磁盘读取到缓冲区，再拷贝到用户空间。
 - 用户程序使用缓冲区中的数据。
 - 文件写入 (write)
 - 用户进程调用 write() 把数据写入文件。
 - 数据先写入文件缓冲区 (内核空间)，不必立即写入磁盘。

- 内核后台线程或定时刷新（flush）将缓冲区数据写入磁盘。
- 方法区中方法执行过程
 - 解析方法调用：JVM 会根据方法的符号引用找到实际的方法地址（如果之前没有解析过的话）。
 - 栈帧创建：在调用一个方法前，JVM 会在当前线程的 Java 虚拟机栈中为该方法分配一个新的栈帧，用于存储局部变量表、操作数栈、动态链接、方法出口等信息。
 - 执行方法：执行方法内的字节码指令，涉及的操作可能包括局部变量的读写、操作数栈的操作、跳转控制、对象创建、方法调用等。
 - 返回处理：方法执行完毕后，可能会返回一个结果给调用者，并清理当前栈帧，恢复调用者的执行环境。

举个例子：

```
java
Animal a = new Dog();
a.speak();    // 调用哪个方法？
```

复制代码

- `Animal` 有一个 `speak()` 方法。
- `Dog` 重写了 `speak()` 方法。

当 JVM 执行 `a.speak()` 这条字节码时，字节码里其实只保存了一个符号引用：

“类 `Animal` 中名为 `speak`、描述符为 `()V` 的方法”。

但真正调用的是 `Dog.speak()`，因为运行时 `a` 实际指向一个 `Dog` 实例。

这个“根据实际类型在运行时确定目标方法地址”的过程 ↓ 就是 **动态链接（或称动态绑定 / 动态分派）**。

- JMM
 - JMM 是线程之间如何共享变量、保证可见性与有序性的抽象模型。它属于“并发语义”层面的内存模型，不等同于上面的运行时内存结构。
 - 主要关注三件事：
 - 可见性：一个线程对共享变量的修改，能否被其他线程看到。
 - 原子性：一个操作是否是不可分割的。
 - 有序性：程序执行的顺序是否与代码顺序一致。
 - JMM 定义了主内存（Main Memory）和工作内存（Working Memory）：
 - 主内存：存放所有共享变量。
 - 工作内存：每个线程都有一份主内存的拷贝（缓存），线程对变量的操作必须先在工作内存中进行，然后同步回主内存。
- 用户线程与内核线程
 - 用户线程
 - 由用户空间的线程库（如 Java 的线程库、POSIX pthread 等）管理，操作系统内核对此并不感知。
 - 用户线程的创建、调度、销毁等都是由用户态代码完成的，不需要系统调用。
 - 用 Java 创建 `new Thread()` 或 Python 的 `threading.Thread()`，实际上对应的是用户线程层面的抽象。
 - 内核线程（Kernel Thread, KLT）
 - 由操作系统内核直接管理。

- 内核会为每个内核线程分配 内核栈、线程控制块 (TCB)，并参与调度 (即由操作系统调度器负责运行)。
- 创建、销毁、上下文切换都要经过 系统调用。
- 用户线程和内核线程的映射模型
 - 1:1 模型 每个用户线程对应一个内核线程。
 - N:1 模型 多个用户线程映射到同一个内核线程，由用户线程库调度。
 - M:N 模型 M 个用户线程映射到 N 个内核线程 (混合模型)。
- JAVA 是哪种模型
 - Java 的线程在现代 JVM (HotSpot) 中采用 1:1 模型：
 - 优点：多核利用率高；
 - 缺点：线程数量受系统资源限制 (通常几千 上万线程后就不行了)。
- Java 对象在 JVM 内存中的创建过程，并详细说一下对象头
 - 假设代码：Foo f = new Foo(...);
 - 字节码执行遇到 new 指令；new 指令会在运行时常量池中查找要实例化的类 (引用常量)，如果类还未解析/加载，会触发类加载 (ClassLoader)、验证、准备、解析、初始化等阶段 (类初始化 在需要时执行——通常在第一次主动使用时触发)。
 - 若类尚未初始化，必须先执行类初始化 (静态字段/静态块)。
 - 分配内存 (Allocate) JVM 为新对象在堆上分配一块内存。HotSpot 的常见策略：
 - TLAB (Thread-Local Allocation Buffer)：线程本地的缓冲区，常规情况下对象直接在 TLAB 内用“bump-the-pointer” (指针推进) 方式分配 —— 非常快，无需加锁。
 - 若 TLAB 空间不足，则从堆的 Eden 区 (Young Gen) 申请或触发一次 GC (或去全局分配)，或者使用分配缓冲区 (allocation slow path)。
 - 逃逸分析 (JIT 优化) 可能导致对象在栈上分配 (栈上分配/消除) 或直接消除 (无对象分配)；但这是编译器优化，不改变语义。
 - 内存清零
 - JVM 要求分配的对象内存被零初始化 (所有字段为 0 / null / false / 0.0)。这样可以保证安全性。
 - 设置对象头 (mark word、klass pointer 等)
 - 在对象内存的起始位置写入对象头信息 (见下文对象头章节)，包括用于 GC、锁和类型指针的字段。对于数组对象还会写入长度字段。
 - 调用构造器
 - 分配并写入 this 引用后，JVM 会执行 构造方法 (先调用父类构造方法，完成字段初始化与构造逻辑)。
 - 在构造期间，this 指向的新对象对 Java 代码是可见的 (可能被泄露到其它线程——若泄露，后续需要注意内存可见性/同步等问题)。
 - 构造器返回，引用赋给变量
 - 构造器执行完毕后，new 指令返回对象引用，并由字节码 astore/赋值操作存入局部变量或字段。
 - 对象头 (Object Header) 详解
 - Mark Word (标记字)
 - 对象身份哈希码 (identity hashCode)：当 System.identityHashCode() 被调用时，哈希可能会被记录在这里 (若已计算)。
 - GC 元数据：用于标记 GC 状态 (年龄、分代信息、是否被标记/转发指针等)，垃圾回收时可能临时存放转发指针 (在某些 GC 算法中)。
 - 锁信息 (Locking bits)：用于对象监视器 (synchronized) 和锁优化
 - 状态标志位：例如表示锁的状态 (偏向/轻量/重量/无锁)、是否是数组、是否已被 GC 标记等等。

- Klass Pointer（类型指针 / klass word）——指向对象的类元数据（对象所属的类）（指向方法区/元空间中的类元信息）。
- 数组专用的 length 字段（仅数组对象）
- java 编译和运行流程
 - 编写源码
 - 你写一个 Java 文件，例如 HelloWorld.java
 - 这一步生成的是 源代码文件，扩展名是 .java。
 - 编译（Compile）
 - 使用 Java 编译器 javac：javac HelloWorld.java
 - 发生的事情：
 - 词法分析（Lexical Analysis）将源码拆分成一个个 token（关键字、标识符、符号等）。
 - 语法分析（Syntax Analysis）根据 Java 语法规则生成 抽象语法树（AST）。
 - 语义分析（Semantic Analysis）检查类型、方法调用、变量声明等是否合理。
 - 生成字节码（Bytecode）编译器将 AST 转换为 Java 字节码，存储在 .class 文件中。每个类对应一个 .class 文件，例如 HelloWorld.class。
 - .class 文件是与平台无关的中间代码。
 - 可以在任何安装了 JVM 的平台上运行。
 - 类加载（Class Loading）
 - 当你运行程序时：java HelloWorld
 - JVM 会做的事情：
 - 加载类（Loading）
 - ClassLoader 读取 .class 文件，加载到内存中。
 - 连接（Linking）
 - 验证（Verification）：确保字节码合法，不破坏内存安全。
 - 准备（Preparation）：为类的静态变量分配内存并赋默认值。
 - 解析（Resolution）：将符号引用转为直接引用。
 - 初始化（Initialization）
 - 为静态变量赋初始值，执行静态代码块。
 - 运行
 - JVM 的执行引擎（Execution Engine）运行字节码：
 - 解释执行（Interpreter）
 - JVM 逐条解释字节码执行。
 - 即时编译（JIT, Just-In-Time Compiler）
 - 将热点代码编译为本地机器码，提高性能。
- 类加载流程
 - 类从被加载到虚拟机内存开始，到卸载出内存为止，它的整个生命周期包括以下 7 个阶段：
 - 加载：通过类的全限定名（包名 + 类名），获取到该类的.class 文件的二进制字节流，将二进制字节流所代表的静态存储结构，转化为方法区运行时的数据结构，在内存中生成一个代表该类的 Java.lang.Class 对象，作为方法区这个类的各种数据的访问入口
 - 连接：验证、准备、解析 3 个阶段统称为连接。
 - 验证：确保 class 文件中的字节流包含的信息，符合当前虚拟机的要求，保证这个被加载的 class 类的正确性，不会危害到虚拟机的安全。验证阶段大致会完成以下四个阶段的检验动作：文件格式校验、元数据验证、字节码验证、符号引用验证
 - 文件格式校验是验证 .class 文件最基础的一步，主要用于 确保文件整体结构符合 JVM 规范，防止非法或损坏的文件被加载。
 - 魔数（Magic Number）检查文件前 4 个字节是否为 0xCAFEBADE。

- 版本号合法性检查，是否在 JVM 支持的版本范围内
- 常量池长度检查
- 元数据验证就是检查类结构、字段、方法和继承信息的合法性，保证：
 - 类的继承关系正确
 - 方法和字段合法
 - 访问权限安全
 - 常量池引用有效
- 字节码验证就是在逻辑上模拟 JVM 执行字节码，确保
 - 指令是合法的
 - 操作数类型正确
 - 栈操作安全
 - 控制流合法性 也就是检查跳转指令（goto, if, tableswitch, lookupswitch）是否安全：
 - 确定跳转目标在方法范围内。
 - 不会跳出方法边界。
 - 局部变量类型在跳转后仍然一致。
- 准备：为类中的静态字段分配内存，并设置默认的初始值，比如 int 类型初始值是 0。被 final 修饰的 static 字段不会设置，因为 final 在编译的时候就分配了
- 解析阶段是虚拟机将常量池的「符号引用」直接替换为「直接引用」的过程。符号引用是以一组符号来描述所引用的目标，符号可以是任何形式的字面量，只要使用的时候可以无歧义地定位到目标即可。直接引用可以是直接指向目标的指针、相对偏移量或是一个能间接定位到目标的句柄，直接引用是和虚拟机实现的内存布局相关的。如果有了直接引用，那引用的目标必定已经存在在内存中了。
- 初始化：初始化是整个类加载过程的最后一个阶段，初始化阶段简单来说就是执行类的构造器方法（`()`），执行类中定义的静态代码块和静态变量的赋值语句。，要注意的是这里的构造器方法（`()`）并不是开发者写的，而是编译器自动生成的。
- 使用：使用类或者创建对象
- 卸载：一个类要被 JVM 卸载，条件非常苛刻，需要同时满足以下三点：
 - 该类所有的实例都已经被回收：这是最显而易见的前提。如果堆中还存在这个类的任何一个实例对象，那么定义这个对象的 Class 对象肯定不能被卸载。
 - 加载该类的 ClassLoader 已经被回收：这是最关键也是最难满足的条件。类与其加载器是双向绑定的共生关系。一个类由哪个类加载器加载，这个信息是存储在 Class 对象里的。要卸载一个类，必须先卸载加载它的类加载器。
 - 类对应的 `java.lang.Class` 对象没有任何地方被引用：不能在任何地方通过反射（如静态字段、全局变量）、静态变量、JNI 等途径引用到这个 Class 对象。一旦这个 Class 对象还存在强引用，GC 就不会回收它，那么这个类也就不会被卸载。
- 类加载器与双亲委派机制
- 类加载器有哪些？
 - 启动类加载器（Bootstrap Class Loader）：这是最顶层的类加载器，负责加载 Java 的核心库（如位于 `jre/lib/rt.jar` 中的类），它是用 C++ 编写的，是 JVM 的一部分。启动类加载器无法被 Java 程序直接引用。
 - 扩展类加载器（Extension Class Loader）：它是 Java 语言实现的，继承自 `ClassLoader` 类，负责加载 Java 扩展目录（`jre/lib/ext` 或由系统变量 `Java.ext.dirs` 指定的目录）下的 jar 包和类库。扩展类加载器由启动类加载器加载，并且父加载器就是启动类加载器。
 - 系统类加载器（System Class Loader）/ 应用程序类加载器（Application Class Loader）：这也是 Java 语言实现的，负责加载用户类路径（`ClassPath`）上的指定类库，是我们平时

编写 Java 程序时默认使用的类加载器。系统类加载器的父加载器是扩展类加载器。它可以通过 `ClassLoader.getSystemClassLoader()` 方法获取到。

- 自定义类加载器 (Custom Class Loader): 开发者可以根据需求定制类的加载方式, 比如从网络加载 class 文件、数据库、甚至是加密的文件中加载类等。自定义类加载器可以用来扩展 Java 应用程序的灵活性和安全性, 是 Java 动态性的一个重要体现。
 - 这些类加载器之间的关系形成了双亲委派模型, 其核心思想是当一个类加载器收到类加载的请求时, 首先不会自己去尝试加载这个类, 而是把这个请求委派给父类加载器去完成, 每一层次的类加载器都是如此, 因此所有的加载请求最终都应该传送到顶层的启动类加载器中。
 - 只有当父加载器反馈自己无法完成这个加载请求 (它的搜索范围中没有找到所需的类) 时, 子加载器才会尝试自己去加载。
- JDK 9 是从“包级开发”迈向“模块化系统开发”的分水岭。它用模块取代了扩展机制, 优化了性能与结构, 从此 JDK 不再是一个“巨大的 rt.jar”, 而是由多个可独立加载的模块组成的系统。
- 双亲委派模型的作用
- 保证类的唯一性: 通过委托机制, 确保了所有加载请求都会传递到启动类加载器, 即使 `ExtClassLoader` 无法加载目标类, 它仍然会先把请求传递给父加载器 (Bootstrap)。并且每个类在同一个 `ClassLoader` 中只会被加载一次, 避免了不同类加载器重复加载相同类的情况, 保证了 Java 核心类库的统一性, 也防止了用户自定义类覆盖核心类库的可能。
 - 保证安全性: 由于 Java 核心库被启动类加载器加载, 而启动类加载器只加载信任的类路径中的类, 这样可以防止不可信的类假冒核心类, 增强了系统的安全性。防止用户自定义与核心类 (如 `java.lang.String`) 同名的类, 污染 JVM 命名空间例如, 恶意代码无法自定义一个 `Java.lang.System` 类并加载到 JVM 中, 因为这个请求会被委托给启动类加载器, 而启动类加载器只会加载标准的 Java 库中的类。
 - 支持隔离和层次划分: 双亲委派模型支持不同层次的类加载器服务于不同的类加载需求, 如应用程序类加载器加载用户代码, 扩展类加载器加载扩展框架, 启动类加载器加载核心库。这种层次化的划分有助于实现沙箱安全机制, 保证了各个层级类加载器的职责清晰, 也便于维护和扩展。
 - 简化了加载流程: 简化了加载流程: 通过委派, 大部分类能够被正确的类加载器加载, 减少了每个加载器需要处理的类的数量, 简化了类的加载过程, 每个加载器只负责自己范围内的类, 减少复杂性, 提高效率
 - 为什么说是「大部分类」而不是「全部类」

⚠️ 二、为什么说是「大部分类」而不是「全部类」

因为——在某些情况下，类加载会被“打破双亲委派”或“委派失败”，从而需要子加载器自己去加载。这类情况包括：

情况	说明
1 自定义类加载器（打破双亲委派）	某些框架（如 Tomcat、OSGi、JSP 编译器）会反向委派：让子加载器先尝试加载，父加载器再兜底。
2 不同模块或 ClassLoader 隔离	比如 JDK 9 的模块化系统、应用服务器的多应用隔离，每个模块只能加载自己范围内的类。
3 类加载路径中确实不存在目标类	当类不在任何父加载器的加载路径下（如外部 jar 包、插件 jar），父加载器无法加载，只能子加载器处理。
4 SPI（Service Provider Interface）机制	某些 SPI 接口（如 JDBC）由 Bootstrap 加载，而实现类由 AppClassLoader 加载，此时加载链断开。
5 特殊场景：热部署 / 插件系统	如 Spring Boot DevTools、IDEA 热加载、Tomcat WebAppClassLoader，都会破坏委派以实现动态替换。

- 双亲委派如何打破
 - 自定义 ClassLoader 并重写 loadClass
 - 线程上下文类加载器
 - 这是 JDBC / JNDI / SPI / ServiceLoader 等框架常用的打破方式。
 - 线程上下文类加载器让“父 ClassLoader”能使用“子 ClassLoader”加载类，从而突破双亲委派模型。
 - 自己实现了 ClassLoader 隔离机制。
 - Tomcat / Spring Boot 使用自己的类加载逻辑
 - 热部署框架、IDE 插件
 - 不走 JVM 默认机制，所以全部会构造自定义 ClassLoader。
- 介绍 threadlocal 原理
 - threadlocal 内部有一个 threadlocalmap
 - Entry
 - key 是一个弱引用的 ThreadLocal 对象。
 - value 是当前线程保存的具体值。
 - table：存储所有 Entry 的数组。
 - threshold：阈值（扩容时用）。
 - size：当前存储的数量。
 - key 为什么是弱引用
 - 如果 ThreadLocal 对象被外部没有强引用持有，GC 就可以回收它；
 - 但是 ThreadLocalMap 里的 Entry 仍然会存在，key 会变成 null（被回收）；
 - 若不清理，会导致 value 无法访问但还在内存中 → 内存泄漏。
 - 因此 ThreadLocalMap 有自清理机制：
 - 在每次 set() 或 get() 时，ThreadLocalMap 会自动清理那些 key == null 的 Entry；也就是 ThreadLocal 已经被 GC 掉的项。
- redis 的 zset 的底层数据结构？ziplist 的作用？hashtable 的作用？跳表的作用？
 - Redis 的 ZSet（有序集合）是由两个核心数据结构共同实现的：
 - hash（字典 / hashtable）

- 作用：用来根据 成员（member）快速查分值（score）。
- 场景：当你要执行 ZSCORE key member、ZINCRBY key member 这样的操作时，通过 hash 能快速定位。
- skiplist（跳表）
 - 作用：用来根据 分值（score）排序成员，支持范围查询和排名操作。
 - 结构：类似多层链表结构，可以实现有序查找。
 - 时间复杂度： $O(\log N)$ 。
- 底层编码优化：ziplist 与 skiplist 的切换
 - ZSet 有两种编码方式：
 - ziplist（压缩列表）
 - skiplist + dict（标准结构）
 - ziplist 的作用（小数据优化）
 - 当 ZSet 中的元素数量较少且每个元素的数据较短时，Redis 会用 ziplist（压缩列表）存储，以节省内存。
 - ziplist 是一块连续的内存区域。查找和范围操作需要线性遍历 $\rightarrow O(N)$ 。存储顺序：
[score][member][score][member]...
 - 使用 ziplist 的条件（默认配置）
 - zset-max-ziplist-entries 128 元素数量小于 128
 - zset-max-ziplist-value 64 每个成员长度小于 64 字节
 - 只要超过这两个阈值，Redis 会自动转成标准结构（dict + skiplist）。
 - skiplist + dict 的作用
 - 当数据量或元素大小超过阈值时：
 - dict：通过 member 查 score
 - skiplist：通过 score 查 member（维持排序）
- Hashtable 的作用（在 ZSet 内外）
 - 在 ZSet 内：
 - 用来映射 member $\rightarrow$ score。
 - 支持 $O(1)$ 查询和更新。
 - 在 Redis 全局：
 - Redis 的所有键值对数据库本身就是一个大 hashtable。
 - 每个 key 都存在于全局的 dict 中（快速查找键名）。

```
ZADD myRank 100 alice
ZADD myRank 200 bob
ZADD myRank 150 carol
```

- 成员查分值 $\rightarrow$ 用 hashtable（member $\rightarrow$ score）分值查成员 $\rightarrow$ 用 skiplist（score $\rightarrow$ member），所以跳表在任意水平层都是按 score 升序排列的。
- myRank 这个 key 本身是存放在 Redis 全局的哈希表（dict）中，它对应的 value 是一个 ZSet 对象（有序集合对象）
- dict 是 Redis 自己实现的一种哈希表结构，用来存储“key $\rightarrow$ value”的映射关系。
- Redis 自己实现的 dict，比普通哈希表更复杂一些，因为它支持渐进式 rehash（扩容）。当 dict 装满到一定比例后，Redis 不会一次性扩容（那样会卡顿），而是采用渐进式 rehash：
 - 新建一个更大的哈希表 dictht[1]；
 - 每次执行普通操作（如查找 / 插入）时，顺便把一小部分数据搬过去；
 - 搬完所有后，dictht[1] 变成主表，dictht[0] 清空。
- 在 Redis 或任何哈希表中：每个 key 对应唯一一个 value。你不能让“myRank”同时是 String 又是 ZSet；

- dict 也是 hashmap 格式，数组+链表，但是数组是存储 key-value 结构，索引是 key 通过哈希后分到的位置。

哈希表里存的是很多个键值对（key → value），

每个键都会经过哈希函数，

被映射到一个槽位（bucket）上：

makefile

 复制代码

哈希表（数组）

```
index:  0    1    2    3    4    ...
value:  [ ]  [ ]  [ ]  [ ]  [ ]
```

现在，如果两个不同的 key（比如 "abc" 和 "xyz"）

通过哈希函数算出的槽位（index）相同，

就会出现 **哈希冲突**：

bash

 复制代码

```
hash("abc") % 8 = 3
hash("xyz") % 8 = 3
```

于是它们俩都放在同一个桶（bucket）里，用链表链接起来：

less

 复制代码

```
bucket[3] → ("abc", value1) → ("xyz", value2)
```



- 为什么不用红黑树？
 - Redis 中一个 score 可能对应多个 member，跳表更容易处理“重复分值”的情况。
 - 跳表实现简单，范围查询更方便（可以顺序遍历）。
- 如命令 ZADD myRank 100 alice
 - myRank 是 Redis 数据库中的 key，对应一个 ZSET 类型的 value。
 - 内部哈希表的 key 是 ZSET 成员名，也就是 alice。内部哈希表的 value 是指向跳表节点的指针（或早期版本存 score）。
- Redis 的 zset 中删除并重新插入数据的时间复杂度是多少？
 - Redis 的 ZSET 底层是跳表（skiplist）+ 哈希表的组合：
 - 删除元素需要
 - 在哈希表中查找元素：O(1)
 - 在跳表中删除元素：O(logN)
 - 插入元素需要
 - 在哈希表中插入元素：O(1)
 - 在跳表中插入元素：O(logN)
- 压缩列表如何保证按 score 排序的
 - 当你插入新元素时，Redis 会从头遍历压缩列表，找到第一个 score 大于目标 score 的位置，然后将新元素插入到这个位置。
- 对于一个游戏排名系统，有玩家账号、得分、排名，这三个数据，要求能根据账号查找到得分，根据账号能查到排名，根据排名也能查找到玩家账号。

- ZSET
 - ZSET 根据账号查得分直接利用 HASHMAP 实现
 - 根据账号查排名需要先通过 HASHMAP 利用账号查到分数，再利用分数在跳表中 SPAN 查找排名
 - SPAN 过程：跳表的查询先从最高层开始查 若下一跳不满足条件（也就是下一个节点比要查找的节点大）就降层 ZAI 再查找所以是 LOGN
 - 根据排名查账号就是直接在跳表中查询，因为跳表天然按排名排序，直接二分查找也是 LOGN
- 了解过 redis 中的 spark 嘛？

🧠 最后总结

场景	技术	主要功能	优点	📄
Redis + Spark	架构级集成	Spark 从 Redis 读/写数据	高效计算 + 快速缓存	
RedisGears	Redis 模块	Redis 内部执行计算逻辑	低延迟、实时触发	
spark-redis Connector	工具包	Spark ↔ Redis 数据映射	原生 DataFrame 支持	

- Spark 是一个基于内存计算的分布式数据处理引擎，设计用于大规模数据处理和分析任务。它提供了高效的内存计算能力，支持多种编程语言（如 Scala、Java、Python 和 R），并且可以与 Hadoop 生态系统无缝集成。
 - Spark 的核心组件包括：
 - Spark Core：提供基本的分布式任务调度、内存管理和容错机制。
 - Spark SQL：用于结构化数据处理，支持 SQL 查询和 DataFrame API。
 - Spark Streaming：用于实时数据流处理。
 - MLlib：机器学习库，提供各种机器学习算法和工具。
 - GraphX：用于图计算和图分析。
 - Spark 的主要特点：
 - 内存计算：通过将数据存储在内存中，Spark 提供了比传统磁盘计算更快的处理速度。
 - 易用性：提供了丰富的 API 和高级抽象，使得开发者可以轻松编写复杂的数据处理任务。
 - 可扩展性：可以在单机模式下运行，也可以扩展到数千个节点的集群。
 - 容错性：通过数据的弹性分布式数据集（RDD）和数据复制机制，Spark 能够在节点故障时自动恢复数据。
 - Spark 的工作原理：
 - 数据分区：Spark 将数据划分为多个分区，并将这些分区分布在集群的不同节点上。
 - 任务调度：Spark 的调度器将任务分配给集群中的工作节点，并协调任务的执行。
 - 内存计算：Spark 利用内存中的数据进行计算，减少了磁盘 I/O，提高了处理速度。
 - 结果输出：计算结果可以存储在内存中，也可以写入外部存储系统（如 HDFS、数据库等）。
- redis 的 stream 和 MQ 的区别

四、性能与使用场景对比

场景	Stream 更适合	MQ 更适合
日志流/监控流	✓ 持续写入和消费、需要回溯	✗ 不便回放
实时数据分析	✓ 可以随时回放并按ID分片消费	一般需外部存储分析
任务分发/解耦	✗ 模型偏流式	✓ 核心场景
可靠消息传递	一般可靠，但依赖ACK机制	✓ MQ天生为可靠传递设计
削峰填谷	可实现，但不是首选	✓ MQ天然支持
多消费者组并行	✓ 支持 Stream Group	✓ Kafka 也支持 Consumer Group

六、简单总结一句话

- Stream 更像是“可回溯的日志流 + 消费状态管理”系统，适合做数据流式处理、实时分析、日志系统。
- MQ 是“可靠传递与异步解耦”的工具，适合微服务通信、任务分发、削峰等场景。

七、一个实际例子帮助理解

目标	使用 MQ (Kafka / RabbitMQ)	使用 Stream (Redis Stream)
用户下单异步通知库存系统	✓ 可靠消息传递，任务解耦	✗ 回溯意义不大
日志收集与实时分析	✗ 只传递消息没意义	✓ 可以保存与回放分析
实时监控报警	✓/✗ 都可行	✓ 支持时间序列与消费状态
微服务事件驱动	✓ Kafka 常用	✗ Stream 较少用作跨系统总线

- redis 怎么从成本或者使用上优化
 - 减少内存占用
 - 选对数据结构
 - 合理设置过期时间 TTL
 - 压缩与编码优化
 - 删除冗余数据
 - 降低硬件成本
 - 使用内存压缩型实例
 - 冷热分离。热数据放 Redis；冷数据放 MySQL / ElasticSearch；
 - 减少副本节点数量
 - 持久化优化
 - 不建议高频 RDB，会大量占用磁盘 I/O。
 - 降低访问压力
 - 本地缓存 + Redis 缓存双层结构
 - 批量操作（Pipeline）把多个命令合并成一次请求，减少网络往返
 - 分布式限流 / 防击穿

- 利用 Lua 脚本原子执行;
- 热 key 使用本地缓存或互斥锁。
- 集群与分片优化
 - 使用 Redis Cluster 分片扩展;
 - 控制 key 的 哈希标签 (HashTag) 保持相关性;
- Redis 哨兵和分片
 - Redis 哨兵 (Sentinel)
 - 提供高可用 (HA) 机制, 主要解决单节点 Redis 宕机后的自动故障转移问题。
 - 监控: Sentinel 持续监控主从节点的状态。
 - 自动故障转移: 当主节点不可用时, Sentinel 会选举新的主节点, 并通知客户端更新。
 - 配置提供者: 客户端可以通过 Sentinel 获取最新的主节点地址。
 - Redis 分片 (Sharding)
 - 解决单节点容量和性能瓶颈, 把数据水平拆分到多个节点上。
 - 客户端分片 (Client-side Sharding)
 - 客户端根据 key 计算哈希, 决定请求哪个 Redis 节点。
 - 缺点: 节点增加/减少时, 哈希重排需要迁移大量数据
 - Proxy 分片 (如 Twemproxy)
 - 客户端只连接 Proxy, Proxy 管理数据分片。
 - 客户端只连接 Proxy, Proxy 管理数据分片。
 - Redis Cluster 将 key 空间划分为 16,384 个 slot, 每个节点负责部分 slot。
 - 哨兵 + 分片
 - 每个分片都可以是一个主从集群, 由 Sentinel 管理。
 - Redis 哨兵 (Sentinel) 为什么通常要至少 3 个, 而不能只有 2 个, 这其实是分布式系统“选举机制 + 容错性”的结果。
 - 哨兵的职责
 - 监控主从实例是否存活
 - 在主节点宕机时选出新的主节点
 - 但! 选主节点时不能单个哨兵自己说了算, 必须经过多数派 (majority) 投票同意。
 - 至少要有 $\lceil N/2 \rceil + 1$ 个哨兵同意, 才能进行主节点故障转移。
 - 2 个哨兵的情况: 如果有 1 个哨兵挂了, 剩下 1 个无法形成“多数派”。
- 如何处理 Redis 中的热点 Key 问题? 可能引发什么问题? 有哪些解决方案?
 - 什么是热点 Key
 - 在短时间内被大量请求访问 (读或写) 的某个 Key, 远高于平均访问量。
 - 热点 Key 会引发什么问题?
 - Redis 是单线程处理命令的 (除非启用多线程 IO), 热点 Key 导致大量请求集中到一个节点的一个 Key 上, 导致单核 CPU 飙高、处理阻塞
 - 所有请求都打向同一个 Redis 节点, 网络流量集中, 连接数激增
 - 热点 Key 过期时, 大量请求同时打到数据库, 数据库被打垮
 - Redis Cluster 中不同节点负载差异极大, 某节点 QPS 高, 其他节点空闲
 - 缓存副本分摊读压力
 - Redis 主从架构, 将读流量分散到多个从节点;
 - 应用层实现 读写分离 (写主、读从);
 - 或使用 Redis Cluster 分片并手动做“副本扩散”。
 - 缺点: 热点仍集中在同一个 Key 上, 主从间同步延迟可能造成不一致。
 - 本地缓存 (多级缓存)
 - 在应用端增加一层 本地缓存 (Guava Cache / Caffeine / Ehcache);
 - 每次从 Redis 取值后, 缓存到本地内存;

- 多实例时还可使用 定期刷新 + 失效通知。
- 缺点：数据一致性差、占内存。
- Key 拆分 / 热点分片
 - 将一个热点 Key 分成多个小 Key
 - 写操作随机打到其中一个；
 - 读操作时再汇总求和。
 - 缺点：读操作略复杂，需要聚合。
- 缓存预热 + 永不过期 / 延迟过期
 - 热点 Key 提前写入 Redis；
 - 设置较长 TTL，或者不设置过期；
 - 用后台任务周期性刷新。
- 用互斥锁或信号量保证同一时刻只有一个线程回源数据库；其他线程等待该线程加载完毕后再返回结果。
- 限流 + 降级
 - 对热点接口进行限流；
 - Redis 异常或延迟时使用降级策略（返回缓存数据或默认值）。
- 异步更新与消息队列
 - 请求先写入 MQ；
 - 后台消费者异步批量更新 Redis；
 - 再定期同步到数据库。
- 检测热点 Key 的方法
 - Redis 命令


```
redis-cli --bigkeys
redis-cli monitor | grep hotkey
```
 - Redis 热点 Key 采样
 - 使用 redis-cli -latency、-stat 查看延迟；
 - 使用 INFO commandstats 分析命令执行频率；
 - 开启 hot key sampling (Redis 4.0+)：

```
CONFIG SET hotkeys-tracking yes
```
- 若 Redis 在执行过程中掉电或集群网络短暂中断，如何恢复数据？如何保证数据一致性？是否存在不一致的时机？
 - Redis 掉电或网络中断后的数据恢复机制
 - RDB
 - AOF
 - RDB + AOF 混合模式
 - 集群或主从同步下的恢复逻辑
 - 主从复制机制
 - 从节点通过复制偏移量 (replication offset) 与主节点保持一致；
 - 网络短暂中断后，从节点通过 PSYNC 增量同步缺失的数据；
 - 若主节点重启或中断时间过长，会触发全量同步（发送整个 RDB 快照）。
 - 集群网络分区（脑裂）问题
 - 当网络分区导致主节点与部分从节点隔离时
 - 两个主节点可能都认为自己是“主”，接受写操作；
 - 网络恢复后可能出现数据不一致（部分写丢失或回滚）。
 - Redis 的防护机制：

- `min-slaves-to-write` / `min-replicas-to-write`：限制主节点在没有足够从节点时停止写入；
- Cluster 模式下：通过 Epoch（配置纪元）和投票机制决定新的主节点；保证只有一个合法主节点继续接受写操作。
- Redis 属于 AP（高可用 + 分区容忍）系统，一致性是弱化的。但可以通过以下措施尽量提高一致性：

方式	说明	一致性等级
<code>appendfsync always</code>	每次写都同步落盘	最强（性能低）
<code>appendfsync everysec</code>	每秒落盘一次	次强（≤1s丢失）
主从 + ACK 机制（ <code>WAIT</code> 命令）	等待指定数量的副本确认写入	可控一致性
<code>min-slaves-to-write</code>	主节点失联时拒绝写入	避免脑裂写入
Redis Cluster + 选举机制	保证只有一个主节点写入	防止双主

- 是否存在不一致的时机？
 - 写入后尚未落盘即掉电，内存数据未同步至磁盘
 - 主从异步复制，写入尚未同步至从节点时主挂掉
 - 网络分区（脑裂），双主各自接受写操作
 - 使用 `everysec` 策略，一秒内未落盘的数据丢失
- 数据校对过程的时效、实现方案是什么？
 - Redis 中的数据校对（Data Reconciliation / Data Consistency Check）主要用于以下几种情况：
 - 主从节点间检查数据偏移量、key 数量、hash 校验，保证主从一致
 - REDIS 和 AOF RDB 之间确认内存数据与磁盘文件数据一致
 - REDIS 和 DB 确保缓存与数据库最终一致（缓存一致性问题）
 - 校对的触发时机（时效性）
 - 主从偏移量对比：每次心跳（默认 1 秒）自动触发秒级
 - AOF/RDB 文件一致性检测：Redis 启动时或定时任务执行时分钟级 / 小时级
 - 缓存与数据库一致性：应用层定期任务触发（如每 5 分钟 / 每小时）
 - 人工检测触发全量校验
 - AOF 的数据校对：
 - Redis 在写入时会维护 AOF 文件的 CRC 校验值；
 - 重启时 Redis 会使用 `redis-check-aof` 工具或自动检测文件末尾；
 - 如果发现 AOF 文件尾部不完整（掉电中断造成），会截断到最后一个完整命令为止；
 - 然后重放所有命令
 - REDIS 和 DB 的数据校对实现方案
 - XXL-JOB 定时校对，不一致时恢复
 - 增量校对（实时修复）：写操作时同时记录变更流水，定时扫描变更流水，对 Redis 中相应 key 重构缓存；增量校对其实就是模仿写日志的流程：更新数据库时同时写日志，然后再根据日志写到 Redis。

- 双写校验 + 异步补偿：每次写数据库后，再写 Redis；若 Redis 返回错误或超时，写入补偿队列（Kafka / RabbitMQ）；消费者异步消费 MQ
- 将本地缓存扩展为 Redis 集群后，如何确定某个 key 存储在哪个机器上？
 - Redis Cluster 并不是随机存放数据的，而是将整个键空间划分为 16384 个槽（Slot）。
 - Redis 使用 CRC16 哈希算法计算每个 key 的哈希值；然后对 16384 取模得到 SLOT
 - 每个 Redis 节点负责一部分连续的 slot 区间。
 - 只要知道 key 对应的 slot 值，就能立即知道它归属于哪个节点。
 - 特殊情况：Hash Tag（哈希标签）
 - 有时你希望多个 key 落在同一个 slot：如


```
mget user:{1001}:name user:{1001}:age
```
 - Redis 会只取 {} 内的部分来计算 slot：


```
slot = CRC16("1001") % 16384
```
 - 这样两个 KEY 会落到同一个节点上
- redis 的主从同步是怎么实现的
 - 从节点发起复制请求
 - 连接主节点。
 - 发送 PING 以测试主从之间网络是否通畅。
 - 进行认证（如果主节点设置了 requirepass）。
 - 发送 PSYNC 命令请求同步。
 - 主从协商同步方式
 - 主节点根据 PSYNC 命令决定同步模式：
 - 全量同步（Full Resync）
 - 部分同步（Partial Resync）
 - 全量同步过程
 - 全量同步是最初始的、最耗时的一次同步
 - 主节点创建 RDB 快照，主节点执行 BGSAVE，在后台生成一个 RDB 文件。
 - RDB 传输，RDB 文件生成完毕后，主节点将文件通过 socket 发送给从节点。从节点接收 RDB 文件后：
 - 清空本地旧数据；
 - 加载新的 RDB 到内存。
 - 命令传播缓冲
 - 在主节点生成 RDB 期间，如果有新的写命令进来，主机会将这些命令写入一个缓冲区（称为 Replication Buffer 或 backlog buffer）。当 RDB 传输完成后，主节点会将缓冲区的写操作再发给从节点，以保持最终一致。
 - 增量同步
 - 当主从断开后重新连接，如果主节点仍然保留了从节点上次同步的复制偏移量（offset）和 runid，则可以进行增量同步。
 - 从节点发送 PSYNC。
 - 主节点检查 backlog buffer 是否仍然有该 offset 之后的数据：
 - 有：返回 CONTINUE，仅发送增量命令。
 - 无：返回 FULLRESYNC，重新全量同步。
 - 这就是 Redis 2.8+ 后改进的部分重同步机制
 - 主从完成一次全量或部分同步后，会进入命令传播阶段：

- 主节点每执行一个写命令（如 SET、DEL、LPUSH 等），都会将该命令传播给所有从节点；
- 从节点按顺序执行这些命令；
- 部分同步的目标
 - 在早期（Redis 2.8 以前），主从断开后，从节点只能重新全量同步；
 - 部分同步让主从短暂断开后，只同步“断开期间”丢失的那一小部分数据，而不是重新同步全部数据。
- 如果同步后有增量同步 那不是只要一次全量同步就好了吗
 - 只要主从完成一次全量同步，之后只要连接不断、网络稳定，主从之间确实就会一直通过增量同步（命令传播）保持一致，不需要再做任何全量同步。
 - 因为在真实环境中，有几种情况会打破这种理想状态，导致无法继续增量同步，只能重新全量。
 - 主从网络断开，错过了太多数据
 - 主节点重启了（runid 改变）
 - 从节点第一次加入复制集
 - 主从数据不一致（人为修改或过期）
- 主从同步的间隔
 - Redis 主从同步不是定时拉取的
 - Redis 的主从复制是事件驱动 + 异步推送机制，
 - 主节点执行写命令 → 主节点立即推送给从节点。
 - 但为了保证主从状态健康，它们确实有“定期通信”
 - 主从之间确实会定期发送一些心跳包、offset 信息等
 - 检测连接是否正常
 - 判断延迟和同步进度
 - 确认是否需要补发数据

🧠 三、主从复制中的三种通信

类型	方向	目的	间隔
① 命令传播（写命令）	Master → Slave	实时推送写操作	实时（事件触发）
② ACK 回执（offset 确认）	Slave → Master	告诉主节点：我已经同步到哪个 offset	每 1 秒
③ PING/PONG 心跳	双向	确保连接健康	每 1 秒（默认）

- 从节点会每秒向主节点发送一次 REPLCONF ACK 命令：主节点会保存每个从节点的最新 offset，用来判断每个从节点的复制延迟与健康状态。
- 主节点保存所有从节点的复制状态信息，主节点通过这些信息：
 - 知道哪些从节点落后；
 - 触发部分同步（PSYNC）时判断是否能补齐；
 - 在 Sentinel / Cluster 模式下评估从节点是否可被提升为主。
- backlog 是个固定长度的环形缓冲区（circular buffer），用于保存最近写入命令的数据流。
 - 当从节点断开后又重连时，如果它上次同步的位置还在 backlog 范围内，就可以执行部分重同步（增量同步）；
 - 否则，只能执行全量同步。

- 因为是环形，当 backlog 写满后会覆盖原来的数据，如果从节点长时间没有与主节点进行部分同步，backlog 一直增加就会造成覆盖

六、如果从节点发现 offset 不一致怎么办？

Redis 复制是“主推从拉确认”的混合机制。

流程如下：

1. 主节点不断把新的写命令推送给从节点；
2. 从节点执行命令并更新本地 offset；
3. 从节点定期（1秒）汇报 offset；
4. 主节点比较自己的 `master_rep1_offset` 与从节点 offset；
5. 如果从节点落后太多、backlog 覆盖区间以外，就要重新同步。

也就是说：

从节点不是“主动发现 offset 不一致就拉数据”，
而是通过持续 ACK + backlog 检查机制由主节点来补数据（部分同步）。

• Redis 的大 key 问题

▸ Redis 的大 Key 问题是什么？

- Redis 大 key 问题指的是某个 key 对应的 value 值所占的内存空间比较大，导致 Redis 的性能下降、内存不足、数据不均衡以及主从同步延迟等问题。
- 到底多大的数据量才算是大 key？
- 没有固定的判别标准，通常认为字符串类型的 key 对应的 value 值占用空间大于 1M，或者集合类型的 k 元素数量超过 1 万个，就算是大 key。
- Redis 大 key 问题的定义及评判准则并非一成不变，而应根据 Redis 的实际运用以及业务需求来综合评估。
- 例如，在高并发且低延迟的场景中，仅 10kb 可能就已构成大 key；然而在低并发、高容量的环境下，大 key 的界限可能在 100kb。因此，在设计与运用 Redis 时，要依据业务需求与性能指标来确立合理的大 key 阈值。

▸ 大 Key 问题的缺点？

- 内存占用过高。大 Key 占用过多的内存空间，可能导致可用内存不足，从而触发内存淘汰策略。在极端情况下，可能导致内存耗尽，Redis 实例崩溃，影响系统的稳定性。
- 性能下降。大 Key 会占用大量内存空间，导致内存碎片增加，进而影响 Redis 的性能。对于大 Key 的操作，如读取、写入、删除等，都会消耗更多的 CPU 时间和内存资源，进一步降低系统性能。
- 阻塞其他操作。某些对大 Key 的操作可能会导致 Redis 实例阻塞。例如，使用 DEL 命令删除一个大 Key 时，可能会导致 Redis 实例在一段时间内无法响应其他客户端请求，从而影响系统的响应时间和吞吐量。
- 网络拥塞。每次获取大 key 产生的网络流量较大，可能造成机器或局域网的带宽被打满，同时波及其他服务。例如：一个大 key 占用空间是 1MB，每秒访问 1000 次，就有 1000MB 的流量。

- 主从同步延迟。当 Redis 实例配置了主从同步时，大 Key 可能导致主从同步延迟。由于大 Key 占用较多内存，同步过程中需要传输大量数据，这会导致主从之间的网络传输延迟增加，进而影响数据一致性。
 - 数据倾斜。在 Redis 集群模式中，某个数据分片的内存使用率远超其他数据分片，无法使数据分片的内存资源达到均衡。另外也可能造成 Redis 内存达到 maxmemory 参数定义的上限导致重要的 key 被逐出，甚至引发内存溢出。
- Redis 大 key 如何解决？
- 对大 Key 进行拆分。例如将含有数万成员的一个 HASH Key 拆分为多个 HASH Key，并确保每个 Key 的成员数量在合理范围。在 Redis 集群架构中，拆分大 Key 能对数据分片间的内存平衡起到显著作用。
 - 对大 Key 进行清理。将不适用 Redis 能力的数据存至其它存储，并在 Redis 中删除此类数据。注意，要使用异步删除。
 - 监控 Redis 的内存水位。可以通过监控系统设置合理的 Redis 内存报警阈值进行提醒，例如 Redis 内存使用率超过 70%、Redis 的内存存在 1 小时内增长率超过 20% 等。
 - 对过期数据进行定期清。堆积大量过期数据会造成大 Key 的产生，例如在 HASH 数据类型中以增量的形式不断写入大量数据而忽略了数据的时效性。可以通过定时任务的方式对失效数据进行清理。
- redis 的过期删除和缓存淘汰
- 过期删除策略和内存淘汰策略有什么区别？
- 区别：
 - 内存淘汰策略是在内存满了的时候，redis 会触发内存淘汰策略，来淘汰一些不必要的内存资源，以腾出空间，来保存新的内容
 - 过期键删除策略是将已过期的键值对进行删除，Redis 采用的删除策略是惰性删除+定期删除。
- 介绍一下 Redis 内存淘汰策略
- 在 32 位操作系统中，maxmemory 的默认值是 3G，因为 32 位的机器最大只支持 4GB 的内存，而系统本身就需要一定的内存资源来支持运行，所以 32 位操作系统限制最大 3 GB 的可用内存是非常合理的，这样可以避免因为内存不足而导致 Redis 实例崩溃。
 - Redis 内存淘汰策略共有八种，这八种策略大体分为「不进行数据淘汰」和「进行数据淘汰」两类策略。
 - 不进行数据淘汰的策略：
 - noeviction (Redis3.0 之后，默认的内存淘汰策略)：它表示当运行内存超过最大设置内存时，不淘汰任何数据，这时如果有新的数据写入，会报错通知禁止写入，不淘汰任何数据，但是如果没用数据写入的话，只是单纯的查询或者删除操作的话，还是可以正常工作。
 - 进行数据淘汰的策略：
 - 针对「进行数据淘汰」这一类策略，又可以细分为「在设置了过期时间的数据中进行淘汰」和「在所有数据范围内进行淘汰」这两类策略。
 - 在设置了过期时间的数据中进行淘汰：
 - volatile-random：随机淘汰设置了过期时间的任意键值；
 - volatile-ttl：优先淘汰更早过期的键值。
 - volatile-lru (Redis3.0 之前，默认的内存淘汰策略)：淘汰所有设置了过期时间的键值中，最久未使用的键值；
 - volatile-lfu (Redis 4.0 后新增的内存淘汰策略)：淘汰所有设置了过期时间的键值中，最少使用的键值；
 - 在所有数据范围内进行淘汰：

- allkeys-random: 随机淘汰任意键值;
 - allkeys-lru: 淘汰整个键值中最久未使用的键值;
 - allkeys-lfu (Redis 4.0 后新增的内存淘汰策略): 淘汰整个键值中最少使用的键值。
- 介绍一下 Redis 过期删除策略
- Redis 选择「惰性删除+定期删除」这两种策略配合使用, 以求在合理使用 CPU 时间和避免内存浪费之间取得平衡。
 - Redis 的惰性删除策略由 db.c 文件中的 `expireIfNeeded` 函数实现
 - Redis 在访问或者修改 key 之前, 都会调用 `expireIfNeeded` 函数对其进行检查, 检查 key 是否过期:
 - 如果过期, 则删除该 key, 至于选择异步删除, 还是选择同步删除, 根据 `lazyfree_lazy_expire` 参数配置决定 (Redis 4.0 版本开始提供参数), 然后返回 null 客户端;
 - 如果没有过期, 不做任何处理, 然后返回正常的键值对给客户端;
 - Redis 的定期删除是每隔一段时间「随机」从数据库中取出一定数量的 key 进行检查, 并删除其中的过期 key。
 - 这个间隔检查的时间是多长呢?
 - 在 Redis 中, 默认每秒进行 10 次过期检查一次数据库, 此配置可通过 Redis 的配置文件 `redis.conf` 进行配置, 配置键为 `hz` 它的默认值是 `hz 10`。特别强调下, 每次检查数据库并不是遍历过期字典中的所有 key, 而是从数据库中随机抽取一定数量的 key 进行过期检查。
 - 随机抽查的数量是多少呢?
 - 我查了下源码, 定期删除的实现在 `expire.c` 文件下的 `activeExpireCycle` 函数中, 其中随机抽查的数量由 `ACTIVE_EXPIRE_CYCLE_LOOKUPS_PER_LOOP` 定义的, 它是写在代码中的, 数值是 20。也就是说, 数据库每轮抽查时, 会随机选择 20 个 key 判断是否过期。接下来, 详细说说 Redis 的定期删除的流程:
 - 从过期字典中随机抽取 20 个 key;
 - 检查这 20 个 key 是否过期, 并删除已过期的 key;
 - 如果本轮检查的已过期 key 的数量, 超过 5 个 (20/4), 也就是「已过期 key 的数量」占比「随机抽取 key 的数量」大于 25%, 则继续重复步骤 1; 如果已过期的 key 比例小于 25%, 则停止继续删除过期 key, 然后等待下一轮再检查。
 - 可以看到, 定期删除是一个循环的流程。那 Redis 为了保证定期删除不会出现循环过度, 导致线程卡死现象, 为此增加了定期删除循环流程的时间上限, 默认不会超过 25ms。
- Redis 的缓存失效会不会立即删除?
- 不会, Redis 的过期删除策略是选择「惰性删除+定期删除」这两种策略配合使用。
 - 惰性删除策略的做法是, 不主动删除过期键, 每次从数据库访问 key 时, 都检测 key 是否过期, 如果过期则删除该 key。
 - 定期删除策略的做法是, 每隔一段时间「随机」从数据库中取出一定数量的 key 进行检查, 并删除其中的过期 key。
- 那为什么我不过期立即删除?
- 在过期 key 比较多的情况下, 删除过期 key 可能会占用相当一部分 CPU 时间, 在内存不紧张但 CPU 时间紧张的情况下, 将 CPU 时间用于删除和当前任务无关的过期键上, 无疑会对服务器的响应时间和吞吐量造成影响。所以, 定时删除策略对 CPU 不友好。
- Redis 是怎么找到 key 存储在哪个节点上
- Redis Cluster 将整个 key 空间划分为 16384 个槽 (slot), 每个主节点负责一部分槽 (slot 范围), 每个槽对应若干个 KEY

- Redis 对 key 的槽编号计算规则是： $\text{slot} = \text{CRC16}(\text{key}) \% 16384$ 得到槽号 (0 ~ 16383)。
- 客户端如何知道 key 在哪个节点？
 - 客户端连接 Redis 集群时，最初只知道一个节点。Redis 使用一种“跳转机制”来引导客户端找到正确节点：
 - 客户端向某个节点发送命令；
 - 若该节点不负责该 key 对应的槽，会返回 MOVED
 - 客户端收到后更新本地槽映射表 (slot → 节点) 直接重定向请求到正确节点。
- 槽迁移
 - 当你添加或删除节点时，Redis 通过迁移槽 (slot) 实现数据再平衡
 - Redis 会将部分槽从 Node A 挪到新节点 Node D；
 - 在迁移期间：
 - 源节点仍接收请求，但可能返回 ASK；这表示：这个槽我正在迁移中，数据可能已经到另一台机器，请你这次请求去目标节点处理，但不要更新路由表！
 - 客户端会临时跳转到目标节点；
 - 迁移完成后，更新全局槽映射。
- String 是使用什么存储的？为什么不用 C 语言中的字符串？
 - Redis 的 String 字符串是用 SDS 数据结构存储的。
 - 结构中的每个成员变量分别介绍下：
 - len，记录了字符串长度。这样获取字符串长度的时候，只需要返回这个成员变量值就行，时间复杂度只需要 $O(1)$ 。
 - alloc，分配给字符串的空间长度。这样在修改字符串的时候，可以通过 $\text{alloc} - \text{len}$ 计算出剩余的空间大小，可以用来判断空间是否满足修改需求，如果不满足的话，就会自动将 SDS 的空间扩展至执行修改所需的大小，然后才执行实际的修改操作，所以使用 SDS 既不需要手动修改 SDS 的空间大小，也不会出现前面所说的缓冲区溢出的问题。
 - flags，用来表示不同类型的 SDS。一共设计了 5 种类型，分别是 sdshdr5、sdshdr8、sdshdr16、sdshdr32 和 sdshdr64，后面在说明区别之处。
 - buf[]，字符数组，用来保存实际数据。不仅可以保存字符串，也可以保存二进制数据。
 - 总的来说，Redis 的 SDS 结构在原本字符数组之上，增加了三个元数据：len、alloc、flags，用来解决 C 语言字符串的缺陷。
 - $O(1)$ 复杂度获取字符串长度
 - C 语言的字符串长度获取 strlen 函数，需要通过遍历的方式来统计字符串长度，时间复杂度是 $O(N)$ 。
 - 而 Redis 的 SDS 结构因为加入了 len 成员变量，那么获取字符串长度的时候，直接返回这个成员变量的值就行，所以复杂度只有 $O(1)$ 。
 - 二进制安全
 - 因为 SDS 不需要用“0”字符来标识字符串结尾了，而是有个专门的 len 成员变量来记录长度，所以可存储包含“0”的数据。但是 SDS 为了兼容部分 C 语言标准库的函数，SDS 字符串结尾还是会加上“0”字符。
 - 因此，SDS 的 API 都是以处理二进制的方式来处理 SDS 存放在 buf[] 里的数据，程序不会对其中的数据做任何限制，数据写入的时候是什么样的，它被读取时就是什么样的。
 - 通过使用二进制安全的 SDS，而不是 C 字符串，使得 Redis 不仅可以保存文本数据，也可以保存任意格式的二进制数据。
 - 不会发生缓冲区溢出
 - C 语言的字符串标准库提供的字符串操作函数，大多数 (比如 strcat 追加字符串函数) 都是不安全的，因为这些函数把缓冲区大小是否满足操作需求的工作交由开发者来保

证，程序内部并不会判断缓冲区大小是否足够用，当发生了缓冲区溢出就有可能造成程序异常结束。

- 所以，Redis 的 SDS 结构里引入了 alloc 和 len 成员变量，这样 SDS API 通过 alloc - len 计算，可以算出剩余可用的空间大小，这样在对字符串做修改操作的时候，就可以由程序内部判断缓冲区大小是否足够用。
- 而且，当判断出缓冲区大小不够用时，Redis 会自动将扩大 SDS 的空间大小，以满足修改所需的大小。
- Redis 实现并发锁的方式
 - setnx
 - redisson
 - 可重入锁（同线程可多次加锁）
 - 自动续期机制（看门狗 Watchdog）
 - Lua 脚本保证原子性
 - redlock
 - 当系统是 多 Redis 实例（分布式 Redis 集群） 时，用 RedLock 算法确保高可靠性。
 - 实现思路
 - 在 N 个独立 Redis 实例 上同时尝试加锁（推荐 N=5）
 - 如果在 超过一半实例 ($N/2+1$) 加锁成功 且时间未超时，则认为加锁成功
 - 加锁失败则在所有实例上释放锁并重试
 - RedLock 的优势
 - 不依赖单点 Redis（一个实例挂了锁仍然安全）
 - 解决了单实例 Redis 在主从延迟或 failover 时的安全问题
- redis 存登陆 token 如果 token 被淘汰了怎么办 如果 token 泄漏了怎么办
 - 如果 token 被 Redis 淘汰了怎么办？
 - 使用独立 Redis 实例或数据库
 - 双层机制：Redis + JWT 自验证
 - 即使 Redis 挂了或丢失缓存，也能通过 JWT 自身验证继续生效。
 - Redis 持久化
 - 如果 token 泄漏了怎么办？
 - 缩短 token 过期时间，被盗的 token 也只能短期生效。
 - 加入 Redis 校验机制（黑名单）当用户登出、改密、异常登录时，主动删除或加入黑名单：
 - 绑定客户端信息在 JWT payload 中嵌入部分信息：验证时比对 IP、UA 等信息不符则可以强制前一个 token 失效
 - 使用 HTTPS
 - Refresh Token 机制
 - 用户用 refresh token 换取新 access token；
 - 一旦泄漏，可立即让 refresh token 失效，彻底断开会话。
- 什么场景下会考虑用 Redisson？
 - 分布式锁
 - 可重入锁（同线程可多次加锁）
 - 自动续期机制（看门狗）
 - 公平锁
 - 分布式信号量 / 限流器
 - 控制某个资源的最大并发数
 - 对应 Redisson：

- RSemaphore (信号量)
- RRateLimiter (限流器)
- 分布式集合、Map、Queue、Topic
 - 对应 Redisson:
 - RMap, RList, RSet, RQueue, RDeque
 - RTopic (发布/订阅消息)
 - RBlockingQueue (阻塞队列, 适合任务调度)
- 分布式同步器 (锁以外的协作机制)
 - 想实现类似 Java 并发包中的 CountDownLatch、CyclicBarrier、Lock 等机制, 但在分布式环境下用。
 - 对应 Redisson:
 - RCountDownLatch: 多个服务等待同一个事件;
 - RReadWriteLock: 分布式读写锁;
 - RPermitExpirableSemaphore: 带有效期的许可。
- Redisson 的看门狗机制了解吗? 它是如何防止锁被提前释放的?
 - 问题: 当业务执行时间超过锁的过期时间, 锁会自动释放, 导致其他线程“误以为”锁可用, 从而出现并发修改。
 - Redisson 为了解决这个问题, 引入了“看门狗 (Watchdog)”。
 - 看门狗的作用: 当一个线程成功获得分布式锁后, Redisson 会在后台启动一个守护线程, 周期性地为这把锁“续命”, 防止锁在业务执行期间被 Redis 自动删除。
 - 工作机制详解
 - Step 1: 加锁时
 - 调用 Lua 脚本执行 SETNX;
 - 设置默认过期时间为 30 秒;
 - 保存当前线程 ID 到 Redis 的锁值中;
 - Step 2: 启动看门狗线程
 - Redisson 会启动一个后台定时任务 (watchdog), 每隔一段时间 (默认 10 秒) 执行一次判断, 确认线程是否还持有该锁, 若还持有则刷新过期时间
 - Step 3: 锁释放时
 - Redisson 会: 删除 Redis 中的锁键; 停止对应的看门狗线程;
- Redisson 实现原理
 - 基于 Netty + Redis 的 Java 客户端
 - Redisson 将每一个 Redis 操作都封装成 Command, 然后通过 Netty I/O 事件循环发往 Redis。
 - 每个命令返回 Promise → Future, 内部使用 RFuture (自定义的 future) 来表示异步返回。
 - 每个连接都有 Redis 请求队列 (待发送) 和 Redis 响应队列 (待解析)
 - 基于 PubSub + Master/Slave 监控
 - Redisson 分布式锁结构
 - 可重入锁:
 - 每把锁在 Redis 里都是一个 Hash
 - 加锁 Lua 脚本
 - 不可重入锁
 - Redis 内部存储是 String key → value
 - 用 SET key value NX PX expire 一次性设置
- 介绍缓存穿透的解决方案及相关经验。
 - 缓存穿透是指查询一个一定不存在的数据, 既不会命中缓存, 也查不到数据库结果, 导致每次请求都要打到数据库。

- 缓存穿透的成因
 - 恶意攻击或爬虫：故意请求不存在的 key；
 - 业务 bug：请求参数校验缺失；
 - 数据确实未存在且未缓存空值；
 - 缓存过期+查询空数据未缓存。
 - 后果
 - 数据库压力骤增；
 - 系统整体响应延迟上升甚至崩溃。
 - 解决方案
 - 缓存空对象
 - 优点：容易实现
 - 缺点：
 - 占用缓存空间，可能被频繁访问
 - 缓存空值 TTL 要设置得当（太长浪费内存，太短不防穿透）。
 - 使用布隆过滤器（Bloom Filter）
 - 原理：使用哈希算法映射一个“存在性标记”；若布隆过滤器判断“不存在”，直接拦截请求；若判断“可能存在”，再去查询缓存/数据库。
 - 优点：
 - 内存占用低，查询速度快
 - 能有效拦截绝大多数不存在的请求
 - 缺点：
 - 有一定误判率（可能误判为存在）
 - 需要维护过滤器数据同步（新增、删除时更新）。
 - 接口层参数校验
 - 直接在 API 层防止非法请求进入缓存/数据库层。
 - 限流 + 黑名单
 - 针对异常访问（如同一 IP 高频访问不存在的 key），进行限流或拉黑。
 - 统计每个 IP 的请求失败率；超阈值后临时封禁。
-
- 除了分布式锁，Redis 缓存还常用在哪些场景？
 - 热点数据缓存
 - 缓存穿透、击穿、雪崩防护
 - 实时排行榜（ZSet）
 - 接口限流（计数器或漏桶算法）
 - 计数器统计，用户访问量统计，点赞数 / 收藏数 / 商品浏览量统计
 - 使用 List / Stream 做简易消息队列
 - 登录态、Session 共享。在分布式部署下，用 Redis 存用户 Session
 - 延迟队列。在 Redisson 中可直接用 RDelayedQueue
 - Redis 是怎么做持久化的？讲讲 RDB 和 AOF。
 - RDB 是快照持久化
 - Redis 会周期性地 将内存中的数据快照 保存到磁盘（生成一个 .rdb 文件）。
 - 可以理解为“某一时刻的全量备份”。
 - RDB 的执行


```
save 900 1      # 900秒内有1次写操作就触发RDB
save 300 10     # 300秒内有10次写操作就触发RDB
save 60 10000   # 60秒内有10000次写操作就触发RDB
```


或

SAVE # 同步执行，阻塞主线程
BGSAVE # 异步执行，fork子进程完成

- BGSAVE 的流程如下

- 主进程执行 fork() 创建一个子进程；
- 子进程将当前内存数据写入一个临时文件；
- 写完后替换原有的 dump.rdb；
- 主进程继续处理请求（几乎无阻塞）。

- 优点

- 性能好：主进程 fork 后，几乎零阻塞。
- 恢复快：加载一个 RDB 文件即可恢复。
- 文件紧凑：二进制格式，占空间小。

- 缺点：

- 丢数据风险高：可能丢失最后一次快照后的所有数据。
- 生成文件开销大：fork + 写磁盘，对大数据量影响较明显。

▸ AOF 是追加日志持久化（Append Only File）

▸ Redis 会把每次执行的写命令（set/del/incr...）追加到日志文件 aof 文件中；

▸ 重启时，Redis 会重放 AOF 日志，重新构建数据。

▸ AOF 开启方式：

```
appendonly yes  
appendfilename "appendonly.aof"
```

▸ 刷盘策略

- always

- 每次写操作都立即 fsync
- 最慢
- 最安全

- everysec

- 默认
- 每秒 fsync 一次
- 折中
- 最常用

- no

- 由操作系统决定（异步写磁盘）
- 最快
- 可能丢失较多数据

▸ AOF 重写

- AOF 文件会越来越大，所以 Redis 会：

- fork 一个子进程；
- 重写（rewrite）文件，只保留能恢复当前状态的最少命令；
- 新文件写好后覆盖旧文件

- 过程完全异步，不阻塞主线程

▸ 优点

- 数据更安全：丢失窗口可控制在 1 秒内（默认 everysec）。
- 日志可读：AOF 文件是纯文本命令，可手动修复。
- 更灵活：支持增量记录、重写。

▸ 缺点：

- 文件更大：比 RDB 占空间多。

- 恢复稍慢：需要重放日志。
- 写入更频繁：性能略低于 RDB。
- 混合持久化
 - 从 Redis 4.0 开始，引入了 RDB + AOF 混合模式
 - 重写 AOF 时，先写入一份 RDB 格式的快照；
 - 然后再追加最近的增量 AOF 命令。
 - 这样既能：快速恢复也能保证数据安全
- redis 为什么快
 - 基于内存 + 紧凑的数据编码（高效的数据结构）
 - Redis 所有数据都存在 RAM 中，读写延迟在微秒级。
 - 但更厉害的是它对每种数据结构都做了高度优化：
 - Redis 会根据数据大小自动切换编码（例如 ziplist → hashtable）
 - 零拷贝 + 高效的内存分配策略
 - Redis 使用自定义的内存分配器（默认是 jemalloc 或 malloc）
 - Redis 几乎所有内存操作都避免了系统调用的开销。
 - 高效的网络模型：非阻塞 I/O + 多路复用 + 单线程事件循环
 - Redis 使用 epoll (Linux) / kqueue (BSD) 做 I/O 多路复用
 - 单线程事件循环
 - 每次从就绪队列中批量处理 I/O
 - 避免上下文切换、锁竞争
 - 极简的通信协议（RESP）
 - 直接基于 TCP 流解析
 - 缓存友好的数据布局（CPU Cache Friendly）
 - Redis 尽可能使用连续内存结构（如 ziplist、intset），保证数据局部性（locality）。
 - 当 CPU 读取内存时，会提前把相邻数据读进 cache line。
 - 减少系统调用和锁竞争
 - Redis 的核心线程只做三件事：
 - 处理网络事件（读写）
 - 执行命令
 - 解析命令
 - 持久化（RDB/AOF）和过期清理都在后台线程异步执行。这意味着主线程几乎无阻塞操作，执行路径非常短。
 - 命令复杂度极低 + 单命令原子性
 - Redis 命令几乎都是 O(1) 或 O(logN) 操作
- 交换机和路由器分别有什么作用
 - 交换机（Switch）
 - 主要用于局域网（LAN）内部连接设备，实现设备之间的数据通信。
 - 数据链路层（OSI 模型第二层），有些高级交换机支持第三层（路由功能）。
 - 功能特点：
 - 设备互联：连接多台电脑、打印机、服务器等设备，形成局域网。
 - 帧转发：根据 MAC 地址转发数据帧，只把数据发送到目标端口。
 - 减少冲突：支持全双工通信，每个端口独立通道，减少网络冲突。
 - VLAN 划分：高级交换机可以划分虚拟局域网，实现逻辑隔离。
 - 路由器
 - 主要用于不同网络之间的互联，实现不同网段或广域网（WAN）的通信。
 - 网络层（OSI 模型第三层）。
 - 网络互联：连接不同子网或不同网络，如家庭网与互联网。

- 路径选择：根据 IP 地址选择最佳路径转发数据包。
- NAT 功能：家庭路由器通常有 NAT，将内网私有 IP 映射为公网 IP。
- 防火墙/访问控制：高级路由器可以进行安全策略配置。
- 交换机管局部通信，保证局域网内设备能高效互联
- 路由器管跨网络通信保证不同网段或互联网之间能互通。
- 家用 Wi-Fi 路由器 = 路由器 + 交换机 + 无线接入点 + 其他功能。
- 如何区分局域网
 - 局域网通常使用相同的 IP 网段，比如 192.168.1.0/24
 - 在 IP 网络上，局域网内的设备
 - 可以直接互相通信（同一广播域）
 - 不需要经过路由器就可以互相发送数据
 - 如果手机和电脑或者两部设备在同一个 Wi-Fi 网络，连接同一个路由器的 WIFI 也可能因为不同网段而属于不同子网
- TCP 的 TIME_WAIT 状态是怎么产生的，用什么命令可以查看当前有多少连接处于 TIME_WAIT 状态？
 - 在 TCP 四次挥手（连接关闭）过程中，主动发起关闭的一方（也就是先发出 FIN 的一方）最终会进入 TIME_WAIT 状态。
 - 四次挥手的过程
 - 客户端 → 服务端：发送 FIN，表示我这边的数据发完了。
 - 服务端 → 客户端：回复 ACK，确认收到你的 FIN。
 - 服务端 → 客户端：当服务端也发送完数据后，发出自己的 FIN。
 - 客户端 → 服务端：回复 ACK 确认收到。
 - 为什么要进入 TIME_WAIT？
 - 确保对方收到最后的 ACK（防止丢失导致连接状态不同步）；
 - 让旧连接的延迟包在网络中自然消失，防止新连接误收旧包。
 - 因此，TCP 规范（RFC 793）要求主动关闭方在进入 TIME_WAIT 后保持 2MSL（Maximum Segment Lifetime）时间。MSL（报文最大生存时间）通常取 30 秒或 60 秒，所以 TIME_WAIT 一般持续 60 120 秒。
 - 查看当前有多少连接处于 TIME_WAIT 状态
 - netstat -an | grep TIME_WAIT | wc -l
 - ss -ant state time-wait | wc -l
- TCP Server 在网络通信时会涉及哪些系统调用？

二、系统调用详解

阶段	系统调用	作用说明
1 创建 socket	<code>socket()</code>	创建一个套接字（socket 文件描述符），指定协议族、类型、协议。例如： <code>socket(AF_INET, SOCK_STREAM, 0)</code> 创建 TCP 套接字。
2 绑定地址端口	<code>bind()</code>	将 socket 与本地 IP 地址和端口号绑定。例如：绑定 <code>0.0.0.0:8080</code> 。
3 开始监听	<code>listen()</code>	将套接字转为被动监听状态，告诉内核可以接受连接请求，建立一个连接队列。
4 接受连接	<code>accept()</code>	从连接队列中取出一个已完成三次握手的连接，返回一个新的 socket 描述符，用于后续数据通信。
5 数据传输	<code>read()</code> / <code>recv()</code> <code>write()</code> / <code>send()</code>	服务器通过这两个系统调用与客户端进行数据收发。底层会涉及内核缓冲区、TCP 滑动窗口、ACK 等机制。
6 关闭连接	<code>close()</code> 或 <code>shutdown()</code>	释放 socket 资源并触发 TCP 四次挥手。 <code>shutdown()</code> 可选择关闭读、写或双向。

四、对应的客户端调用对比

Server 端	Client 端
<code>socket()</code>	<code>socket()</code>
<code>bind()</code>	(通常不调用，系统自动分配临时端口)
<code>listen()</code>	(无)
<code>accept()</code>	<code>connect()</code>
<code>read()</code> / <code>recv()</code>	<code>write()</code> / <code>send()</code>
<code>close()</code>	<code>close()</code>

- io 多路复用 SELECT 和 EPOLL 和 POLL

项	select	epoll
注册监听	每次调用都传入所有 FD	调用一次 <code>epoll_ctl</code> 注册
内核检测方式	轮询（遍历全部 FD）	事件驱动（内核回调通知）
返回结果	整个集合中标记哪些可用	内核直接返回活跃 FD 列表
文件描述符上限	默认 1024（可调）	理论无限制（受内存限制）

2 性能复杂度对比

操作	select	epoll
监听 FD 数量	$O(n)$	$O(1)$
每次调用需要拷贝 FD 集合	✔ 需要	✗ 不需要
可同时监听数量	一般 1024	理论上可达上百万



- ▶ `select` 每次调用都要遍历整个 FD 集合，即使只有一个就绪。`select` 使用固定大小的位图（通常 `FD_SETSIZE = 1024`），超过就不行；
- ▶ `epoll` 由内核事件回调机制维护就绪队列，只返回活跃的 FD。
- ▶ `epoll` 底层
 - 红黑树+就绪链表+等待队列
 - `epoll_ctl(ADD)` 时，内核会将要监控的文件描述符（fd）加入红黑树；所有监听 FD 存在红黑树中，增删改查为 $O(\log N)$
 - 当某个 fd 对应的 I/O 事件发生（比如可读/可写）时，内核会把这个事件对应的节点放入这个就绪链表中；事件触发时直接放入 ready list，不需要扫描全部 FD
 - 回调机制触发由内核事件驱动而非轮询。当底层设备驱动检测到该 fd 状态改变时，会触发这个回调，把该 fd 挂入就绪链表。
 - 一次注册、持续监听
 - 等待队列的作用是让调用 `epoll_wait()` 的进程在“没有就绪事件”时睡眠。
- ▶ `poll` 相比 `select` 的改进
 - `poll` 使用一个可变长的数组 `pollfd[]` 代替 `select` 的固定大小的 FD 集合，数量只受系统内存限制。不再受 FD 数量限制
 - `poll` 的 events 支持更精细的事件，如 `POLLIN`, `POLLOUT`, `POLLERR`, `POLLHUP` 等，比 `select` 的 `read/write/exception` 更灵活。
- Redis 网络 IO 瓶颈是什么？
 - ▶ 指客户端与 Redis 之间的 socket 通信过程：数据包收发、TCP 读写、内核缓冲区拷贝等。
 - ▶ 即使命令执行极快（微秒级），网络传输、系统调用、协议解析仍可能占主要耗时。
- 登陆方案
 - ▶ 单 token + JWT + Redis（可选黑名单/设备绑定）
 - ▶ 双 token（Access + Refresh）
 - ▶ 双 token + 单点登录系统（SSO） + OAuth2 / OpenID Connect
- 你之前用过哪些 Spring 版本？有什么改动？

- Spring 4 主要是全面支持 Java 8，引入 @RestController。
- Spring 5 是企业最常用的版本，引入 WebFlux，支持响应式编程。
- Spring 6 是最大的变化，要求 Java 17，并把所有 javax.* 改成 jakarta.*，同时支持 AOT 和原生镜像，是面向未来的架构升级。
- Springboot 相比较 spring 好处
 - 先看 Spring 有哪些“痛点”
 - 配置繁琐（XML 地狱）
 - 依赖管理复杂
 - 版本冲突等
 - 环境部署麻烦
 - 传统 Spring Web 应用需要：打成 .war 部署到 Tomcat 或其他外部容器中
 - Springboot 相比较 spring 好处
 - 自动配置
 - 不再需要写大量 XML 或 @Configuration；
 - 通过“约定大于配置”的理念，自动为你配置好常用组件。
 - 起步依赖
 - starter 依赖
 - 内嵌服务器
 - 无需外部部署 WAR 包
 - 健康检查与监控
 - 一致的配置方式（application.yml / .properties）
 - 与微服务体系兼容良好
- SPRINGBOOT 的自动配置
 - Spring Boot 会在应用启动时自动帮你配置好常用的 Bean，不需要你手动在 @Configuration 或 applicationContext.xml 中写一堆繁琐的配置。
 - 如果你引入了依赖


```
<dependency>
  <groupId>spring-boot-starter-web</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

 - Spring Boot 会自动配置好：
 - DispatcherServlet
 - Tomcat 服务器
 - Jackson JSON 解析器
 - 静态资源处理器
 - 常用的 MVC 组件
 - 自动配置的核心注解
 - @SpringBootApplication
 - 这是一个组合注解，包含了 @EnableAutoConfiguration、@ComponentScan 和 @Configuration
 - @EnableAutoConfiguration
 - 告诉 Spring Boot 启用自动配置功能
 - @EnableAutoConfiguration 的工作原理
 - @EnableAutoConfiguration 实际上是通过 @Import 导入了一个叫 AutoConfigurationImportSelector 的类，这个类负责真正的“自动加载配置”。
 - 它从 META-INF/spring.factories（Spring Boot 2.x）或 META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports（Spring Boot 3.x 之后）

里读取所有可以被自动配置的类；然后判断哪些条件满足（比如是否有某个类、Bean、环境变量等），再有选择性地加载对应的配置类。

- 这些文件来自你通过 Maven（或 Gradle）导入的依赖包（JAR）中的资源文件，
- Spring Boot 不会盲目加载所有配置类，它依赖于大量的 条件注解（@Conditional）。
 - ConditionalOnClass 当某个类存在于 classpath 时才生效
 - ConditionalOnMissingBean 当某个 Bean 不存在时才生效
 - ConditionalOnProperty 根据配置属性的值决定是否生效
 - ConditionalOnBean 当某个 Bean 存在时才生效
 - ConditionalOnWebApplication 仅在 Web 应用环境下生效
- Spring 事务注解@Transactional 失效场景
 - 方法调用方式问题（自调用导致失效）
 - 在同一个类内部，一个方法调用另一个被 @Transactional 注解的方法，事务不起作用。
 - Spring 的事务是通过 AOP 代理实现的（默认是基于 JDK 动态代理或 CGLIB）。
 - 事务增强只对 代理对象有效。
 - 类内部自调用不会经过代理对象，而是直接调用本类方法，所以事务被绕过。
 - 事务注解作用位置不正确
 - 默认对 public 方法生效，private/protected/package 默认不生效。
 - 事务只对 RuntimeException 回滚
 - Spring 事务默认只对 unchecked 异常（RuntimeException 和 Error）回滚。
 - checked 异常（Exception 的子类）不会触发回滚。
 - Bean 未被 Spring 托管
 - Spring AOP 代理只会增强 Spring 容器管理的 Bean。
- 两种动态代理和在两种代理下会出现的@Trasncational 注解失效问题
 - JDK 动态代理生成一个实现目标接口的代理类，通过 InvocationHandler.invoke() 调用目标方法
 - CGLIB 代理生成目标类的子类，重写方法，通过 MethodInterceptor.intercept() 实现增强
 - @Trasncational 注解失效问题
 - JDK 代理下调用类中非接口方法
 - JDK 动态代理只代理接口方法
 - @Transactional 标注在接口上但使用 CGLIB 代理
 - CGLIB 只看类，不看接口，事务切面可能丢失
- Spring Boot 自动装配原理
 - SpringBoot 的自动装配原理是基于 Spring Framework 的条件化配置和 @EnableAutoConfiguration 注解实现的。这种机制允许开发者在项目中引入相关的依赖，SpringBoot 将根据这些依赖自动配置应用程序的上下文和功能。
 - SpringBoot 定义了一套接口规范，这套规范规定：SpringBoot 在启动时会扫描外部引用 jar 包中的 META-INF/spring.factories 文件，将文件中配置的类型信息加载到 Spring 容器（此处涉及到 JVM 类加载机制与 Spring 的容器知识），并执行类中定义的各种操作。对于外部 jar 来说，只需要按照 SpringBoot 定义的标准，就能将自己的功能装置进 SpringBoot。
 - 通俗来讲，自动装配就是通过注解或一些简单的配置就可以在 SpringBoot 的帮助下开启和配置各种功能，比如数据库访问、Web 开发。
 - @EnableAutoConfiguration: 这个注解是 Spring Boot 自动装配的核心。它告诉 Spring oot 启用自动配置机制，根据项目的依赖和配置自动配置应用程序的上下文。通过这个注解，SpringBoot 将尝试根据类路径上的依赖自动配置应用程序。
 - @AutoConfigurationPackage, 将项目 src 中 main 包下的所有组件注册到容器中，例如标注了 Component 注解的类等

- ▶ `@Import({AutoConfigurationImportSelector.class})`，是自动装配的核心，接下来分析一下这个注解
 - `AutoConfigurationImportSelector` 是 Spring Boot 中一个重要的类，它实现了 `ImportSelector` 接口，用于实现自动配置的选择和导入。具体来说，它通过分析项目的类路径和条件来决定应该导入哪些自动配置类。
 - 扫描类路径: 在应用程序启动时，`AutoConfigurationImportSelector` 会扫描类路径上的 `META-INF/spring.factories` 文件，这个文件中包含了各种 Spring 配置和扩展的定义。在这里，它会查找所有实现了 `AutoConfiguration` 接口的类，具体的实现为 `getCandidateConfigurations` 方法。
 - 条件判断: 对于每一个发现的自动配置类，`AutoConfigurationImportSelector` 会使用条件判断机制（通常是通过 `@ConditionalOnXxx` 注解）来确定是否满足导入条件。这些条件可以是配置属性、类是否存在、Bean 是否存在等等。
 - 根据条件导入自动配置类: 满足条件的自动配置类将被导入到应用程序的上下文中。这意味着它们会被实例化并应用于应用程序的配置。
- 标准 Web 项目（如基于 Spring MVC 的 HTTP 服务）中，Spring Boot 提供了哪些模块来实现相关能力？其集成能力如何？
 - ▶ Web 层
 - `spring-boot-starter-web`
 - `spring-boot-starter-webflux`
 - `spring-boot-starter-thymeleaf` / `freemarker`
 - `spring-boot-starter-tomcat` / `jetty` / `undertow`
 - ▶ 数据库层
 - `spring-boot-starter-jdbc`
 - `spring-boot-starter-data-redis` / `mongodb` / `elasticsearch`
 - ▶ 安全认证层
 - `spring-boot-starter-security`
 - `spring-boot-starter-oauth2-resource-server` / `client`
 - ▶ 监控运维层
 - `spring-boot-starter-logging`
 - `spring-boot-starter-test`
 - ▶ 消息与异步任务
 - `spring-boot-starter-amqp`
 - `spring-boot-starter-kafka`
 - ▶ Spring Boot 的集成能力
 - Spring Boot 的核心集成机制是基于以下几点：
 - 自动配置机制
 - Starter 启动器统一依赖管理
 - 内嵌容器与一体化运行
 - 配置与环境管理
- 如何在 Spring 中连接 MySQL？具体连接过程（初始化时机、初始化方式）是怎样的？
 - ▶ 依赖准备
 - ▶ 配置数据库连接
 - `SPRINGBOOT` 在 `application.properties` 或 `application.yml` 中
 - 非 Boot 手动配置 `DataSource`。`HikariCP` 是 Spring Boot 默认使用的连接池，也可以选择 `DBCP`、`C3P0` 等。
 - ▶ 初始化时机
 - Spring 容器启动阶段

- 容器扫描@Configuration 类，解析 @Bean。
- DataSource Bean 被创建（初始化连接池，准备连接）。
- 第一次使用时
 - 对于懒加载 DataSource，可能连接池会延迟初始化，第一次获取连接时才真正连接数据库。
- 事务管理与数据库访问
 - DataSource 初始化完成后，JdbcTemplate、MyBatis SqlSessionFactory 或 JPA EntityManager 就可以使用 DataSource 进行数据库操作。
- 总结：初始化分为 容器启动初始化 Bean 和 第一次获取连接使用 两个阶段。
- 为什么需要 Mybatis 这类 ORM 框架？它相比“裸写 SQL”有什么优势？
 - 裸写 SQL 的问题
 - 重复代码多,每次操作都需要写连接、关闭、异常处理、ResultSet 转对象的逻辑，代码冗余。
 - 维护成本高
 - 表结构改变或字段变化时，需要手动修改大量 SQL 和映射逻辑。
 - 类型安全差
 - ResultSet.getXXX 全部手动处理，容易出错。
 - 动态 SQL 难维护
 - 复杂条件拼接 SQL 很麻烦，容易出错。
 - MyBatis 的优势
 - 代码简洁：自动映射 ResultSet 到 Java 对象，不用手动遍历 ResultSet
 - 可维护性：SQL 写在 XML 或注解里，业务逻辑和数据库逻辑分离，修改 SQL 不影响 Java 代码结构
 - 动态 SQL：提供 <if>、<choose>、<foreach> 等动态 SQL 标签
 - 类型安全：<resultType> 会做类型转换和防 SQL 注入
 - 缓存支持：一级缓存、二级缓存
 - 与 Spring 等框架无缝集成
- Mybatis 的二级缓存
 - 一级缓存
 - 同一个 SqlSession 对同一条 SQL 查询，如果参数相同，第二次会直接从缓存取，不会再查询数据库
 - 清空时期：
 - SqlSession 关闭
 - 执行了增删改操作（INSERT、UPDATE、DELETE）
 - 手动调用 clearCache()
 - 二级缓存（Global Cache）
 - 默认关闭
 - 同一个 Mapper 的查询结果可以被不同的 SqlSession 共享。
 - 清空时期：
 - Mapper 下执行了增删改操作
 - 手动清除缓存
 - 过期时间或容量策略触发清理
 - mapper 底层是靠 sqlSession 实现的
 - 若在事务中，也就是代码加上 transaction 注解，所有在这个线程中执行的 Mapper 方法都会共用这个 SqlSession。

- 没有事务，每次调用 Mapper 方法时，SqlSessionTemplate 检查线程上下文：没有事务 → 当前线程无 SqlSession → 新建一个。用完立即关闭

- mybatis 中 #{} 与 \${} 的区别

- #{} 会被 MyBatis 转换为 ? 占位符，底层使用 PreparedStatement 绑定变量。安全
- \${} 是字符串直接替换，不做任何处理。
 - 因为它直接拼接 SQL，所以只应该用在不能用预编译的地方

- 依赖注入底层

- 依赖注入：依赖注入和控制反转恰恰相反，它是一种具体的编码技巧。我们不通过 new 的方式在类内部创建依赖类的对象，而是将依赖的类对象在外部创建好之后，通过构造函数、函数参数等方式传递（或注入）给类来使用。
- Spring 依赖注入怎么把 private 属性注入的
 - Spring 能把 private 属性注入进去，原理是通过反射（reflection）绕过可见性限制来设置字段值。虽然字段注入（field injection）直接在 private 字段上加注解最简单，但在可测试性、可维护性和设计上通常推荐使用构造器注入。
- setter 注入有空对象问题

```
@Component
public class A {
    private B b;

    @Autowired
    public void setB(B b) {
        this.b = b;
    }

    // 构造函数里访问 b（此时 b 还没注入）
    public A() {
        System.out.println(b.doSomething()); // ❌ NullPointerException
    }
}
```

- 此时 b 还没注入，调用 b.doSomething() 会抛空指针异常。

- 除了使用反射还有什么方法来进行值的设置
 - Spring 的优化方式：BeanWrapper + PropertyAccessor
 - Spring 后来为了性能引入了：CGLIB / ASM 动态字节码访问
 - Spring 为了提升性能，在频繁调用 setter 的场景下（比如大批量注入）可能会启用
 - ReflectUtils（基于 CGLIB）
 - BeanUtils（基于 ASM）
 - 来动态生成访问器类（Accessor）。
 - 这样它可以在运行时生成类似下面的代码：

```
class User$$BeanAccessor {
    void setName(User bean, String value) { bean.setName(value); }
}
```

- 然后后续注入时就直接调用这个方法，而不是用反射。

- Spring 还能利用：MethodHandle / VarHandle（JDK 8+ 优化）
 - 这些是 Java 的轻量级反射机制，底层能直接使用 JVM 的调用指令

- spring 是如何解决循环依赖的？

- ▶ 循环依赖指的是两个类中的属性相互依赖对方：例如 A 类中有 B 属性，B 类中有 A 属性，从而形成了一个依赖闭环
- ▶ 循环依赖问题在 Spring 中主要有三种情况：
 - 第一种：通过构造方法进行依赖注入时产生的循环依赖问题。
 - 第二种：通过 setter 方法进行依赖注入且是在多例（原型）模式下产生的循环依赖问题。
 - 第三种：通过 setter 方法进行依赖注入且是在单例模式下产生的循环依赖问题。
- ▶ 只有【第三种方式】的循环依赖问题被 Spring 解决了，其他两种方式在遇到循环依赖问题时，Spring 都会产生异常。
- ▶ Spring 在 DefaultSingletonBeanRegistry 类中维护了三个重要的缓存 (Map)，称为“三级缓存”：
 - singletonObjects (一级缓存)：存放的是完全初始化好的、可用的 Bean 实例，getBean() 方法最终返回的就是这里面的 Bean。此时 Bean 已实例化、属性已填充、初始化方法已执行、AOP 代理（如果需要）也已生成。
 - earlySingletonObjects (二级缓存)：存放的是提前暴露的 Bean 的原始对象引用 或 早期代理对象引用，专门用来处理循环依赖。当一个 Bean 还在创建过程中（尚未完成属性填充和初始化），但它的引用需要被注入到另一个 Bean 时，就暂时放在这里。此时 Bean 已实例化（调用了构造函数），但属性尚未填充，初始化方法尚未执行，它可能是一个原始对象，也可能是一个为了解决 AOP 代理问题而提前生成的代理对象。
 - singletonFactories (三级缓存)：存放的是 Bean 的 ObjectFactory 工厂对象。这是解决循环依赖和 AOP 代理协同工作的关键。当 Bean 被实例化后（刚调完构造函数），Spring 会创建一个 ObjectFactory 并将其放入三级缓存。这个工厂的 getObject() 方法负责返回该 Bean 的早期引用（可能是原始对象，也可能是提前生成的代理对象），当检测到循环依赖需要注入一个尚未完全初始化的 Bean 时，就会调用这个工厂来获取早期引用。
- ▶ Spring 通过 三级缓存 和 提前暴露未完全初始化的对象引用 的机制来解决单例作用域 Bean 的 setter 注入方式的循环依赖问题。
- ▶ 假设存在两个相互依赖的单例 Bean：BeanA 依赖 BeanB，同时 BeanB 也依赖 BeanA。当 Spring 容器启动时，它会按照以下流程处理：
 - 第一步：创建 BeanA 的实例并提前暴露工厂。
 - Spring 首先调用 BeanA 的构造函数进行实例化，此时得到一个原始对象（尚未填充属性）。紧接着，Spring 会将一个特殊的 ObjectFactory 工厂对象存入第三级缓存（singletonFactories）。这个工厂的使命是：当其他 Bean 需要引用 BeanA 时，它能动态返回当前这个半成品的 BeanA（可能是原始对象，也可能是为应对 AOP 而提前生成的代理对象）。此时 BeanA 的状态是“已实例化但未初始化”，像一座刚搭好钢筋骨架的大楼。
 - 第二步：填充 BeanA 的属性时触发 BeanB 的创建。
 - Spring 开始为 BeanA 注入属性，发现它依赖 BeanB。于是容器转向创建 BeanB，同样先调用其构造函数实例化，并将 BeanB 对应的 ObjectFactory 工厂存入三级缓存。至此，三级缓存中同时存在 BeanA 和 BeanB 的工厂，它们都代表未完成初始化的半成品。
 - 第三步：BeanB 属性注入时发现循环依赖。
 - 当 Spring 试图填充 BeanB 的属性时，检测到它需要注入 BeanA。此时容器启动依赖查找：
 - ▶ 在一级缓存（存放完整 Bean）中未找到 BeanA；
 - ▶ 在二级缓存（存放已暴露的早期引用）中同样未命中；
 - ▶ 最终在三级缓存中定位到 BeanA 的工厂。
 - Spring 立即调用该工厂的 getObject() 方法。这个方法会执行关键决策：若 BeanA 需要 AOP 代理，则动态生成代理对象（即使 BeanA 还未初始化）；若无需代理，则直接返回原始对象。得到的这个早期引用（可能是代理）被放入二级缓存

(earlySingletonObjects)，同时从三级缓存清理工厂条目。最后，Spring 将这个早期引用注入到 BeanB 的属性中。至此，BeanB 成功持有 BeanA 的引用——尽管 BeanA 此时仍是个半成品。

- 第四步：完成 BeanB 的生命周期。
 - BeanB 获得所有依赖后，Spring 执行其初始化方法（如 PostConstruct），将其转化为完整可用的 Bean。随后，BeanB 被提升至一级缓存（singletonObjects），二级和三级缓存中关于 BeanB 的临时条目均被清除。此时 BeanB 已准备就绪，可被其他对象使用。
- 第五步：回溯完成 BeanA 的构建。
 - 随着 BeanB 创建完毕，流程回溯到最初中断的 BeanA 属性注入环节。Spring 将已完备的 BeanB 实例注入 BeanA，接着执行 BeanA 的初始化方法。这里有个精妙细节：若之前为 BeanA 生成过早期代理，Spring 会直接复用二级缓存中的代理对象作为最终 Bean，而非重复创建。最终，完全初始化的 BeanA（可能是原始对象或代理）入驻一级缓存，其早期引用从二级缓存移除。至此循环闭环完成，两个 Bean 皆可用。
- ▶ 三级缓存的设计的精髓：
 - 三级缓存工厂（singletonFactories）负责在实例化后立刻暴露对象生成能力，兼顾 AOP 代理的提前生成；
 - 二级缓存（earlySingletonObjects）临时存储已确定的早期引用，避免重复生成代理；
 - 一级缓存（singletonObjects）最终交付完整 Bean。
- ▶ 整个机制通过中断初始化流程、逆向注入半成品、延迟代理生成三大策略，将循环依赖的死结转化为有序的接力协作。
- ▶ 值得注意的是，此方案仅适用于 Setter/Field 注入的单例 Bean；构造器注入因必须在实例化前获得依赖，仍会导致无解的死锁。
- Spring 为什么用 3 级缓存解决循环依赖问题？用 2 级缓存不行吗？
 - ▶ Spring 必须用三级缓存解决循环依赖，核心是为了正确处理需要 AOP 代理的 Bean。如果只用二级缓存，会导致注入的对象形态错误，甚至破坏单例原则。
 - ▶ 举个例子：假设 Bean A 依赖 B，B 又依赖 A，且 A 需要被动态代理（比如加了 Transactional）。如果只有二级缓存，当 B 创建时去注入 A，拿到的是 A 的原始对象。但 A 在后续初始化完成后才会生成代理对象，结果就是：B 拿着原始对象 A，而 Spring 容器里存的是代理对象 A——同一个 Bean 出现了两个不同实例，这直接违反了单例的核心约束。
 - ▶ 三级缓存中的 ObjectFactory 就是解决这个问题的关键。它不是直接缓存对象，而是存了一个能生产对象的工厂。当发生循环依赖时，调用这个工厂的 getObject() 方法，这时 Spring 会智能判断：如果这个 Bean 最终需要代理，就提前生成代理对象并放入二级缓存；如果不需要代理，就返回原始对象。这样一来，B 注入的 A 就是最终形态（可能是代理对象），后续 A 初始化完成后也不会再创建新代理，保证了对象全局唯一。
 - ▶ 简单说，三级缓存的本质是“按需延迟生成正确引用”。它既维持了 Bean 生命周期的完整性（正常流程在初始化后生成代理），又在循环依赖时特殊处理，避免逻辑矛盾。而二级缓存缺乏这种动态决策能力，因此无法替代三级缓存。
 - ▶ 当 A 创建时，A 还没完成，无法立即知道是否需要代理（AOP 是在初始化后才判定的）。
 - AOP 代理的生成，不是在「实例化」或「依赖注入」阶段，而是在：初始化阶段（initializeBean）调用 BeanPostProcessor 的 postProcessAfterInitialization() 时才发生。
 - 实例化阶段创建 Bean 的空对象（裸对象），仅调用构造函数
 - 依赖注入阶段给对象注入依赖（属性赋值）
 - 初始化阶段调用各种后置处理器、@PostConstruct、InitializingBean、生成 AOP 代理等
- spring 三级缓存的数据结构是什么？
 - ▶ 都是 Map 类型的缓存，比如 Map {k:name; v:bean}。

- 一级缓存 (Singleton Objects): 这是一个 Map 类型的缓存, 存储的是已经完全初始化好的 bean, 即完全准备好可以使用的 bean 实例。键是 bean 的名称, 值是 bean 的实例。这个缓存在 DefaultSingletonBeanRegistry 类中的 singletonObjects 属性中。
- 二级缓存 (Early Singleton Objects): 这同样是一个 Map 类型的缓存, 存储的是早期的 bean 引用, 即已经实例化但还未完全初始化的 bean。这些 bean 已经被实例化, 但是可能还没有进行属性注入等操作。这个缓存在 DefaultSingletonBeanRegistry 类中的 earlySingletonObjects 属性中。
- 三级缓存 (Singleton Factories): 这也是一个 Map 类型的缓存, 存储的是 ObjectFactory 对象, 这些对象可以生成早期的 bean 引用。当一个 bean 正在创建过程中, 如果它被其他 bean 依赖, 那么这个正在创建的 bean 就会通过这个 ObjectFactory 来创建一个早期引用, 从而解决循环依赖的问题。这个缓存在 DefaultSingletonBeanRegistry 类中的 singletonFactories 属性中。
- 即使是三级依赖注入工厂对象, 又是如何知道要生成代理对象还是原始对象的?
 - spring 在暴露对象到二级缓存之前, 在 singletonFactories.put(beanName, () -> getEarlyBeanReference(beanName, beanInstance)); 这个函数里, 会询问所有的 BeanPostProcessor, 这些后置处理器可以决定: 这个 Bean 是否需要被代理。
 - Spring 判断一个 Bean 是否要被代理 (例如 @Transactional), 是通过扫描其类上的注解、切面表达式、接口匹配等静态信息实现的, 根本不用等 Bean 初始化完成

```

createBean(A)
├─ 实例化 A (原始对象)
├─ 注册 ObjectFactory(A): () -> getEarlyBeanReference(A)
├─ 注入依赖 B
│   └─ createBean(B)
│       ├─ 实例化 B
│       ├─ 注册 ObjectFactory(B)
│       ├─ 注入依赖 A
│       │   └─ 从 ObjectFactory(A) 获取早期引用
│       │       → AOP 判断 A 需要代理 → 生成 A$$Proxy
│       │       → 返回代理对象注入到 B
│       └─ 初始化 B 完成
│   └─ 放入单例缓存
├─ 初始化 A (其实已经有代理对象了)
└─ 放入单例缓存 (最终也是 A$$Proxy)
  
```

- 除了三级缓存之外, 还有没有其他办法避免这种问题?
 - 构造器注入直接拒绝循环依赖
 - 使用 @Lazy, 通过懒加载让依赖延后注入, 从而打破循环:
 - 底层原理: 注入 B 的 LazyProxy, 而不是立即创建 B 实例 → 无环
 - 禁用循环依赖 → 启动直接失败
- AOP 在哪一层缓存实现的
 - 在 Spring 的三级缓存机制中, AOP 代理对象最早是由三级缓存中的 ObjectFactory 创建的, 并被放入 二级缓存 (earlySingletonObjects) 中, 以解决循环依赖问题。所以说——AOP 代理最终出现在二级缓存中, 但其创建逻辑源自三级缓存的工厂。
- 什么是 IOC

- 即控制反转的意思，它是一种创建和获取对象的技术思想，依赖注入(DI)是实现这种技术的一种方式。传统开发过程中，我们需要通过 new 关键字来创建对象。使用 IoC 思想开发方式的话，我们不通过 new 关键字创建对象，而是通过 IoC 容器来帮我们实例化对象。通过 IoC 的方式，可以大大降低对象之间的耦合度。
- Spring IOC 实现机制
 - 反射：Spring IOC 容器利用 Java 的反射机制动态地加载类、创建对象实例及调用对象方法，反射允许在运行时检查类、方法、属性等信息，从而实现灵活的对象实例化和管理。
 - 依赖注入：IOC 的核心概念是依赖注入，即容器负责管理应用程序组件之间的依赖关系。Spring 通过构造函数注入、属性注入或方法注入，将组件之间的依赖关系描述在配置文件中或使用注解。
 - 设计模式 - 工厂模式：Spring IOC 容器通常采用工厂模式来管理对象的创建和生命周期。容器作为工厂负责实例化 Bean 并管理它们的生命周期，将 Bean 的实例化过程交给容器来管理。
 - 容器实现：Spring IOC 容器是实现 IOC 的核心，通常使用 BeanFactory 或 ApplicationContext 来管理 Bean。BeanFactory 是 IOC 容器的基本形式，提供基本的 IOC 功能；ApplicationContext 是 BeanFactory 的扩展，并提供更多企业级功能。
- 什么是 Aop
 - 是面向切面编程，能够将那些与业务无关，却为业务模块所共同调用的逻辑封装起来，以减少系统的重复代码，降低模块间的耦合度。Spring AOP 就是基于动态代理的，如果要代理的对象，实现了某个接口，那么 Spring AOP 会使用 JDK Proxy，去创建代理对象，而对于没有实现接口的对象，就无法使用 JDK Proxy 去进行代理了，这时候 Spring AOP 会使用 Cglib 生成一个被代理对象的子类来作为代理。
 - Spring AOP 实现机制
 - Spring AOP 的实现依赖于动态代理技术。动态代理是在运行时动态生成代理对象，而不是在编译时。它允许开发者在运行时指定要代理的接口和行为，从而实现在不修改源码的情况下增强方法的功能。
 - 基于 JDK 的动态代理：使用 java.lang.reflect.Proxy 类和 java.lang.reflect.InvocationHandler 接口实现。这种方式需要代理的类实现一个或多个接口。
 - 基于 CGLIB 的动态代理：当被代理的类没有实现接口时，Spring 会使用 CGLIB 库生成一个被代理类的子类作为代理。CGLIB (Code Generation Library) 是一个第三方代码生成库，通过继承方式实现代理。
 - 其实两者都是生成一个代理类来实现的，JDK 动态代理的代理类通过反射调用原方法，在反射前后调用增强逻辑，cglib 则是继承你要代理的类，并在该类上加逻辑
 - AOP 在 spring 中的应用
 - 事务管理
 - 日志记录
 - 权限控制
- 如果让你不用框架实现 spring 的 aop 和 IoC 你会怎么实现
 - IoC (控制反转) 怎么实现？
 - 自己写个 Bean 容器 BeanFactory,用 HashMap 存实例，把对象创建权从 new 转到容器。
 - 定义@Component 注解标记哪些类要托管
 - 扫描指定包，把这些类找出来
 - 通过 ClassLoader 找到所有 class 文件
 - 用反射创建对象
 - DI (依赖注入) 怎么实现？

- 类的成员变量如果需要注入，就自动填进去。
- 定义注解标记字段
- 容器 BeanFactory 初始化后，反射注入
- ▶ AOP（切面编程）怎么实现？
 - 用 动态代理 在方法前后加逻辑而不用改原代码
 - 定义一个切面注解
 - 给方法加注解
 - 使用 JDK 动态代理（可选 CGLIB）
 - 在 BeanFactory 中判断是否需要代理,把原对象替换成代理对象
- 依赖注入了解吗？怎么实现依赖注入的？
 - ▶ 而依赖注入则是将对象的创建和依赖关系的管理交给 Spring 容器来完成，类只需要声明自己所依赖的对象，容器会在运行时将这些依赖对象注入到类中，从而降低了类与类之间的耦合度，提高了代码的可维护性和可测试性。
 - ▶ 具体到 Spring 中，常见的依赖注入的实现方式，比如构造器注入、Setter 方法注入，还有字段注入。
 - ▶ 构造器注入：通过构造函数传递依赖对象，保证对象初始化时依赖已就绪。
 - ▶ SETTER 方法注入：通过 Setter 方法设置依赖，灵活性高，但依赖可能未完全初始化。
 - ▶ 字段注入：直接通过@Autowired 注解字段，代码简洁但隐藏依赖关系，不推荐生产代码。
- 运行时如何判定一个对象的类型？具体怎么用？
 - ▶ 使用 instanceof
 - instanceof 用来判断对象是否是某个类或其子类（或实现某个接口）的实例
 - 返回值是 boolean 类型（true 或 false）
 - 如果对象是 null，instanceof 会直接返回 false，不会抛出异常
 - ▶ 使用 getClass()
 - 和 instanceof 不同，它不会匹配子类；
 - getClass() 返回对象的 实际运行时类型
 - 如果要判断一个对象的精确类型，必须使用 getClass()；
- 能否通过反射拿到 class 上所有的方法（包括私有方法）？静态变量可以获得吗？
 - ▶ 可以。用 Java 反射你能拿到类上（以及父类链上）的所有方法、字段，包括私有的；静态变量也可以读取和修改，但有若干限制（模块系统 / final / 安全管理器等）。
 - ▶ getMethods(): 返回 所有 public 方法（包含从父类/接口继承的 public）。
 - ▶ getDeclaredMethods(): 返回 当前类声明的所有方法（包括私有、受保护、默认访问），但不包含父类的私有方法。
 - ▶ 如果想拿到父类链上的私有方法，需要沿着 Class getSuperclass() 逐级向上遍历并对每个类调用 getDeclaredMethods()。
 - ▶ 对于私有方法 / 字段，调用前需 method.setAccessible(true) 或 field.setAccessible(true)（在 Java 9+ 模块化后，可能触发非法反射访问/受限，需要 -add-opens 或使用 MethodHandles）。
 - ▶ 静态字段可以通过 Field.get(null) 读取，Field.set(null, value) 修改（null 表示没有实例）。如果字段是 final / 编译时常量，修改可能无效或行为不可预测。
- 反射能获取私有方法时，作用域范围是什么？能否调用私有方法？
 - ▶ 反射可以获取并调用私有方法，但前提是你 在同一个 JVM 内部 并且 有权限 绕过访问检查（通过 setAccessible(true)）
 - ▶ 作用域范围
 - 如果 不调用 setAccessible(true):
 - 只能调用当前可见范围内的成员；

- 私有方法、受保护方法、包私有方法等将抛出 `IllegalAccessException`。
- 调用 `setAccessible(true)` 后
 - 你就告诉 JVM：“我确认要跳过 Java 语言层级的访问检查”。
 - 这是在同一个 JVM、同一个进程内的反射访问；
- ▶ Java 9 引入 模块系统 (Module System)，即使 `setAccessible(true)`，跨模块访问未导出的包中的私有成员也会抛出：
- 私有方法可被反射获取，是否会导致私有属性/方法的安全问题？这种情况合理吗？
 - ▶ 从安全角度来看，这确实打破了 Java 封装性，如果被滥用，可能造成敏感数据泄露或破坏对象状态，因此在生产环境下应谨慎使用。
 - ▶ 但设计上这是有意为之：反射是框架和底层工具（如 Spring、JPA、序列化框架）实现通用功能的重要手段，它让框架能在不知道具体类结构的情况下动态创建对象、注入属性、调用方法。
 - ▶ 因此，从语言设计上讲这是合理的“受控开放”。
 - ▶ 如果确实需要限制反射访问，可以：
 - 使用 Java 模块系统 (`module-info.java`) 限制反射访问
 - 或通过框架约束和安全规范控制反射使用范围
 - 启用安全管理器 (Security Manager，虽然在新版本中逐步弃用)
 - ▶ 反射确实能访问私有成员，会带来一定安全隐患，但这是 Java 有意提供的能力，用于框架、工具等特殊场景。正常业务代码应避免直接反射操作私有成员。
- A 调用 B 的 `method1`，B 的 `method1` 调用 `this.method2`，代理会生效吗，讲下原理
 - ▶ 代理不会生效

A.call() → bProxy.method1() → target.method1() → target.method2()

- ▶ 解决方案
 - 通过代理调用自己

```
@Autowired
private UserService self; // 注意，注入的是代理对象自己

public void method1() {
    self.method2(); // ✅ 走代理
}
```

- 也可以用 `((UserService) AopContext.currentProxy()).method2()`；但要先在配置类中启用：
`@EnableAspectJAutoProxy(exposeProxy = true)`
- 方案 2：把 `method2` 移到另一个类中

```
@Service
public class B {
    @Autowired
    private C c;

    public void method1() {
        c.method2(); // 通过代理类调用
    }
}

@Service
public class C {
    @Transactional
    public void method2() { ... }
}
```


}

- 数据库主从同步延迟导致读脏数据，如何解决？

- 为什么会出现“读脏数据”

- 主从复制机制本身存在延迟。主从复制一般是：主库写入 → 写 binlog → 从库读取 binlog → 回放执行。如果主库压力大、从库落后（比如网络、IO、SQL 执行慢），就会出现从库延迟几百毫秒甚至几秒的情况。

- 解决方案

- 读写分离 + 延迟检测（读主）

```
if (isAfterWriteOperation(userId)) {  
    readFromMaster();  
} else {  
    readFromSlave();  
}
```

- 缺点：增加了读主的压力，降低了读性能。

- 写后延迟读（等待主从同步）

- 在写完后，等从库追上主库再读。
 - 基于 binlog 位点（GTID 或 File+Pos）
 - 记录写入时主库的 GTID；
 - 查询时要求从库已追到该 GTID 才允许读；

- 读写一致性缓存层（推荐）

- 写主库后，同时写缓存；
 - 读时优先读缓存；
 - 从库落后时不会读出旧数据；
 - 缓存可在主从同步后自动刷新。
 - 缓存一致性策略要设计好（写穿、失效）。

- 半同步复制 (Semi-sync Replication)

- 主库写入后必须等待至少一个从库确认接收 binlog 才返回成功。

- 分库分表后，如何解决跨分片查询？

- 为什么分库分表后会有跨分片问题

- 某些查询语句无法只在一个分片上完成：

- 聚合类查询：SELECT COUNT(*) FROM user;
 - 跨分表 JOIN：SELECT * FROM order o JOIN user u ON o.user_id = u.id;
 - 全局排序 / 分页：ORDER BY create_time LIMIT 10
 - 模糊匹配 / 范围查询：WHERE id BETWEEN 1000 AND 2000
 - 全局唯一约束 / 唯一索引

- 跨分片查询常见解决方案

- 中间件层统一路由 + SQL 合并（推荐通用方案）

- 大多数业务推荐用 ShardingSphere-JDBC（Java 应用内嵌）或 ShardingSphere-Proxy（独立代理层）；

- 原理：

- SQL 解析：中间件拦截 SQL，分析路由规则；
 - 分片路由：判断需要访问哪些分片；
 - 并行查询：分发 SQL 到对应分片；
 - 结果合并：将结果集聚合、排序、去重后返回。

- 缺点：

- 查询性能受限：跨分片 SQL 聚合代价大；

- 中间件成为瓶颈；
- 应用层聚合（手动路由 + 合并结果）
 - 应用程序根据分片键自己决定查询哪些分表；
 - 在程序中执行多次查询
 - 在内存中合并结果。
- 引入全局索引 / 全局表
 - 将经常参与关联或查询的关键字段放到一个独立全局索引表；查询时先在索引表定位，再访问目标分片
- 使用分布式数据库（自动路由 + 全局视图）
 - 代表：TiDB、OceanBase、PolarDB、CockroachDB
- 5000w 数据查询电话号码后 4 位，如何优化
 - LIKE '%1234' 不能使用普通 B-tree 索引，数据库必须全表扫描 50M 条——不可接受。解决思路是把可查询的后缀显式化为列或倒排索引，从而使用索引查找。
 - 最简单、首选：关系库的生成列 + 索引，直接给后四位单独起一列，给该列加索引
 - 倒排表 / 桶分组
 - 后 4 位只可能有 10,000 种 (0000-9999)，平均每个桶约 $50,000,000 / 10,000 = 5,000$ 条记录。可以为每个 last4 建一个桶（表或键）
 - 使用表分区（partition by list/hash on last4），查询 WHERE last4='1234' 只访问一个分区，I/O 更小。
 - 哈希/映射表（last4 -> 列表）
 - 将 last4 映射到一个存储结构（文件、KV、rocksdb），值是该桶的 user_id 列表（按需压缩/分页）。查询只需一次键查并返回 ID 列表，再去主表回填详情或直接存储所需字段。
 - 搜索引擎
 - 将 phone 字段建立 keyword 索引，或额外字段 last4 建 keyword，查询 term 非常快。
- MySQL 三范式
 - 第一范式
 - 数据表中的每一列（字段）都是不可再分的最小数据单元。

要求:

- 每个字段都只保存一个值。
- 不允许出现“集合”或“数组”类型的字段。

例子 (不符合 1NF):

学号	姓名	电话号码
1	张三	13800001111, 13800002222

📌 电话号码 列包含多个号码, 违反 1NF。

改进 (符合 1NF):

学号	姓名	电话号码
1	张三	13800001111
1	张三	13800002222

✅ 每个字段都不可分, 满足 1NF。

▸ 第二范式

- 在满足第一范式的基础上, 表中的每个非主属性 (非主键列) 必须完全依赖于主键, 而不能只依赖主键的一部分。
- 主键是 (学号, 课程号), 课程名 只依赖 课程号, 而不是 (学号, 课程号) 的组合 → 违反 2NF

▸ 第三范式

- 在满足第二范式的基础上, 非主属性不依赖于其他非主属性 (即消除传递依赖)。
- 主键是 学号, 院系名 依赖 院系号, 院系号 又依赖 学号 →, 学号 → 院系号 → 院系名 是传递依赖, 违反 3nf

• MySQL 索引的实现原理有哪些?

▸ InnoDB 是最常用的存储引擎, 它的索引采用 B+ 树 (Balance Plus Tree) 结构。

- 每个节点是一个数据页 (默认 16KB);
- 非叶子节点 存储键值和子节点指针;
- 叶子节点 存储实际数据 (或主键引用);
- 所有叶子节点通过双向链表相连, 方便范围查询。
- 为什么是 B+树 而不是 B 树?
 - B+ 树的所有数据都在叶子节点, 查询性能更稳定;
 - 叶子节点间有链表结构, 支持高效的范围查询;
 - 非叶子节点只存键值, 不存数据, 单页可容纳更多索引项, 降低树高, 提高检索效率。

▸ Hash 索引 (Memory 引擎)

- 基于 哈希表 (Hash Table) 实现;
- 只适用于等值查询 (=、IN), 不支持范围、排序;
- 时间复杂度 O(1), 但是:
 - 不支持范围扫描;
 - 不稳定 (哈希冲突会影响性能);
 - 不支持部分匹配 (like 'abc%')。

▸ R-Tree (空间索引)

- 用于地理空间类型（如 GEOMETRY、POINT）；
- 支持矩形范围查询；
- 应用场景如地图位置匹配。
- 全文索引（Full-Text Index）
 - 实现原理：倒排索引（Inverted Index）；
 - 维护词 -> 文档 ID 的映射；
 - 适合文本搜索（如新闻标题、商品描述）。
- 为什么为什么 B+树层数代表着 io 次数
 - 我们说 B+树的层数 = 查询时的 I/O 次数（或接近），其实是在描述数据库索引在磁盘中的查找过程，因为 每访问一层节点，就意味着一次磁盘 I/O。
 - 这是因为 B+ 树的节点都存储在磁盘页（page）中，并不是都在内存里。
 - 也就是说 b+树每个节点的数据都存储在磁盘上 每次访问节点都要从磁盘上取
- 什么时候联合索引失效
 - 违反最左匹配原则
 - 最左列使用了范围查询后再查别的列
 - like 不是前缀匹配 如 '%Tom%' 开头会导致索引失效
 - 使用函数或运算
 - 发生隐式类型转换可能失效
 - OR 混用未建索引列 WHERE age = 30 OR gender = 'M' gender 无索引导致整体失效
 - 查询优化器判断全表扫描更优
- 用过 explain 吗？介绍其返回结果中主要字段的意义。
 - EXPLAIN 用于分析 MySQL 查询语句的执行计划，帮助我们了解优化器是如何访问表、使用哪些索引、扫描了多少行数据。
 - 重点字段深入说明
 - type（访问类型）是最关键指标，反映查询性能的好坏，常见取值从好到坏：
 - SYSTEM：表中只有一行数据（系统表）
 - const：通过主键或唯一索引一次命中
 - eq_ref：多表连接时，主键或唯一索引匹配
 - ref：非唯一索引或前缀索引匹配
 - range：范围扫描索引
 - index：全索引扫描
 - ALL：全表扫描，性能最差
 - 优化目标是尽量让 type 到达 range 级别以上（最好是 ref 或 const）。
 - Extra 常见取值（反映优化空间）
 - using index：使用了覆盖索引（性能好）
 - using where：需要通过 WHERE 过滤数据
 - using temporary：使用了临时表（性能差）
 - using filesort：使用了文件排序（性能差）
 - Using join buffer：使用了连接缓存（说明没走索引）
 - Impossible WHERE：条件恒为假
 - Select tables optimized away：优化器优化掉了子查询
 - key：实际使用的索引名称
 - rows：估算需要扫描的行数，行数越少越好
 - filtered：表示经过 WHERE 过滤后剩余行的比例
- 基于“主键为 xxxid，查询未删除（软删，有 deleted_at 字段）的数量，explain 显示扫描 10 条，filter 命中 50%”的场景，说明 SQL 执行时做了什么事情？

- 解析 SQL 语句、确定表和列，检查语法、确认字段名
 - 优化阶段，选择执行计划，决定是否走索引
 - MySQL 优化器分析 WHERE 条件：
 - deleted_at IS NULL
 - 如果 deleted_at 没有索引，优化器决定 全表扫描 (type=ALL)；
 - 如果有索引但选择性低（很多都是 NULL 或非 NULL），优化器可能仍选择 不走索引；
 - 执行器拿到执行计划后，做以下几件事：
 - 从存储引擎读取行数据
 - 若 type=ALL，执行器会扫描整个表（假设有 10 行数据）；
 - 若 type=range，则按索引范围扫描部分行。
 - 逐行应用过滤条件
 - 对每行判断 deleted_at IS NULL；
 - 统计符合条件的行数
 - 对于 COUNT(*)，不返回行内容，只增加计数器。
 - 返回聚合结果
 - 从 binlog 层面介绍上述 SQL 执行过程中的相关操作。
 - 这条 SQL 是 查询语句 (SELECT)，因此它是只读操作，不会对数据产生修改。所以它不会生成 binlog 日志。
 - binlog 记录所有会修改数据库内容的语句或行事件（用于主从复制、数据恢复）。
- 介绍 MySQL 事务？
1. 事务的定义
 - 事务 (Transaction) 是一组操作的逻辑单元，这些操作要么全部执行成功，要么在发生错误时全部回滚（撤销）。
 2. 事务的四大特性 (ACID)
 - 原子性
 - 一致性
 - 隔离性
 - 持久性
 3. 事务的隔离级别
 4. 事务实现原理
 - Redo Log 持久性
 - undo log 原子性
 - MVCC 可重复读和并发性能
 - 锁机制（行锁/间隙锁）保证隔离性
- MySQL 事务隔离级别？分别解决什么问题？
- 读未提交
 - 解决脏读问题
 - 每次 select 生成快照
 - 脏读：读到事务未提交的数据
 - 可重复读
 - 解决脏读和不可重复读问题
 - sql 标准：锁当前行，读数据加共享锁，写数据加排他锁
 - 不可重复读：两次读到的数据不一样
 - 可串行化
 - 强制加锁，串行执行
 - 解决幻读问题

- 幻读：两次读到的数据条数不一样
- sql 标准的 rr 和 innodb 的 rr 不一样
 - SQL 标准的 RR 只保证“已读行不变”，但不锁定查询范围，所以其他事务可以插入新行导致“幻读”。RR 在标准里 ≠ “事务级快照”，而是“行级锁定可重复读”。
 - InnoDB 的 RR 加了 MVCC + 间隙锁，既能保持行内容一致，又能锁定范围，从而防止幻读。
- 可重复读如何实现的
 - 可重复读（Repeatable Read）的实现核心是：在同一个事务中，多次读取同一条数据时，保证结果一致，即防止不可重复读。实现方式主要依赖于 锁机制 或 多版本控制（MVCC）
 - 基于锁的实现（Pessimistic Lock）
 - 当事务读取数据时，会对读取的行加共享锁（S 锁）。
 - 如果事务要修改数据，会申请排他锁（X 锁）。
 - 其他事务在未释放共享锁前，不能修改这条数据。
 - 幻读可能通过范围锁（Range Lock）进一步解决（MySQL InnoDB 可加间隙锁）。
 - 基于 MVCC 的实现（Optimistic, MySQL InnoDB 的实现方式）
 - 数据库为每行数据维护一个版本号或时间戳。
 - 事务开始时，会记录当前数据库版本号（snapshot）。
 - 事务读取数据时，总是读取该事务开始时存在的最新版本。
 - 写操作仍然会加行锁，保证写一致性。
 - 幻读仍可能出现（除非加额外间隙锁）。
- 水平分表和垂直分表和水平分库和垂直分库
 - 水平分表
 - 一张表的数据太多，把同一张表的数据行按照某种规则拆分到多张表中
 - 垂直分表
 - 一张表字段太多或访问频率差异大，把字段按业务或访问频率拆到多张表中
 - 水平分库
 - 一个数据库太大或访问量太高，把相同结构的表拆到多个数据库中
 - 垂直分库
 - 一个数据库中表太多、业务耦合，把不同业务模块的表分到不同数据库中
 - 水平分库二度
 - 在做完第一次水平分库之后，再在每个分库内部继续进行一次水平拆分。
 - 假设系统的用户数据非常大，我们按 `user_id % 4` 拆分为 4 个库：
 - 当每个库内部数据仍然太大或访问压力过高时，我们就可以在每个一级分库内部再水平拆分一次。
 - 也就是说：
 - 一级按 库维度 分库
 - 二级再按 库内表维度 分表
 - 热点数据水平分表是根据什么依据分
 - 哈希随机化
 - 热点问题往往是因为：某个 key（如热门商品、热门用户）被集中访问。
 - 如果你仅用：`table_index = product_id % 4`
 - 那么所有访问热门商品 `product_id = 1001` 的请求都会落在同一个表。
 - 解决方案是：对 key 再加一层随机化打散
 - `table_index = hash(product_id + random_suffix) % 4`
 - 于是商品 1001 的访问可能被打散成：

product_1001#0 → 落在 product_0
product_1001#1 → 落在 product_1
product_1001#2 → 落在 product_2
product_1001#3 → 落在 product_3

‣ 热点请求不再集中到一张表上，而是平均分布。

- 热点打散

- 对单一热点 key 再人为拆成多个子 key
- 缺点：应用层要合并统计结果

- 读写分离 + 缓存

- 把热点数据缓存或读写分流
- 缺点：缓存一致性需处理

- 预分片

- 事先为可能成为热点的 key 预建多个表

• 三类存储引擎分别支持哪些索引？

存储引擎	支持的索引类型	说明	📄
InnoDB	B+树索引（默认）、唯一索引、主键索引、全文索引（FULLTEXT，从 5.6 起 InnoDB 支持）、空间索引（SPATIAL，MySQL 5.7+）	聚簇索引：主键就是聚簇索引，数据行按主键顺序存储。支持外键约束。	
MyISAM	B+树索引（默认）、唯一索引、主键索引、全文索引（FULLTEXT）、空间索引（SPATIAL）	非聚簇存储，数据和索引分开存储，全文索引支持比 InnoDB 更早。	
Memory	哈希索引（默认）、B+树索引、唯一索引、主键索引	哈希索引用于快速等值查找，但范围查询效率低，B+树可以支持范围查询。数据存储在内存中，重启丢失。	↓

‣ InnoDB —— B+ Tree 索引 + 聚簇索引

- 主键索引：聚簇索引（Clustered Index）
 - 叶子节点存储整行数据。
- 二级索引（辅助索引）：B+ Tree
 - 叶子节点存储主键值，而非数据本身。
- 还使用了自适应哈希索引（Adaptive Hash Index），自动维护高频查询的哈希缓存。

‣ MyISAM —— B+ Tree 索引 + 非聚簇结构

- 主索引与数据分离：
 - .MYI 文件存索引；
 - .MYD 文件存数据。
- 叶子节点存的是数据文件的物理地址（偏移量）。
- 读操作非常快，但不支持事务与崩溃恢复。
- 读操作流程
 - 解析 SQL
 - MySQL 解析器分析 SQL，生成执行计划。
 - 确定使用哪个索引（例如 PRIMARY KEY(id)）。
 - 访问索引文件（.MYI）
 - MyISAM 的索引通常是 B+ 树。
 - 根据索引查找对应的记录在数据文件中的偏移地址（row offset）。

- 访问数据文件（.MYD）
 - 根据索引返回的偏移量，直接跳到 .MYD 文件对应位置。
 - 读取数据记录并返回。
- MEMORY —— Hash 索引（默认）或 B+Tree
 - 默认使用 Hash 索引（等值查找快，范围查找差）；
 - 也可以手动指定 USING BTREE；
 - 数据存于内存，掉电即失。
- 不同存储引擎的优缺点？
 - InnoDB
 - 优点：
 - 支持事务（ACID），保证数据一致性。
 - 行级锁，高并发写入性能好。
 - 支持外键约束，保证参照完整性。
 - 默认采用聚簇索引，查询主键非常快。
 - 支持崩溃恢复（redo log、undo log）。
 - 缺点：
 - 相比 MyISAM，占用空间大。
 - 全文索引支持起步晚（MySQL 5.6+）。
 - 写入稍慢（事务、行锁、日志开销）。
 - MyISAM
 - 优点
 - 存储格式简单，查询速度快，尤其是读密集型场景。
 - 全文索引支持早，适合全文搜索。
 - 占用空间比 InnoDB 小。
 - 缺点：
 - MyISAM 不支持事务（Transaction），也没有 ROLLBACK、COMMIT、SAVEPOINT 等机制。
 - 仅支持表级锁（Table Lock）
 - MyISAM 表结构简单但容易损坏（尤其是异常关机或系统崩溃时）。
 - 不支持外键（Foreign Key）
 - 写入性能差（在高并发环境下）
 - 不支持崩溃恢复与日志（无 redo/undo log）
 - 以读为主的系统（如日志分析、搜索引擎索引）；
 - 不要求事务一致性的离线分析场景；
 - 小型嵌入式应用或内存受限系统。
 - Memory
 - 优点：
 - 数据全部存储在内存，读写速度极快。
 - 支持表级锁、哈希索引，精确查找速度快。
 - 缺点：
 - 数据断电或重启丢失，不持久化。
 - 哈希索引只适合精确匹配，范围查询性能差。
 - 表大小受内存限制，通常用于临时表或缓存。
 - CSV
 - 每行数据存成 CSV 文本，易于导入导出
- MySQL 跟 hbase 区别

- 聚簇索引和非聚簇索引
 - 聚簇索引：通常是 B+ 树叶子节点存储实际数据行。查询主键数据直接命中叶子节点，无需再回表。
 - 非聚簇索引：B+ 树叶子节点只存索引列 + 指向数据行的地址（InnoDB 是主键，MyISAM 是物理行地址）。查询非主键列时，需要先查索引，再根据 rowid 回表取数据。
- MySQL 的索引说一下，整个索引查找的过程
 - 索引的类型
 - 主键索引 (PRIMARY KEY)
 - 唯一索引 (UNIQUE)
 - 普通索引 (INDEX)
 - 联合索引 (Composite Index)
 - 全文索引 (FULLTEXT)
 - 哈希索引 (MEMORY 引擎)
 - InnoDB 索引的底层结构：B+Tree
 - 每个节点对应一个磁盘页（默认 16KB）
 - 内部节点（非叶子节点）只存储键值（Key）和子节点指针。
 - 叶子节点存储完整数据行（聚簇索引）或主键引用（辅助索引）。
 - 所有叶子节点之间用双向链表连接，方便范围查询。
 - 索引查找过程详解
 - 主键索引
 - 根节点：从磁盘加载根页（B+Tree 顶层）。
 - 比较 id：根页里存放键范围（如 id=1, 5, 10, 20），找到指针指向的子节点页。
 - 向下查找：进入对应的子页继续二分查找。
 - 叶子节点：找到 id=8 的叶子节点项。
 - 返回整行数据（因为聚簇索引叶子节点就包含完整行）。
 - 二级索引
 - 从 name 索引树查找
 - 找到“Alice”对应的记录：('Alice', id=8)。
 - 用上面找到的 id=8 再去主键 B+Tree 查找，回表
 - 走一次主键索引，找到完整行。
- MySQL 的主键索引为什么要定义这个主键呢？
 - InnoDB 的数据文件本质上是一棵以主键为有序排列的 B+ 树，所以必须要有一个主键；如果你不定义，它也会想办法给你造一个。
 - 如果你不定义主键：
 - 有唯一非空的键 → 使用它当主键
 - 没有 → MySQL 自动生成一个 6 字节的隐藏 rowid 主键
 - 主键影响所有二级索引（因为二级索引都要回表）
 - InnoDB 的二级索引不存储 row address，它们存储的是：索引值 + 主键值
 - 如果你的主键很大（例如 UUID 36 字节），二级索引膨胀非常严重
 - 二级索引体积变大
 - 内存 buffer pool 命中率下降
 - 查询性能变差
 - 磁盘占用增加
 - 这就是为什么建议主键：
 - 自增 INT/BIGINT

- 越小越好
- 主键是否递增影响写入性能（页分裂严重）
 - 如果主键是自增，新增数据永远写在 B+ 树的尾部，几乎不会：
 - 页分裂
 - 页移动
 - B+ 树重平衡
 - 如果主键是随机值（如 UUID）：
 - 新行可能落到 B+ 树中间位置
 - 导致频繁页分裂、磁盘碎片、B+ 树重排
 - 写入性能极差
- 主键影响锁（行锁、间隙锁）的粒度与范围
 - InnoDB 行锁是按主键索引加的。
 - 选择一个递增紧凑的主键，可以：
 - 减少锁范围
 - 减少间隙锁和幻读冲突
 - 提高并发减小死锁概率
- 索引下推
 - 把部分 WHERE 条件的判断逻辑下推到存储引擎层，在扫描索引时就能过滤掉不满足的行，从而减少回表次数。
 - MySQL Server 层把一部分条件（涉及索引列的）下推给存储引擎
 - 没有索引的过滤条件在存储引擎把记录交回给 server 层处理
 - 存储引擎在扫描索引时，直接判断这些条件；
 - 仅当索引层条件满足时，才去回表取整行。
- MySQL 查询的逻辑执行顺序
 - FROM → WHERE → GROUP BY → 聚合函数计算 → HAVING → SELECT → ORDER BY → LIMIT
- MySQL 执行 select 语句的流程，特别说明 buffer pool 的作用
 - 客户端连接
 - 查询缓存（MySQL 8.0 已移除）
 - 语法解析与预处理，生成解析树（Parse Tree）。
 - 查询优化器
 - 决定使用哪个索引（索引选择）。
 - 选择连接顺序、是否使用排序、是否使用临时表。
 - 最终生成执行计划（Execution Plan）。
 - 执行器（Executor）
 - 执行器根据优化器提供的执行计划，调用存储引擎接口取数据。
 - 这时进入 InnoDB 层
 - 检查 Buffer Pool（缓冲池）
 - InnoDB 不会直接从磁盘读取数据页，而是先看 Buffer Pool 里是否有目标页。
 - Buffer Pool 是什么？
 - 是 InnoDB 用来缓存数据页、索引页、Undo 页等内存区域。
 - 一个页（Page）通常为 16KB。
 - 它的本质是一个内存缓冲区，用于减少磁盘 I/O。
 - 若数据页已在 Buffer Pool 中（命中缓存）
 - 直接从 Buffer Pool 读取数据。无需访问磁盘，速度极快（内存级别）。
 - 若数据页不在 Buffer Pool 中（未命中）

- InnoDB 通过页号 (Page ID) 从磁盘读取相应的数据页。
- 将该页加载到 Buffer Pool 中。
- 如果 Buffer Pool 已满, 触发 LRU (最近最少使用) 算法淘汰旧页。
- 脏页刷新 (Flush)
 - 当数据被修改 (例如 UPDATE) 后, 对应页会被标记为脏页 (dirty page)。
 - 脏页不会立即写回磁盘, 而是异步地通过后台线程 (innodb_flush_method) 定期刷新。
 - 查询 (SELECT) 不会产生脏页, 但依赖读取缓存页。
- 返回结果
 - 存储引擎层取出数据页中的行, 返回给 Server 层。
 - Server 层进一步:
 - 做过滤 (WHERE 条件);
 - 排序 (ORDER BY);
 - 分组 (GROUP BY);
- MySQL 中聚合函数之后可以使用子查询嘛?
 - 可以, 但要分场景。
 - 在 SELECT 中使用子查询 (允许)
 - 在 WHERE 中使用聚合函数不允许
 - WHERE 在 GROUP BY 之前执行; 此时聚合函数还没执行, AVG(salary) 还不存在;
 - HAVING 可以使用聚合函数
 - HAVING 在 GROUP BY 之后执行, 此时聚合结果已生成, 可以用 AVG(salary) 过滤分组;

```
SELECT department_id, AVG(salary)
FROM employees
WHERE AVG(salary) > 5000 -- ✗ 报错
GROUP BY department_id;
```

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 5000;
```

- 在 FROM 子句中使用子查询 (聚合在子查询内)
- 在聚合函数内部使用子查询 (允许但要小心性能)

```
SELECT
  SUM((SELECT MAX(price) FROM products WHERE category_id = c.id)) AS
  total_max_price
FROM categories c;
```

- 每行都要执行一次子查询, 性能可能很差。

- mysql 中的主流的日期函数?
 - NOW() 2025-10-27 16:05:33
 - SYSDATE() 2025-10-27 16:05:33
 - CURDATE() 2025-10-27
 - CURRENT_TIMESTAMP 2025-10-27 16:05:33
 - CURTIME() 16:05:33
 - DATE_ADD(date, INTERVAL n unit) SELECT DATE_ADD('2025-10-27', INTERVAL 7 DAY);
2025-11-03
 - SELECT DATE_FORMAT(NOW(), '%Y/%m/%d %H:%i:%s');
- MySQL 中查询前两天的数据该使用什么时间函数?

```

SELECT *
FROM your_table
WHERE date_column >= DATE_SUB(CURDATE(), INTERVAL 2 DAY)
AND date_column < CURDATE();

```

OR

```

SELECT *
FROM your_table
WHERE date_column >= NOW() - INTERVAL 2 DAY;

```

- mysql 中 delete 和 drop 的区别?

- 删除表中的部分数据

```
DELETE FROM user WHERE id = 5;
```

- 删除表中所有数据

```
DELETE FROM user;
```

- 删除整个表

```
DROP TABLE user;
```

-删除整个数据库 DROP DATABASE testdb;

- DELETE 可以使用 WHERE 子句来删除符合条件的行，而 DROP 是直接删除整个表或数据库，无法指定条件。
- DELETE 删除数据后，表结构和索引仍然存在，而 DROP 会删除表结构和所有相关的索引。
- DELETE 操作可以通过事务回滚，而 DROP 操作一旦执行，数据无法恢复。
- DELETE 会触发触发器，而 DROP 不会触发任何触发器。
- DELETE 不会立即释放磁盘空间（仅标记删除），而 DROP 会立即释放表所占用的所有磁盘空间。

- 什么是触发器

- 触发器 (Trigger) 是一种由数据库自动执行的特殊存储过程，当特定的数据库事件（如 INSERT、UPDATE、DELETE）在某张表上发生时，数据库会自动执行触发器中定义的逻辑。
- 在 MySQL 中，触发器分为两种触发时机：
 - BEFORE 触发器：在数据操作之前执行，可以用来验证或修改即将插入/更新的数据。
 - AFTER 触发器：在数据操作之后执行，通常用于记录日志或

```

-- 创建触发器：在插入 user 表之前，自动将 name 转为大写
CREATE TRIGGER before_user_insert
BEFORE INSERT ON user
FOR EACH ROW
BEGIN
    SET NEW.name = UPPER(NEW.name);
END;

```

- 常见用途

- 数据验证和清洗
- 自动生成衍生数据
- 维护审计日志
- 实现复杂的业务规则

- mysql 中 truncate 和 delete 的区别?

- delete 可以条件删除，truncate 清空整张表

- delete 会逐行删除，触发删除触发器，性能较慢；truncate 是快速删除，重置表，不能触发删除触发器，性能快。
- delete 删除的数据可以回滚，truncate 删除的数据不能回滚。
- truncate 会重置自增 ID，delete 不会。
- truncate 是 DDL 语句，会隐式提交事务，delete 是 DML 语句
- DDL 和 DML
 - DDL 管结构，定义数据库对象。
 - DML 管数据，操作数据库内容。

✅ 二、功能区别对比

对比项	DDL（数据定义语言）	DML（数据操作语言）
作用对象	表结构、数据库结构（定义/修改结构）	表中的数据（增删改查）
主要操作	CREATE, ALTER, DROP, TRUNCATE, RENAME	INSERT, UPDATE, DELETE, SELECT
是否自动提交事务	✅ 一般自动提交，不可回滚	❌ 不自动提交，可回滚
执行结果	改变表结构、数据库结构	改变表中存储的数据
日志记录	通常只记录结构变更日志（不是逐行）	逐行记录数据变更日志（可用于回滚）
示例	ALTER TABLE user ADD COLUMN age INT;	UPDATE user SET age = 20 WHERE id = 1;

- MySQL 中行转列，列转行如何操作？
 - 行转列

假设我们有一个成绩表 `score`：

name	subject	score
张三	语文	80
张三	数学	90
李四	语文	85
李四	数学	70

✅ 目标结果（行转列后）

name	语文	数学
张三	80	90
李四	85	70

- 使用 CASE WHEN 手动透视（推荐方式）

```

SELECT
    name,
    MAX(CASE WHEN subject = '语文' THEN score END) AS 语文,
    MAX(CASE WHEN subject = '数学' THEN score END) AS 数学
FROM score
GROUP BY name;

```

– 用 WHERE

- 你只能做两个查询然后手动去 join:

```

SELECT
    a.name,
    a.语文,
    b.数学
FROM
    (SELECT name, score AS 语文 FROM score WHERE subject='语文') a
LEFT JOIN
    (SELECT name, score AS 数学 FROM score WHERE subject='数学') b
ON a.name = b.name;

```

- 结果虽然一样，但需要写两次子查询和一次 JOIN,

▸ 列转行 (Column → Row)

一、列转行 (Column → Row)

也叫“**逆透视 (Unpivot)**”，指的是将列名转成行值。

示例数据

name	语文	数学
张三	80	90
李四	85	70

目标结果 (列转行后)

name	subject	score
张三	语文	80
张三	数学	90
李四	语文	85
李四	数学	70



– 方法：使用 UNION ALL

```

SELECT name, '语文' AS subject, 语文 AS score FROM score_table
UNION ALL
SELECT name, '数学' AS subject, 数学 AS score FROM score_table;

```

- 了解过游标吗？

▸ 游标 (Cursor) 是一种能让你“逐行”处理查询结果集的机制。

- 通常我们执行 SQL 查询时，是一次性返回整个结果集。但如果你希望 逐行地读取、判断、处理 结果集（比如循环更新、计算、条件控制），这时候普通 SQL 不够用了，就可以用“游标”。
- 游标的使用
 - 定义

```
DECLARE emp_cursor CURSOR FOR
SELECT id, name, salary FROM employee WHERE salary < 5000;
```

- 声明变量（用于接收每行数据）

```
DECLARE v_id INT;
DECLARE v_name VARCHAR(100);
DECLARE v_salary DECIMAL(10,2);
```

- 打开游标

```
OPEN emp_cursor;
```

- 读取数据，使用 FETCH 从游标中逐行取数据：

```
FETCH emp_cursor INTO v_id, v_name, v_salary;
```

- 通常会配合一个循环结构：

```
read_loop: LOOP
  FETCH emp_cursor INTO v_id, v_name, v_salary;
  IF done THEN
    LEAVE read_loop;
  END IF;
```

- 关闭游标

```
CLOSE emp_cursor;
```

- 有两个数据库，一个查询数据库，一个写入数据库，如何使用 mybatis 随意切换这两个数据库？
 - 在 Spring Boot 的配置文件里配置两个数据库连接
 - 定义枚举标识不同数据源
 - 使用 ThreadLocal 保存当前线程使用的数据源
 - 创建动态数据源路由类
 - 这个类继承 Spring 的 AbstractRoutingDataSource，根据 ThreadLocal 动态返回目标数据源
 - 注册 Bean（组合两个数据源）
 - 配置 MyBatis 使用动态数据源
 - 提供注解方式切换数据源（推荐）
 - 这样，你在方法上加个注解即可随意切换数据库。
- mabatis 中如何进行批量插入
 - 使用 <foreach> 标签批量插入（最常见、推荐）

```
<!-- Mapper.xml -->
<insert id="insertBatch" parameterType="list">
  INSERT INTO user (name, age)
  VALUES
  <foreach collection="list" item="item" separator=",">
    (#{item.name}, #{item.age})
  </foreach>
</insert>
```

- ▶ 使用 MyBatis 的批处理模式 (ExecutorType.BATCH)
 - 这种方式使用 MyBatis 的底层批处理特性，每次调用 insert() 都不会立即发送 SQL，而是累积到批中，最后统一执行。

```
SqlSession sqlSession = sqlSessionFactory.openSession(ExecutorType.BATCH);
UserMapper mapper = sqlSession.getMapper(UserMapper.class);
```

```
for (int i = 0; i < 10000; i++) {
    User user = new User();
    user.setName("User_" + i);
    user.setAge(20 + i % 10);
    mapper.insert(user); // 单条 insert
}
```

```
sqlSession.flushStatements(); // 执行批量提交
sqlSession.commit();
sqlSession.close();
```

- Mysql 死锁产生的情况
 - ▶ 不同顺序更新相同表的行
 - 如事务一先 update 行 1 等待 update 行 2, 事务二 update 行 2 等待 update 行 1, 两个事务都要修改对方持有的行 → 死锁
 - 解决方法：遵循固定的加锁顺序（比如：总是先按主键从小到大加锁）
 - ▶ 唯一索引冲突导致隐式锁等待

```
-- 事务1
BEGIN;
INSERT INTO user VALUES (1, 'Tom'); -- 成功，占用 name='Tom' 唯一索引锁
```

```
-- 事务2
BEGIN;
INSERT INTO user VALUES (2, 'Tom'); -- 等待唯一索引锁
```

- 如果事务 1 在提交前又去修改别的行，事务 2 也去修改那行，就可能出现交叉等待而死锁。

- ▶ 间隙锁导致死锁

sql

Copy code

```
BEGIN;
SELECT * FROM t WHERE age > 10 FOR UPDATE; -- 锁住 (10, +∞)
INSERT INTO t VALUES (4, 25); -- 尝试插入 age=25
```

事务 2:

sql

Copy code

```
BEGIN;
SELECT * FROM t WHERE age < 30 FOR UPDATE; -- 锁住 (-∞, 30)
INSERT INTO t VALUES (5, 15); -- 尝试插入 age=15
```

💣 死锁分析

- 事务1 锁 $(10, +\infty)$ ，插入 25 时等待 $(20, 30)$ 间隙；
- 事务2 锁 $(-\infty, 30)$ ，插入 15 时等待 $(10, 20)$ 间隙；
- 两者互相等待对方释放间隙 → **形成死锁**。

这类死锁非常常见在：



- 说说死锁必要条件
 - 互斥条件 (Mutual Exclusion)
 - 资源一次只能被一个线程/进程占用。
 - 请求并保持条件
 - 线程在持有至少一个资源的同时，又提出对其他资源的请求；
 - 不可剥夺条件 (No Preemption)
 - 资源不能被强制抢夺，只能由持有它的线程主动释放。
 - 循环等待条件
 - 存在一个资源循环等待链，每个线程都在等待下一个线程持有的资源。
- 如何破坏死锁
 - 破坏互斥条件 (让资源可共享)
 - 使用 读写锁 (ReadWriteLock)：读共享、写独占，减少互斥概率。
 - 无锁编程 (CAS、自旋)
 - 破坏请求保持条件
 - 线程要么一次性拿到所有资源，要么一个都拿不到。
 - 所有资源集中申请
 - 带超时的锁，失败后会释放已获得的资源 → 不满足“请求保持”。
 - 破坏不可剥夺条件
 - 当线程申请不到新资源时，释放已占有资源，让出机会。
 - 破坏循环等待条件 (Circular Wait)
 - 资源按固定顺序加锁
 - 为所有资源编号，线程必须按编号顺序加锁
- shardingSphere 工作流程
 - SQL 接收阶段
 - 应用通过 JDBC (或 Proxy) 发送 SQL 给 ShardingSphere。
 - ShardingSphere 先接收 SQL 语句和参数。
 - SQL 解析阶段 (SQL Parser)
 - 使用 ANTLR 解析 SQL，生成语法树 (AST)。

- 类似数据库的语法分析过程。
 - 路由阶段 (SQL Router)
 - 根据分片规则 (Sharding Rule) 决定 SQL 去哪些库、哪些表。
 - 改写阶段 (SQL Rewriter)
 - ShardingSphere 将原始 SQL 改写成对应分片 SQL。
 - 执行阶段 (SQL Executor)
 - ShardingSphere 并行向各目标数据库发送 SQL。
 - 可配置连接池、线程池并发执行。
 - 结果归并阶段
 - 当返回结果来自多个分片时
 - ShardingSphere 在内存中进行结果合并
 - 可合并, 排序, 聚合, 分页
 - 返回结果
- 了解 elasticsearch 吗
 - 如果传统 mysql 查询上百万级的数据的模糊查询, 需要全表遍历非常慢
 - elasticsearch 采用倒排索引提升效率
 - 倒排索引
 - 将所有数据进行分词
 - 如一个句子 i like study 分成三个 term
 - 每个句子对应一个文本 id, 那每个句子的文本 id 都对应若干个词
 - 这样, 可以构建以 term 为 key, 文本 id 为 value 的键值对结构, 就可以知道每个词对应在哪一个句子有
 - 但是 term 的数量也有很多, 如果不做处理顺序遍历 term 是 O(n) 也很慢, 于是将 term 按字典序排列, 这样查找的时候就可以二分查找, 提高效率
 - 这样排列好后, term 这一列就叫做 term dictionary, 文本 id 这一列就叫做 posting list
 - 倒排索引问题
 - term dictionary 的数量很大, 放到内存不现实, 只能放到磁盘, 但是查磁盘又很慢, 如何优化?
 - term index
 - 由于每个 term 都会有相同的前缀, 将相同的前缀合并指向后缀就可以节省空间
 - 因此可以构建一个目录树, 每个带有前缀信息的节点存放这些词项在磁盘中的偏移量, 也就是指向磁盘中的位置
 - 目录树体积小, 适合放到内存中
 - stored fields
 - 由于前面倒排索引存放的是句子的 id, 我们拿到 id 之后还需要找到原句子
 - 原句子就需要一个空间去存储, 我们把这个空间叫做 stored fields, 是一个行式存储结构
 - doc value
 - 由于业务中我们经常需要对一些字段进行排序查询, 如果每次先把和字段有关的数据读出来再查询就很慢
 - 于是我们采用一种空间换时间的方法, 将字段集中存放, 这个存放的地方就是 doc value, 这样排序的时候只需要在 doc value 中读出来排序即可, doc value 是列式结构
 - segment
 - 以上四个结构: 倒排索引, term index, sorted fields, doc value 组成一个复合文件, 是具有搜索功能的最小单元
 - lucene
 - 如果多个文件读写到同一个 segment 时, 要对 segment 多个文件进行修改, 并发时性能受影响

- 于是我们定义 segment 在生成后无法修改, 如果要写新的文件就新生成 segment, 这样 segment 只负责读, 新 segment 负责写, 并发性能提高
- 如果 segment 数量越来越多, 我们查找数据的时候就要并发读 segment, 保证读效率
- segment merging
 - 随着 segment 越来越多, 文件句柄有被耗尽的可能, 因此会定期将小 segment 合并成大 segment
- 到此为止, 就构成了一个单机文本检索库 lucene
- ▶ 但是 lucene 并不具备高性能, 高可用, 高扩展, 因此需要优化
 - 高性能优化
 - 如果多个服务同时读 lucene 肯定会造成资源占用导致没有抢占到的服务等待
 - 我们可以将不同类别的信息分到不同的 lucene, 每个 lucene 有各自的 index name, 这样服务访问不同的信息时可以访问不同的 lucene, 可以降低单个 lucene 压力
 - 但是这样单个 index name 的数据依然可能过多
 - 因此我们可以将单个 index name 的数据进行分片, 每一分片的数据叫做一个 shard, 每一个 shard 本质上就是一个独立的 lucene 库, 这样多个读写操作可以分摊到多个 shard
 - 高扩展性
 - 随着分片变多, 如果多个分片都在同一个机器上, 导致单机 cpu 和内存占用过高
 - 于是我们可以增加机器, 每个机器存放一部分分片, 这些机器就叫做 node
 - 高可用
 - 如果 node 挂了, 那 node 里所有分片都无法对外提供服务
 - 可以给 node 增加多个副本, 分为 primary shard 和 replica shard
 - 主分片将数据同步给副本分片, 副本分片可以提供读操作, 也可以在主分片挂了的时候升级成新的主分片
- ▶ node 角色分化
 - 一个 node 含有的功能很多, 如集群管理, 存储数据, 处理请求, 如果其中一个功能的能力不足需要扩容节点, 就会将其他的功能也扩容, 但是其他功能并不需要这么强的能力, 造成资源浪费
 - 因此可以将功能拆开分到不同的 node, 每个 node 负责不同的功能
- ▶ 去中心化
 - 在 node 间引入魔改后的 raft, 在节点间互相同步数据, 让所有 node 看到的集群数据状态一致, 这样集群内的 node 就能参与选主过程, 还能了解到集群内的其他节点是否挂了
- ▶ ES
 - 以上组件合在一起就是 es, 他对多个语言提供 http 接口
- ▶ 写入流程
 - 当客户端发起请求写入到 es 时
 - 请求会先到协调节点, 协调节点根据 hash 路由判断数据应该写入到哪个数据节点的哪个分片, 找到主分片并写入
 - 分片底层是 lucene, 所以最终是将数据写入到 lucene 库里的 segment 内
 - 将数据固化为倒排索引, termindex, storedfields 以及 docvalues 多个结构
 - 主分片数据写入后会将数据同步给副本分片
 - 副本分片写完后, 主分片会响应给协调节点一个 ack
 - 最后协调节点响应客户端写入完成
- ▶ 搜索流程
 - 查询阶段
 - 客户端发起请求到协调节点
 - 协调节点根据 indexname 的信息可以了解到 indexname 被分为了几个分片, 以及这些分片分散到哪个数据节点
 - 将请求转发到这些数据节点的分片上

- 分片底层的 lucene 库并发搜索多个 segment
- 利用每个 segment 内的倒排索引获得文档 id,并结合 docvalues 获得排序信息
- 分片将结果聚合返回给协调节点
- 协调节点对信息进行排序聚合,舍弃大部分不需要的数据
- 获取阶段
 - 协调节点再次拿着 id 去请求数据节点里的分片
 - 分片底层的 lucene 库会从 segment 内的 storedfields 读取完整文档内容返回给协调节点
 - 协调节点最终将结果返回给客户端
- 怎么保证 kafka 消费的顺序
 - Kafka 的顺序保证是“分区级别的”
 - Kafka 只能保证同一分区 (Partition) 内消息的顺序。不同分区之间的消息是无法保证顺序的。
 - 原因: 一个 Topic 通常有多个 Partition; 每个 Partition 内消息按偏移量 (offset) 有序; 多个 Partition 并行消费, 因此整体上消息到达消费者的顺序可能乱序。
 - 在同一 Partition 内, Kafka 如何保证顺序?
 - 日志结构 (append-only log)
 - Kafka 的 Partition 是一个顺序写入的日志文件;
 - 每条消息都有自增的 offset;
 - 消费者按 offset 顺序拉取数据, 因此天然有序。
 - 单线程读写
 - Producer 向某个 Partition 发送消息时是顺序写;
 - Consumer 在一个 Partition 上消费时也是单线程拉取消息。
 - 如果业务要“全局顺序”怎么办?
 - Kafka 本身无法保证跨分区顺序, 但可以通过“合理分区策略”实现局部顺序或业务内顺序:
 - 按业务 key (如 userId) 做分区
 - Kafka 会将相同 key 的消息映射到同一个 Partition。
 - 同一用户的所有消息在同一 Partition 内, 自然顺序不乱
 - 消费者保持单线程消费同一 Partition
 - 每个 Partition 只能被一个消费者线程消费:
- Kafka 为什么快
 - 顺序写入日志
 - Kafka 不像数据库那样频繁随机写磁盘, 而是采用顺序写日志 (append-only log) 的方式。
 - 数据写入时只追加到文件末尾, 不做修改或随机插入。
 - 磁盘顺序写速度非常快 (接近内存的速度), 可达上百 MB/s
 - 页缓存 (Page Cache) + 零拷贝 (Zero Copy)
 - Kafka 充分利用了操作系统提供的 Page Cache (文件系统缓存), 并通过零拷贝机制提升性能:
 - Page Cache
 - Kafka 不自己缓存数据, 而是依赖 OS 文件系统缓存。
 - 数据写入文件后会留在 OS 内核缓存中 (dirty page), 读写都走缓存, 避免频繁磁盘 IO。
 - 零拷贝 (Zero Copy)
 - Kafka 使用 sendfile() 系统调用, 将文件直接从内核缓冲区发送到 Socket。
 - 避免了“内核态 ↔ 用户态”的多次数据拷贝和上下文切换。

- 批量发送与压缩 (Batch + Compression)
 - Producer 将多条消息合并成一个 batch 再发送。
 - Broker 落盘时也按 batch 写入。
 - Consumer 拉取时一次性取多个 batch。
 - 此外支持 GZIP、LZ4、ZSTD 等压缩算法，对整个 batch 压缩，大幅降低带宽和磁盘使用。
- 顺序读 + 零索引查找
 - Kafka 的消费是顺序读取日志文件（按 offset），无需复杂索引查找。
 - 每个 partition 是一个追加日志文件，消费者根据 offset 直接定位位置：
 - 消费者 offset 存储在独立的 topic（consumer_offsets）中，轻量又持久。
- 分区 (Partition) 并行 + 多副本复制
 - 每个 Topic 拆成多个 Partition；
 - 不同 Partition 可分布在不同 Broker 上；
 - Producer、Consumer 并发访问不同 Partition；
 - 副本 (replica) 异步复制，不阻塞主写流程。
- 拉取模型 (Pull) 代替推送 (Push)
 - Consumer 自己决定何时、拉多少数据；
 - 减少了 Broker 的推送压力；
- 简单的存储结构 (Log Segment + Index)
 - 每个 Partition 是一个 append-only 文件；
 - 通过 offset + index 文件快速定位；
 - 不需要复杂的锁竞争 (append-only，写锁简单)；
 - 旧数据只需按 segment 删除 (无需 GC)。
- 内部线程模型优化
 - I/O 线程负责网络收发；
 - 后台线程负责落盘和副本同步；
 - Reactor 模型 (Selector + poll) 高并发处理连接；
 - 少量线程即可支撑成千上万个连接。

二、传统文件 → Socket 的底层数据流动路径

假设文件在磁盘上，Socket 连接指向客户端。

整个过程可以分为 4 次拷贝，2 次上下文切换：

阶段	操作	拷贝方向	拷贝次数	用户/内核切换
①	内核读取文件数据	磁盘 → 内核空间缓冲区 (Page Cache)	1	进入内核态
②	read() 系统调用返回	内核缓冲区 → 用户空间缓冲区 (程序中的 byte[])	2	回到用户态
③	write() 系统调用	用户空间缓冲区 → 内核 Socket 缓冲区	3	再次进入内核态
④	网络发送	Socket 缓冲区 → 网卡驱动缓冲区 (DMA 到网卡)	4	回到用户态

🔧 四、Kafka 使用的零拷贝 sendfile() 流程

Kafka 通过调用 `sendfile()` 系统调用，让内核直接从文件缓冲区把数据“送”到网卡，不经过用户空间。流程如下：

阶段	拷贝方向	拷贝次数	用户/内核切换
①	磁盘 → 内核文件缓冲区 (Page Cache)	1	进入内核态
②	内核缓冲区直接 → 网卡缓冲区 (DMA)	2	回到用户态

✅ 总共：2 次拷贝 + 1 次上下文切换

而且这个过程完全不需要把数据搬到用户空间 (Kafka 的 JVM)，节省了大量 CPU 时间和内存带宽。

- KAFKA 如何实现死信队列
 - Kafka 本身不自动处理死信消息，需要应用层自己创建一个单独的 topic 作为死信队列
 - 消费者在消费主 Topic 时：若多次重试仍失败 → 将该消息发送到死信 Topic。
 - 和 rabbitmq 不同
 - RabbitMQ 有内置的死信交换机和队列机制，配置后自动转发死信消息。
 - Kafka 需要应用代码显式处理死信逻辑。
- 死信队列的作用
 - 避免失败消息被系统直接丢弃
 - 防止坏消息反复重试、阻塞正常消费
 - 可以看到失败消息的内容、时间、异常原因
 - 可以人工修复或系统定时重新推送
- 消息丢失，消息重复消费怎么解决
 - 消息丢失
 - 生产端
 - ACK
 - 重试机制（重试次数、间隔）
 - 持久化消息到数据库即本地消息表（事务消息）
 - Broker 端
 - 多副本机制（replication factor ≥ 2）
 - 开启持久化（RabbitMQ durable queue + persistent message；Kafka 本身通过磁盘日志持久化）
 - 定期备份
 - 消费端
 - 消费者手动 ACK（处理成功后再确认）
 - 消费位点持久化（offset commit）
 - 消费确认机制
 - 消息重复消费
 - 每条消息带唯一 ID（如订单号、事务 ID）消费者处理前检查数据库是否存在该消息 ID 的处理记录
 - 用 Redis SET 或 String 记录已处理消息 ID，SETNX messageId true EX 3600
 - 利用数据库幂等约束唯一索引
 - 专门的“消息消费表”记录 messageId 和状态

- 怎么在 mq 做订单业务的时候保证事务
 - 利用 MQ 自身的「半消息 (half message)」机制来实现分布式事务。
 - 缺点：
 - 并非所有 MQ 支持 (RabbitMQ 事务性能差, Kafka 要 2.5+ 版本)
 - 增加 MQ 管理复杂度 (事务回查)
 - 优点：
 - 原生支持事务, 消息与业务操作强一致
 - 不依赖外部数据库表
 - 支持回查机制 (MQ 会主动询问事务状态)
 - 本地消息表 (经典方案)
 - 比如订单业务写入数据库时, 同时写入一个消息表, 这样, 即使后续 MQ 宕机, 消息也不会丢失。
 - 定时任务 (或异步线程) 扫描消息表：
 - 读取未发送的消息
 - 调用 MQ 发送消息
 - 更新消息状态为已发送
 - 不断重试, 直到 MQ 发送成功为止。
 - 最终一致性 + 幂等消费 (适用于高并发)
 - 每条消息带唯一 msgId
 - 消费端保存 MSGid 的消费状态
 - 超时未成功处理的消息由补偿任务重发
- 消息队列堆积如何快速处理?
 - 分析堆积原因
 - 消费者挂了/宕机
 - 恢复消费者服务
 - 消费逻辑太慢
 - 优化消费逻辑
 - 消息积压分布不均
 - 增加分区或调整分配策略
 - 消费者线程数不足
 - 提高并发消费
 - 下游系统 (DB/接口) 慢
 - 异步写入或限流下游
 - 短期“应急”处理方案
 - 临时扩容消费者实例
 - 注意: Kafka 分区数决定最大并行度, 如果分区不够, 消费实例再多也无效。可考虑临时增加 partition 数量。
 - 提升消费并发度
 - 每个消费者实例内使用 线程池 并行处理消息
 - 批量拉取 N 条消息一次处理 (减少网络往返)。
 - 对部分非关键任务 (如日志、统计) 暂存到本地或缓存中, 延迟处理。
- 为何将“丢弃最老消息”作为消息队列满时的拒绝策略? 该策略适合什么场景? 哪些应用的 MQ 会侧重时效性?
 - 常见策略
 - 阻塞生产者
 - 丢弃新消息

- 丢弃最老消息
- 抛异常
- 为什么选择“丢弃最老消息 (Drop Oldest)”
 - 系统优先保证数据的时效性 (freshness) 而非完整性 (completeness)。
 - 优势
 - 维持系统实时性 (最新消息总能进入队列)
 - 避免队列阻塞上游系统 (不会阻塞生产者)
 - 可保持系统可用性与响应性
 - 劣势
 - 丢弃旧消息意味着数据不完整;
 - 不适用于必须处理“每条消息”的系统。
 - 适合“时效性优先”的场景
 - 股票/行情推送系统
 - 游戏状态
 - 传感器数据
 - 视频
 - 实时监控系统
- MQ 适合的场景有哪些? 在容量有限的场景下, 延迟消息和削峰填谷场景分别适合什么拒绝策略?
 - MQ 适合的典型场景
 - 系统解耦
 - 异步分流
 - 流量削峰
 - 延时/定时任务
 - 流式处理
 - 日志收集
 - 广播 / 分发
 - 延迟消息 (可靠性优先)
 - 阻塞生产者 (Block)
 - 重试入队
 - 抛异常 + 业务回退
 - 削峰填谷 (实时性优先)
 - 丢弃最老消息 (Drop Oldest)
 - 限流
 - 降级处理 (Fallback)
 - 丢弃最新消息 (让上游重试)
 - 视频转码、文件处理、订单清算。
 - Elasticsearch 构建倒排索引时, 文档和分词数量多导致内存占用大, 有哪些节省空间、提高性能的办法?
 - 减少分词数量 (源头减量)
 - 控制分词粒度 (使用合理分词器)
 - 中文分词器如 ik_max_word 会把句子切得非常碎, 产生指数级 term 数量。
 - 建议改为“analyzer”: “ik_smart”
 - 一般可减少 40% 60% 的 term 数量。
 - 对不搜索的字段禁用分词
 - 例如商品的 ID、类别、品牌代码、商户 ID、SKU 等, 只需精确匹配, 不用分词。

- 合并字段（减少冗余）
- 索引层优化（节省倒排索引空间）
 - 开启压缩算法
 - 减少 source 或存储字段
 - 如果 source 太大（商品描述、图片 URL 等），可以只保留必要字段
 - 合理使用 doc_values
- 索引过程与 segment 管理优化
 - 调整 segment merge 策略
 - 索引构建时，Lucene 会不断合并小 segment → 大 segment，非常耗内存和 CPU。
 - 调整 refresh 与 flush 策略
 - 默认 ES 每秒刷新一次内存 buffer 为 segment，产生大量小段。批量导入时可关闭
- 运行时内存与 JVM 调优
 - 增大堆外内存利用率
 - ES 大部分索引结构（倒排表、FST、term dictionary）在堆外（off-heap）管理。
 - 禁用 fielddata（或按需加载）
 - fielddata 是 ES 在内存中为 text 字段构建的倒排缓存，非常吃内存。
- jvm 调参常用参数有哪些

✅ 一、常见 JVM 调优参数分类总览

类型	作用	常见参数
内存分配	控制堆、栈、元空间大小	<code>-Xms</code> , <code>-Xmx</code> , <code>-Xmn</code> , <code>-XX:MetaspaceSize</code> , <code>-XX:MaxMetaspaceSize</code> , <code>-Xss</code>
垃圾回收器选择	选择合适的 GC 策略	<code>-XX:+UseG1GC</code> , <code>-XX:+UseParallelGC</code> , <code>-XX:+UseZGC</code> , <code>-XX:+UseShenandoahGC</code>
GC 行为调整	控制 GC 周期、停顿、并行度	<code>-XX:MaxGCPauseMillis</code> , <code>-XX:GCTimeRatio</code> , <code>-XX:ParallelGCThreads</code> , <code>-XX:ConcGCThreads</code>
日志与诊断	输出 GC、类加载、内存信息	<code>-Xlog:gc*</code> , <code>-XX:+HeapDumpOnOutOfMemoryError</code> , <code>-XX:HeapDumpPath</code> , <code>-XX:+PrintGCDetails</code> , <code>-XX:+PrintGCDateStamps</code>
性能与优化	JIT 编译、逃逸分析、偏向锁	<code>-XX:+TieredCompilation</code> , <code>-XX:+UseStringDeduplication</code> , <code>-XX:+DoEscapeAnalysis</code> , <code>-XX:+UseBiasedLocking</code>
容器友好	在 Docker/K8S 中限制内存与 CPU	<code>-XX:+UseContainerSupport</code> , <code>-XX:MaxRAMPercentage</code> , <code>-XX:InitialRAMPercentage</code> , <code>-XX:ActiveProcessorCount</code>



- xms 初始堆大小 如：-Xms512m 这是 VM 启动时分配的堆内存大小。通常与 -Xmx 设为相同，避免动态扩容。
- mxm 最大堆大小 如：-Xmx2g 这是 VM 可使用的最大堆内存大小。根据应用需求和服务器内存设置。限制 JVM 可用的最大堆内存，防止 OOM。
- xmn 年轻代大小 如：-Xmn256m 设置年轻代内存大小。年轻代越大，Minor GC 越少，但会占用更多堆空间。
- xss 线程栈大小 如：-Xss512k 设置每个线程的栈大小。栈太小可能导致 StackOverflowError，太大可以有更多线程空间但线程数受限。

- -XX:MetaspaceSize / -XX:MaxMetaspaceSize 元空间初始 / 最大大小 如： -XX:MaxMetaspaceSize=256m
 - -XX:+UseG1GC 启用 G1 垃圾收集器 适用于大内存应用，低延迟需求。
 - -XX:+UseConcMarkSweepGC 启用 CMS 垃圾收集器 适用于低延迟应用，但已被 G1 取代。
 - -XX:+UseParallelGC 启用并行垃圾收集器 适用于吞吐量优先的应用。
 - XX:+UseZGC 启用 ZGC 垃圾收集器 适用于超大内存应用，极低延迟需求（JDK 11+）。
 - XX:MaxGCPauseMillis 最大停顿时间目标 JVM 希望（但不保证） 每次垃圾回收导致的应用线程停顿时间 不超过这个目标值。
 - XX:GCTimeRatio GC 吞吐量目标 如： -XX:GCTimeRatio=9 表示 10% 时间用于 GC，90% 用于应用。
 - XX:+HeapDumpOnOutOfMemoryError OOM 时生成堆快照也就是堆转储文件，便于分析内存泄漏。
 - -Xlog:gc* GC 日志记录（JDK 9+） 如： -Xlog:gc*:file=gc.log:time,uptime,level,tags
- 一个实时聊天系统，一个文件上传系统，这两个系统 JVM 参数怎么设置
 - 实时聊天系统（低延迟、高并发）
 - 要求 GC 延迟极低
 - 一般是 CPU/线程调度密集型
 - 基础内存：固定内存，避免动态扩容导致停顿。
 - GC 算法： G1 适合低延迟，ZGC 适合极低延迟（JDK 11+）
 - 暂停时间目标： G1 的目标暂停时间（可根据业务调到 50ms）
 - 线程栈大小-Xss256k： 聊天系统线程多（例如 Netty），减小栈空间节约内存
 - 直接内存-XX:MaxDirectMemorySize=512m： Netty 使用直接内存；需根据连接数和 buffer 估算
 - GC 日志
 - 文件上传系统（I/O 密集、吞吐优先）
 - 要求 高吞吐、较大堆空间
 - 通常有异步线程池、缓存、临时文件缓冲
 - 基础内存-Xms2g -Xmx4g： 文件缓存较多，适度放大堆
 - GC 算法： Parallel 吞吐优先； G1 更平衡
 - GC 吞吐目标-XX:GCTimeRatio=9： 表示 10% 时间用于 GC（吞吐优先）
 - 直接内存-XX:MaxDirectMemorySize=1g： 文件传输常用 NIO DirectBuffer
 - 文件缓存优化： -Dio.netty.maxDirectMemory=1g： 若使用 Netty 传文件
 - 线程池优化： 降低过多上传线程，防止上下文切换浪费 CPU
 - GC 日志
- io 密集型和 cpu 密集型有什么区别
 - I/O 密集型
 - 主要瓶颈在输入/输出（I/O）操作上，比如 磁盘读写、网络请求、数据库访问。
 - CPU 大部分时间在等待数据（磁盘、网络）返回；
 - CPU 利用率低，而 I/O 子系统繁忙；
 - 并发数量多可以显著提高吞吐量（因为 I/O 等待时可切换任务）。
 - CPU 密集型
 - 主要瓶颈在处理器计算能力上，比如 大量数学运算、图像处理、视频编码等。
 - CPU 利用率高，几乎一直在工作
 - 并发数太多不会提升性能，反而增加上下文切换负担。
- IO 模型有哪些
 - 阻塞 I/O，阻塞等待数据。recvfrom() 一直阻塞直到数据就绪并复制完成

- 非阻塞 I/O，轮询检查是否有数据，有则读，无则立即返回 EWOULDBLOCK
- 信号驱动 I/O，数据到来时内核发信号通知用户进程，然后再读数据，用户调用 read() 获取数据
 - 用户进程先调用 fcntl(fd, F_SETOWN, getpid());fcntl(fd, F_SETFL, O_ASYNC);告诉内核：“这个 socket 可读时，请发一个 SIGIO 信号给我。”
 - 当内核检测到该 socket 上有数据到达时内核给用户进程发送一个 SIGIO 信号。
 - 用户进程的信号处理函数（signal handler）被触发，在 handler 里或之后，用户再调用 read(fd, buf, size) 去真正读数据。
 - 在接收到 SIGIO 信号之后，才会同步等待内核把数据拷贝到用户空间。在调用 fcntl(fd, F_SETOWN, getpid());fcntl(fd, F_SETFL, O_ASYNC)到 SIGIO 信号之间不等待
- 异步 I/O，用户发起 I/O 后，内核完成所有工作并回调通知
 - 用户发起 I/O 请求后：
 - 内核开始执行 I/O 操作。
 - 用户线程立即返回，可以继续干别的事。
 - 等内核把数据准备好、拷贝完后，再通过信号、回调或事件通知用户“搞定了”。
- I/O 多路复用
- 程序申请 100 字节的内存，操作系统是马上拿出 100 字节的内存吗？
 - malloc 只是向操作系统申请“虚拟内存地址空间”
 - 操作系统只是在页表中登记了这块地址空间属于该进程，但并没有真正分配物理内存页。
 - 真正的物理内存存在以下时机才会被分配：
 - 当程序第一次访问（读/写）那块虚拟地址空间时，CPU 触发缺页异常（page fault）；
 - 缺页异常
 - 如果页表项中发现该页不存在（没有分配物理页），或者在磁盘上（被换出去了），或者权限不匹配（只读页被写入）
 - 缺页异常处理的详细步骤
 - CPU 发现页表无效
 - 内核判断缺页原因

缺页类型	示例	处理方式
合法但未分配	malloc 后第一次访问	分配新物理页
在磁盘上（换出）	被 swap out	从 swap 读回
文件映射页	mmap 文件	从文件读入内存
非法访问	访问空指针、越界等	发送 SIGSEGV 杀进程

- 如果合法 → 分配/加载物理页
 - 恢复执行
- 操作系统捕获异常后，才会从物理内存（或交换区）中分配实际的页（通常 4KB 一页）；
- 然后更新页表，使这块虚拟地址指向真实的物理页。
- 操作系统以页（page）为最小分配单位（通常一页=4KB）。
- 所以实际上你“只用了 100 字节”，但系统给了你 4096 字节的物理内存。
- 物理内存和虚拟内存
 - 物理内存：
 - 计算机中真实存在的内存芯片（RAM，Random Access Memory）。

- 容量有限，由硬件决定。
- 速度快，是 CPU 可以直接访问的存储空间。
- 当程序运行时，它最终的数据和代码都需要加载到物理内存里才能被 CPU 执行。
- ▶ 虚拟内存 (Virtual Memory)
 - 操作系统通过软件手段，把 硬盘空间 和 物理内存 合成一个比物理内存大的“逻辑内存”给程序使用。
 - 每个进程都有自己独立的虚拟地址空间。
 - 地址转换由 内存管理单元 (MMU) 完成，把虚拟地址映射到物理地址。
 - 可以用硬盘（如交换分区/页面文件）扩展实际可用内存。
- 线程和进程的区别
 - ▶ 定义
 - 进程是是程序的一次执行实例，是资源分配的最小单位。
 - 线程是是进程中的一个执行流，是 CPU 调度的最小单位。
 - ▶ 资源占用
 - 进程有独立的内存空间、文件描述符、全局变量等资源。创建和销毁成本高。
 - 线程共享同一进程内的线程共享内存、文件句柄等资源。每个线程有自己的栈空间和程序计数器。创建、切换开销比进程小。
 - ▶ 调度和切换开销
 - 进程切换需要保存和恢复整个进程上下文（内存映射等），开销大。
 - 线程切换只需要保存线程上下文（寄存器、栈等），开销小，速度快。
 - ▶ 通信方式
 - 进程间通信
 - 管道
 - 套接字
 - 消息队列
 - 共享内存
 - 信号量
 - 线程间通信
 - 共享变量
 - 共享内存
 - 锁
 - ▶ 稳定性与安全性
 - 一个进程崩溃不会影响到其他进程。
 - 同一进程内一个线程挂掉会导致整个进程崩溃。
- 和协程的区别呢？
 - ▶ 协程是用户态的轻量级线程，由程序调度，内核不参与（用户态），没有独立内存空间，共享线程栈
 - ▶ 上下文切换成本最低，只切栈帧指针
 - ▶ 调度方式的切换方式是主动让出的，而进程和线程是抢占式的。
 - ▶ 为什么需要协程
 - 高并发下线程太慢、太贵、太多、太浪费
 - 协程不会阻塞线程（遇到 I/O 会让出）你让线程睡，CPU 也跟着睡了；你让协程睡，线程还在干活。
- 可以不用进程直接使用线程吗
 - ▶ 不可以

- 线程必须运行在进程之中，不能脱离进程单独存在。进程才是操作系统分配资源的基本单位，而线程只是依附在进程中的执行单位。
- 线程没有自己的独立资源空间
- 是进程创建线程，而不是系统直接创建线程
- 没有进程就没有地址空间，线程就没法执行
- 进程间通信的大概流程
 - 发送方进程准备数据（用户态）
 - 调用系统调用（syscall）进入内核，从用户态 → 内核态
 - 数据进入内核提供的 IPC 载体
 - 根据 IPC 类型不同
 - 进入内核缓冲区（管道、消息队列、套接字）
 - 或写入共享内存区域
 - 或写入内核同步结构（信号、信号量等）
 - 接收进程调用系统调用从内核读取
 - 数据从内核态复制回用户态内存
- 线程的安全怎么保证？
 - 互斥锁
 - 原子操作，只能应对单变量
 - volatile 保证可见性（但不保证原子性）
 - THREADLOCAL
 - 要保证线程安全，代码必须同时满足
 - 原子性
 - 安全性
 - 有序性
- 对于一个计算机，有 2GB 的存储空间，现在使用 malloc 函数每次申请 512MB 的空间，最多可以申请几次？
 - 理论上是 4 次，但实际会有额外的开销损耗，最多申请 3 次，第 4 次回报错
 - malloc() 分配的空间中，不仅包括用户申请的内存，还包含额外的管理开销（metadata）。
 - 这些开销主要包括：
 - 内存分配器保存块信息（如大小、状态）的头部信息；
 - 内存页对齐（alignment）；
 - 操作系统页级分配粒度（通常为 4KB）；
 - 有时还包括堆起始地址的对齐损耗。
 - 每次 malloc(512MB)，实际上系统分配的可能是：
 - 512MB + 结构体开销 + 对齐开销 ≈ 512MB + 几 KB。
- 假如我现在要从磁盘上去读一个文件，读完之后对它进行修改，然后再写回去。这个过程操作系统的内核它会做什么事情？
- 从 Redis 视角，接收“get key”请求时，网络及操作系统层面的处理过程是怎样的？
- 怎么理解 cap
 - C – Consistency（一致性）
 - 所有节点在同一时间看到的数据都是一样的。
 - A – Availability（可用性）
 - 每个请求都能得到一个响应（不管这个响应是不是最新的）。
 - P – Partition tolerance（分区容错性）
 - 系统能容忍网络分区，即部分节点之间无法通信，但系统整体仍能运行。

- CAP 定理核心
 - 在一个分布式系统中，你不可能同时完美满足 C、A、P 三者。最多只能选其中两个。
 - CA（一致性 + 可用性）
 - 放弃 P → 系统在网络分区时不可用。
 - 适用场景：单机或内部网络稳定的系统。
 - CP（一致性 + 分区容错性）
 - 放弃 A → 系统在网络分区时可能变得不可用。
 - 放弃 A → 系统在网络分区时可能变得不可用。
 - AP（可用性 + 分区容错性）
 - 放弃 C → 系统可能返回旧数据，但总能响应请求。
 - 适用场景：社交网络、缓存系统、购物车等允许一定延迟的场景。
- 让你去做一个分布式的数据库，你应该怎么去考虑 CP 的设计？
 - 选用一致性协议
 - Paxos / Raft：在写入数据时，确保大多数节点达成共识后才返回成功。
 - 用途：保证即使部分节点故障或网络延迟，系统依然不会出现冲突数据。
 - 分区容错设计（P）
 - 分布式架构
 - 多节点部署，支持节点之间的消息重试与网络断开恢复。
 - Leader 节点机制
 - 为每个数据分片选一个 leader，保证写入通过 leader 进行一致性控制。
 - 当网络分区发生时，如果 leader 与大多数节点断开连接，leader 会下线以避免分裂脑（split-brain）。
- 支付里面有个非常经典的场景——转账。用户的钱减了，商户的钱一定要加。这里要做到强一致性，你觉得应该怎么做？
 - 单库事务
 - 在微服务或分库场景下不可用。
 - 事务原子性天然保证，简单可靠。
 - 分布式事务
 - 两阶段提交
 - 准备阶段：各方事务先执行 prepare，锁住资源但不提交。
 - 提交阶段：协调者决定提交或回滚。
 - 性能慢（同步阻塞）分布式锁定资源时间长
 - 基于消息的可靠事务（TCC / Saga）
 - TCC
 - Try：预扣用户余额，预加商户金额
 - Confirm：最终提交
 - Cancel：回滚操作
 - Saga
 - 每一步执行本地事务，失败则调用补偿事务回滚
 - 异步执行，延迟最终一致性，比较适合电商支付类场景
 - 协调者驱动（Orchestration）
 - 用户发起支付请求 → 请求到支付服务
 - 支付服务把这笔业务交给 Saga 协调者（可以是内置的服务，或者消息队列）
 - 协调者发消息让用户账户扣钱
 - 协调者发消息让商户账户收钱
 - 如果任意一步失败，协调者调用前面已成功步骤的补偿事务
 - 为什么需要补偿事务

- 在分布式事务中，一个大事务被拆成多个小的本地事务按顺序执行：如果中间某一步失败，前面的事务已经提交了，系统就处于“半完成状态”，需要回滚。不能直接用数据库回滚（因为本地事务已提交），所以需要设计一个补偿事务来撤销之前的操作
- 补偿事务的特性
 - 与原事务逻辑相反
 - 幂等性
 - 局部执行
 - 只撤销已经完成的本地事务，不影响还未执行的事务。
 - 可延迟执行
 - 可以异步执行补偿事务，只要最终一致性得到保证。
- 事件驱动（Choreography）
 - 用户账户服务扣钱成功 → 发布事件 UserDebited
 - 商户账户服务订阅事件 → 收钱成功 → 发布事件 MerchantCredited
 - 如果收钱失败 → 发布事件触发用户账户服务回滚
- 可靠消息 + 异步最终一致性
 - 先在本地数据库扣钱，生成消息（Event / MQ）
 - 消息发送给商户服务，商户服务收到后加钱
 - 如果消息发送失败，重试机制保证最终一致性
 - 高可用，但不是强一致性
- 行业实践
 - 核心账户系统在单库或强事务数据库中，保证扣钱和加钱的核心操作在同一事务
 - 对于跨库或跨服务场景，用 TCC 或 Saga 模式处理。
 - 幂等性设计：防止消息重复执行导致多扣。
 - 可靠消息：保证最终一致性，避免资金丢失。